

**NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA BUCHAREST
FACULTY OF ENGINEERING IN FOREIGN LANGUAGES
ELECTRONIC ENGINEERING, TELECOMMUNICATIONS
AND INFORMATION TECHNOLOGIES
APPLIED ELECTRONICS**

DIPLOMA PROJECT

Project coordinator

Prof. Paul GAGNIUC

Student:

Vlad-Andrei PARASCHIV

**Bucharest
2025-2026**



**NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA BUCHAREST
FACULTY OF ENGINEERING IN FOREIGN LANGUAGES
ELECTRONIC ENGINEERING, TELECOMMUNICATIONS AND
INFORMATION TECHNOLOGIES - APPLIED ELECTRONICS**



Machine Learning Powered Antivirus with Image-Based Signatures

Project coordinator

Prof. Paul GAGNIUC

Student:

Vlad-Andrei PARASCHIV

**Bucharest
2025-2026**

Academic Honesty Statement

I, Vlad-Andrei PARASCHIV, hereby declare that the work with the title Machine Learning Powered Antivirus with Image-Based Signatures, to be openly defended in front of the diploma theses examination commission at the Faculty of Engineering in Foreign Languages, National University of Science and Technology "Politehnica" of Bucharest, as partial requirement for obtaining the title of Engineer is the result of my own work, based on my work.

The thesis, simulations, experiments and measurements that are presented are made entirely by me under the guidance of the scientific adviser, without the implication of persons that are not cited by name and contribution in the Acknowledgements part.

The thesis has never been presented to a higher education institution or research board in the country or abroad.

All the information used, including the Internet, is obtained from sources that were cited and indicated in the notes and in the bibliography, according to ethical standards. I understand that plagiarism is an offense and is punishable under law.

The results from the simulations, experiments and measurements are genuine. I understand that the falsification of data and results constitutes fraud and is punished according to regulations.

Vlad-Andrei PARASCHIV

Abstract

Malware continues to grow in scale and sophistication, while traditional signature-based antivirus systems struggle to detect zero-day, packed, and polymorphic threats in a timely and reliable manner. Signature-based engines inherently depend on prior knowledge of specific malware samples and require continuous database updates, which leaves a window of exposure during which novel variants can propagate without being recognized. Polymorphic and metamorphic malware further complicate detection by automatically mutating their code structure while preserving malicious behavior, generating large numbers of variants that evade exact signature matching. Industry analyses highlight that these evasive techniques now characterize the vast majority of modern malware families.

In response, visualization and machine-learning-based approaches have emerged that convert executables into images and apply pattern recognition to distinguish malicious from benign software. In this project, a Machine Learning Powered Antivirus system is proposed that performs static analysis of Portable Executable files, computes section-level entropy and related information-theoretic metrics, and encodes these features into Red-Green-Blue color matrices serving as image-like signatures. A Random Forest classifier is trained on these representations to classify files as malware or benign, enabling detection without dynamic execution or sandboxing.

The main contributions of this work are:

- (1) A hybrid feature-extraction pipeline that combines PE structural information with entropy-based visualization to produce robust image signatures.
- (2) A detailed study showing how Random Forests can serve as a computationally efficient and interpretable alternative to deep convolutional neural networks for image-based malware detection,
- (3) A prototype antivirus component capable of taking user files as input, generating color matrices on the moment and output classification decisions suitable for deployment on endpoint devices.

Keywords: PE files, entropy-based features, Random Forest classifier, malware detection, malware categories, machine learning antivirus.

LIST OF ABBREVIATIONS

Abbreviation	Definition
AV	Antivirus
PE	Portable Executable
CNN	Convolutional Neural Network
ML	Machine Learning
AI	Artificial Intelligence
EDR	Endpoint Detection and Response
RGB	Red-Green-Blue
GPU	Graphics Processing Unit
IoT	Internet of Things
CIA	Confidentiality, Integrity, Availability
DoS	Denial-of-Service
RATs	Remote Access Trojans
API	Application Programing Interface
DOS	Disk Operating System
NT	New Technology
DLL	Dynamic Link Library
SVMs	Support Vector Machines
k-NN	k-Nearest Neighbors
ANNs	Artificial Neural Networks
MLP	Multilayer Perceptron
ROC-AUC	Receiver Operating Characteristics – Area Under the Curve
NGAV	Next-Generation Antivirus
AML	Advanced Machine Learning
RAT	Remote Access Trojan
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve

LIST OF FIGURES

Figure 1. PE File Structure	17
Figure 2. Entropy-to-RGB Color Mapping	24
Figure 3. Full Project Workflow	38
Figure 4. Benign Entropy Matrix	39
Figure 5. Malware Entropy Matrix.....	39
Figure 6. Main Application Window	44
Figure 7. Malware File Scanned	46
Figure 8. Benign File Scanned	46
Figure 9. Binary Test Confusion Matrix.....	47
Figure 10. Binary Validation Confusion Matrix.....	47
Figure 11. Binary Test Normalized Confusion Matrix	48
Figure 12. Binary Validation Normalized Confusion Matrix	48
Figure 13. Binary Test ROC-AUC Curve	48
Figure 14. Binary Validation ROC-AUC Curve	48
Figure 15. Binary Test Precision-Recall Curve.....	49
Figure 16. Binary Validation Precision-Recall Curve	49
Figure 17. Binary Test Precision/Recall/F1-Score	49
Figure 18. Binary Validation Precision/Recall/F1-Score	49
Figure 19. Binary Test Probability Distribution.....	50
Figure 20. Binary Validation Probability Distribution	50
Figure 21. Multi-Class Test Confusion Matrix	51
Figure 22. Multi-Class Validation Confusion Matrix.....	51
Figure 23. Multi-Class Test Precision/Recall/F1-score	52
Figure 24. Multi-Class Validation Precision/Recall/F1-score.....	52
Figure 25. Multi-Class Test Normalized Confusion Matrix	52
Figure 26. Multi-Class Validation Normalized Confusion Matrix.....	52
Figure 27. Binary Top Feature.....	54
Figure 28. Multi-Class Top Feature.....	54

LIST OF TABLES

Table 1. Comparison of Malware Datasets	10
Table 2. Malware Category Comparison	12
Table 3. Detection Method Comparison	14
Table 4. Dataset Organization.....	37
Table 5. Test Precision, Recall, F1-Score	49
Table 6. Binary Validation Precision, Recall, F1-Score.....	50
Table 7. Multi-Class Test Precision, Recall, F1-score	52
Table 8. Multi-Class Precision, Recall, F1-Score	53

Contents

LIST OF FIGURES.....	
LIST OF TABLES.....	
1. Introduction	1
1.1. Background and Motivation.....	1
1.2. Machine Learning and Image-Based Malware Detection.....	1
1.3. Research Gap and Problem Statement	2
1.4. Research Objectives and Contributions	3
1.5. Rationale and Significance.....	4
1.6. Research Questions and Hypotheses.....	4
1.7. Methodology Overview.....	5
1.8. Scope and Limitations	5
1.9. Structure of the Paper	6
2. State of the Art.....	8
2.1. Static Analysis of PE Files Using Machine Learning	8
2.2. Entropy and Information-Theoretic Methods in Malware Detection.....	8
2.3. Binary Visualization Techniques	8
2.4. Image-Based Malware Classification.....	9
2.5. Malware Datasets and Benchmarks	9
2.6. Adversarial Attacks on Machine Learning Based Antivirus Systems.....	10
2.7. Existing Antivirus Architectures Incorporating Machine Learning	11
3. Fundamentals of Cybersecurity and Malware Detection	12
3.1. Fundamental principles of information security	12
3.2. Malware Classification and General Characteristics	12
3.2.1. Viruses	13
3.2.2. Worms	13
3.2.3. Trojans.....	13
3.2.4. Ransomware	13
3.2.5. Rootkits and spyware	14
3.3. Traditional Malware Detection Methods	14
3.3.1. Signature-Based Detection.....	14
3.3.2. Heuristic Detection.....	15

3.3.3. Behavioral Analysis.....	15
3.4. Limitations of Classical Antivirus Systems	16
4. Representation of Malware Samples	17
4.1. Analysis of Executable Files	17
4.2. Representation of Binary Data for Automated Analysis	18
4.3. Transforming Binary Files into Visual Representations	19
4.4. Image-Based Signature for Malware Classification.....	19
4.5. Advantages and Limitations of Visual Representations.....	20
5. Fundamentals of Digital Image Processing.....	22
5.1. Representation of Digital Images	22
5.2. Visual Features and Feature Extraction.....	22
5.3. Image Preprocessing	23
5.3.1. Resizing and Spatial Consistency	24
5.3.2. Data Augmentation.....	24
5.3.3. Class Balancing	24
5.4. Image Classification and Pattern Recognition	25
5.4.1. Classical Machine Learning Approaches	25
5.4.2. Random Forest for Image Classification.....	25
5.4.3. Deep Learning Approaches	25
5.4.4. Evaluation Metrics for image Classification.....	25
5.4.5. Cross-Validation and Generalization.....	26
6. Machine Learning and Neural Network Based Malware Detection	27
6.1. Fundamentals of Machine Learning.....	27
6.2. Classical Machine Learning for Malware Detection.....	27
6.2.1. Random Forest	28
6.3. Artificial Neural Networks Fundamentals	28
6.3.1. The Artificial Neuron Model.....	28
6.3.2. Multilayer Perceptron.....	29
6.3.3. Convolutional Neural Networks.....	29
6.3.4. Deep Neural Networks	29
6.4. Training and Optimization	30
6.5. Neural Network-Based Malware Detection	30

6.5.1. Malware Classification Using Neural Networks.....	30
6.5.2. Detection Models Based on Visual Representations.....	31
6.6. The Classification Pipeline.....	31
6.7. Performance Evaluation Metrics.....	31
6.8. Challenges and Current Research Directions.....	32
7. Modern Antivirus Systems and Intelligent Architectures	33
7.1. Architecture of an antivirus system.....	33
7.2. Integration of machine learning models into antivirus software.....	34
7.3. Adaptive Detection and Response Systems	34
7.4. Limitations and security considerations.....	35
8. Dataset Construction and Preparation	36
8.1. Dataset Design Acquisition	36
8.2. Malware Sample Acquisition	36
8.3. Benign Sample Acquisition.....	36
8.4. Dataset Organization	37
8.5. Dataset Preparation Methodology	38
9. Feature Engineering and Visual Representation	39
9.1. Entropy matrix generation.....	39
9.2. Additional Static Features	40
9.3. Combined 199-Dimensional Representation	40
9.4. Feature Alignment and Data Integrity	40
10. Model Training and Evaluation.....	42
10.1. Binary Classification Objective	42
10.1.1. Binary Model Configuration	42
10.2. Multi-Class Classification Objective	42
10.2.1. Multi-Class Model Configuration	42
10.3. Evaluation Artifacts Generated During Training	43
11. Application Design and Architecture	44
11.1. Application Purpose and Design Goals.....	44
11.2. User Interface Structure	44
11.3. Training-Aligned Inference Pipeline.....	45
11.3.1. Use of Cached Artifacts.....	45

11.4.	Prediction Workflow and Result Presentation.....	45
11.5.	Practical Characteristics of the Prototype	46
12.	Experimental Results and Interpretation	47
12.1.	Binary Classification Results	47
12.2.	Binary Metrics and Threshold Behavior	47
12.3.	Multi-Class Classification Results	50
12.4.	Per-Class Interpretation.....	51
12.5.	Feature Importance Analysis	53
12.6.	Discussion of Observed performance	54
13.	Limitations, Practical Reflections and Future Improvements	56
13.1.	Dataset-Related Limitations	56
13.2.	Representation-Related Limitations.....	56
13.3.	Model-Related Limitations	57
13.4.	Application-Related Limitations	57
13.5.	Future Improvements	57
14.	Conclusions	59
	Bibliography.....	61
	Appendix 1 – File Collectors	67
1.1.	Benign Files (benign.py)	67
1.2.	Malware Collector (dataset.py)	68
	Appendix 2 – Features Extractors	72
2.1.	Entropy Calculator (Entropy_calc.py)	72
	Appendix 3 – Random Forest Models.....	77
3.1.	Malware Versus Benign Model (MvsB_feat_plots.py).....	77
3.2.	Multi-Class Model (rf_train_feat_plots.py)	84
	Appendix 4 – Application (MalwareScanner.py).....	91

1. Introduction

1.1. Background and Motivation

MODERN cyber threats are dominated by increasingly sophisticated malware targeting individuals, enterprises, and critical infrastructure, often through ransomware, data-stealing trojans, remote access trojans (RATs) and network-propagating worms. Traditional Antivirus (AV) solutions, which have been the foundational layer of endpoint security for decades, rely primarily on signature-based detection: files are scanned and compared against a database of known malicious patterns. While this approach is effective against well-cataloged threats, it has a big problem: a new malware sample must first be discovered, analyzed, and signed before other systems can be protected.

This reactive model creates a vulnerability window in which zero-day attacks and previously unseen variants can spread undetected. At the same time, malware authors employ polymorphic and metamorphic techniques that automatically vary code layout and byte patterns while retaining the same malicious functionality, increasing the difficulty of maintaining comprehensive signatures. Packed and encrypted binaries further obfuscate the underlying code; such samples exhibit high entropy profiles that defeat simple pattern matching. These factors, combined with the growth of remote work, bring-your-own-device environments, and resource-constrained endpoints, motivate the need for detection mechanisms that generalize beyond exact byte-level signatures and can recognize structural and statistical regularities in malicious software.

Beyond technical limitations, traditional AV solutions impose operational burdens: they require continuous internet connectivity for database updates, consume system resources through frequent deep scans and struggle with obfuscated or encrypted payloads. These constraints are particularly problematic for disconnected endpoints, embedded systems and lightweight user devices, all of which represent a growing share of the attack surface. In this context, lightweight machine learning based detectors that rely on compact static representations and can operate offline become more attractive as components to existing AV engines.

1.2. Machine Learning and Image-Based Malware Detection

Machine learning (ML) and artificial intelligence (AI) have been widely adopted in next-generation AV and endpoint detection and response (EDR) products because they can learn complex patterns of maliciousness directly from data instead of relying exclusively on handcrafted rules. ML-based malware detectors are trained on labeled datasets comprising both benign and malicious samples, and they learn to approximate a decision boundary based on extracted features such as byte n-grams, imported API calls, or structural properties. As a result, they can identify previously unseen malware that shares latent patterns with known threats, including zero-day and polymorphic variants. [1], [2]

Within this broader trend, visualization- and image-based techniques have emerged as a particularly promising class of approaches. In such methods, executables or derived features are converted into images typically by mapping byte values or feature vectors to pixel intensities and then processed with image classification algorithms [1], [3]. Abdullah et al. proposed an enhancement for image-based malware classification using normalized low-dimensional images and conventional ML algorithms, achieving high accuracy with reduced computational overhead. Naeem et al. applied grayscale visualization and deep neural

networks to malware binaries and observed that samples from the same family often yield visually similar textures and structures, enabling effective classification [4]. Han et al. introduced a method that employs both visualized images and entropy graphs to classify malware families, demonstrating that entropy-based visual signatures effectively capture structural similarities among variants [4]. A follow-up study by the same group introduced RGB image matrices constructed from binary code sequences, achieving robust classification performance [5].

Systematic reviews confirm that deep learning models, especially Convolutional Neural Network (CNN) architectures such as ResNet, VGG, and EfficientNet, trained on malware images can achieve accuracies in the 95 - 99% range on publicly available benchmark datasets like Maling [6], [7], [8]. However, these deep models typically demand substantial computational resources, GPU acceleration, and careful hyperparameter tuning, which can limit deployment on low-power devices or lightweight endpoint agents [6], [7], [9]. This creates an opportunity to investigate alternative classifiers, such as Random Forests, that can exploit engineered image-based features while offering better efficiency and interpretability [2], [10], [11]. In this thesis, this idea is materialized by combining a compact entropy-based image representation derived from the first 16 KB of a file with a small set of handcrafted static features, and training Random Forest models to perform both binary malware-versus-benign discrimination and a deeper family-level classification.

1.3. Research Gap and Problem Statement

Despite promising results, current image-based malware detection research shows several notable gaps that limit both practical deployment and theoretical understanding.

First, many approaches treat the binary as a flat byte stream and convert it directly to a grayscale image, ignoring the rich structural information encoded in the PE format. Portable Executables are composed of multiple sections such as .exe (executable code), .data (initialized data), .rdata (read-only data), .idata (import tables), .reloc (relocations), and .rsrc (resources) that serve distinct purposes and exhibit different statistical characteristics. Recent work on malware visualization that segments images according to PE sections suggests that section-aware processing can improve classification by highlighting functionally meaningful regions. However, systematic integration of explicit PE parsing, section-level feature extraction, and visualization remains underexplored. In practice, many systems either ignore PE structure entirely or only incorporate it via a limited set of header-derived scalar features. [12]

Second, entropy analysis is a long-established tool for detecting packed or encrypted malware, as such samples tend to exhibit higher entropy than typical uncompressed executable code. Lyda and Hamrock showed that analyzing the entropy distribution across PE sections can quickly flag suspicious files and regions, providing a lightweight yet effective heuristic for packed or encrypted content [13]. Yet in many image-based systems, entropy appears only as a global statistic or separate diagnostic plot, rather than being directly embedded into the image representation itself [3]. Integrating section-level entropy and related information-theoretic metrics directly into the RGB channels of image-based signatures has the potential to highlight suspicious areas in a form that is both machine and analyst-friendly, but this design space has not been thoroughly explored.

Third, much of the highest reported accuracy in image-based malware detection comes from deep CNNs trained on large datasets, which, while powerful, are computationally expensive and often opaque to human interpretation [6], [7], [9]. Random Forests, in contrast, have shown strong performance on engineered feature sets for malware detection and offer advantages including robustness to overfitting, ease of training, and built-in feature-importance measures [2], [10], [11]. However, their application specifically to structured, entropy-

based image signatures derived from PE sections combined with simple static descriptors has not been adequately investigated.

Finally, there exists a gap between academic research achieving high accuracy on curated benchmark datasets and practical deployment in real-world antivirus systems that must balance detection performance with computational efficiency, user experience, and operational constraints. Many studies stop at offline evaluation and do not demonstrate how their models would operate within a real scanning workflow or run on resource constrained endpoints.

These gaps lead to the central problem addressed in this thesis: how to construct entropy-aware image-based signatures that, together with a small set of handcrafted static features, support accurate, efficient, and interpretable malware/benign classification using a Random Forest classifier, in a manner suitable for real-world endpoint deployment.

1.4. Research Objectives and Contributions

To address the identified gaps, this thesis pursues the following objectives:

1. Feature design: To design and implement a section and entropy-aware feature extraction pipeline that reads the first 16 KB of PE files, extracts section-level information, computes Shannon entropy and related metrics per section or chunk, and encodes these features into RGB color matrices representing the structural and statistical profile of each file.

2. Classifier Evaluation: To evaluate the performance of a Random Forest classifier trained on the resulting feature set for binary malware/benign classification and multi-class family prediction, comparing its accuracy, precision, recall, and computational efficiency against expectations from CNN-based approaches reported in the literature.

3. Prototype implementation: To develop a functional prototype antivirus system that integrates the feature-extraction pipeline and trained models into an end-to-end workflow capable of accepting files from a user's system, generating color matrices on the fly, and returning classification decisions with confidence scores in near real time.

4. Interpretability analysis: To analyze feature importance in the trained Random Forest models in order to determine whether the classifier relies more heavily on localized entropy-image cells or on higher-level statistical summaries, therefore providing insight into which structural characteristics most reliably differentiate malicious from benign files.

The main contributions of this thesis are:

- Methodological contribution: A new visualization-based signature that fuses entropy-based visualization of the first 16 KB of an executable with a small set of handcrafted entropy statistics, producing interpretable RGB color matrices and scalar features that emphasize suspicious regions and patterns while remaining lightweight enough for desktop deployment.

- Algorithmic contribution: An empirical assessment of Random Forests for image-based malware detection, demonstrating that suitably engineered entropy-derived signatures can deliver competitive accuracy relative to deep CNNs while significantly reducing computational demands.

- Practical contribution: A working prototype antivirus-like component that shows how the proposed technique can be integrated into real-world scanning workflows as a complementary detection engine alongside traditional signature-based AV and next-generation EDR systems.

- Interpretability contribution: Feature-importance analyses that clarify whether the classifier's decisions are driven primarily by global entropy statistics or by specific positions in the entropy matrix, offering practical insight into the structural statistical properties of PE binaries that influence malware-versus-benign deci

1.5. Rationale and Significance

The rationale for this work is rooted in the ongoing shift from reactive, signature-centric defenses toward proactive, data-driven, and ML-enabled security controls. As attackers increasingly automate the generation of polymorphic variants and leverage AI-assisted tooling, defenders must adopt techniques that generalize beyond exact byte patterns and exploit deeper structural and statistical regularities in malicious code. Visualization-based approaches are attractive because they transform complex binary structures into images amenable to both human analysis and powerful classification algorithms [1], [4].

The proposed method provides several strategic advantages:

- Evasion resistance: By focusing on information-theoretic properties, such as local entropy within the first 16 KB, rather than exact byte sequences, the system is more resilient to many forms of code obfuscation, packing and polymorphic transformation that preserve high-level statistical characteristics.[13].

- Computational efficiency: Random Forest classifiers typically require less computational power and training time than deep CNNs, making them attractive for deployment on devices without GPUs and in scenarios where frequent model updates are necessary.

- Interpretability: Random Forest models provide feature-importance measures that reveal which PE sections and entropy-related features drive classification decisions, aiding in incident response and forensic analysis [2], [10].

- Proactive detection: By treating malware detection as a pattern recognition problem over structural and statistical features, the system can, in principle, detect previously unseen malware families that share properties with known malicious code, complementing reactive signature-based defenses [1].

Given the continued growth of remote work, heterogeneous endpoints and IoT ecosystems, efficient and interpretable malware-detection techniques that can operate close to the edge are of significant practical importance. This thesis contributes to that goal by exploring a design space that combines information theory, binary structure, visualization, and classical ML in a coherent and deployable framework.

1.6. Research Questions and Hypotheses

This thesis is guided by the following research questions:

- RQ1: Do entropy-based features derived from the first 16KB of PE files provide sufficiently discriminative information to distinguish malware from benign executables with high accuracy?

- RQ2: How does a Random Forest classifier trained on hybrid entropy/image signatures compare, in terms of accuracy and computational efficiency, to CNN-based image classifiers reported in the literature for similar malware-detection tasks?

- RQ3: Within the proposed 199-dimensional feature space, which features contribute most to classification performance, as measured by model feature-importance scores?

- RQ4: Can the end-to-end pipeline, from file input through entropy-based feature extraction to binary and multi-class classification, be implemented with latency and resource usage compatible with average desktop endpoint deployment?

From these questions, the following hypotheses are formulated:

- H1: Malware and benign executables exhibit statistically significant differences in entropy distributions within the first 16 KB, but when mapped into RGB color matrices and augmented with other entropy statistics, these differences enable ML-based classification with accuracy exceeding 90%.
- H2: A Random Forest classifier trained on the proposed entropy-centered hybrid signatures can achieve accuracy of at least 95%, competitive with CNN-based approaches reported in the literature, while requiring substantially less computational resources.
- H3: Features capturing global entropy behavior and entropy-image cells corresponding to the earliest bytes of the file will, on average, contribute more to classification accuracy than cells from later positions, reflecting the concentration of structural and packing information near the file header and early code region.

1.7. Methodology Overview

The research adopts a quantitative experimental methodology composed of the following phases:

Phase 1: Dataset acquisition and labeling. Malware samples are collected from the MalwareBazaar repository and grouped into four families: ransomware, trojan, remote access trojan and worm, based on tag metadata and aliases, while benign PE samples are collected from standard Windows system and program directories to form balanced or appropriately weighted datasets. The resulting datasets forms the basis for both binary (malicious versus benign) and five-class (four families plus benign) classification experiments.

Phase 2: Entropy computation and feature extraction. For each sample, the first 16.384 bytes are read and partitioned into 64 fixed-size chunks of 256 bytes. Shannon entropy is computed for each chunk, and a set of global descriptors, such as overall window entropy, mean and standard deviation of chunk entropies, minimum and maximum entropy, and the fraction of chunks exceeding a high entropy threshold, are derived.

Phase 3: The 64 per-chunk entropy values are normalized and mapped into an 8×8 grid, with each cell's entropy encoded into an RGB triplet using a green-to-red color mapping. The resulting 8×8×3 matrix is flattened into 192 image features, which are concatenated with the 7 global descriptors to form a 199-dimensional feature vector per file.

Phase 4: Model training and validation. Two Random Forest models are trained: a binary classifier for malware-versus-benign prediction and a multi-class classifier for family-level prediction. Both are evaluated using test sets and standard metrics (accuracy, precision, recall, F1-score, ROC–AUC), alongside confusion matrices and probability distributions.

Phase 5: Prototype implementation. The trained models and feature-extraction pipeline are integrated into a desktop application that allows users to select files, generates entropy matrices on demand, and returns classification decisions with associated confidence scores. The application also visualizes the entropy matrix to provide transparency.

Phase 6: Experimental results are analyzed to assess overall performance, error patterns, and feature importance. The findings are compared against expectations from related work, and the strengths, limitations, and potential extensions of the approach are discussed.

1.8. Scope and Limitations

This research focuses specifically on static analysis of Windows PE-format executables. Dynamic behaviors observable only at runtime (e.g., process injection, network communication) are outside the direct reach of the proposed method, though entropy and structural features may still indirectly reflect some obfuscation

strategies. The system does not target other executable formats such as Linux ELF or Android APK, though the general approach could be adapted in future work. While entropy and structure-based features are robust to many forms of obfuscation, adversaries could theoretically craft binaries with benign-like entropy profiles, which may degrade detection effectiveness [3].

The experimental evaluation relies on a dataset constructed from MalwareBazaar tags and a locally collected benign set. Although this reflects realistic collection conditions, it may not fully represent emerging threats or highly targeted attacks. In practice, continuous model retraining and dataset updates would be necessary to maintain performance over time, an operational consideration acknowledged but not fully addressed within the scope of this thesis.

1.9. Structure of the Paper

This paper is organized into fourteen chapters, followed by a list of references.

Chapter 1 introduces the background, motivation, research gap, objectives, rationale, research questions, methodology, and scope of the work, situating the thesis within the broader shift toward machine-learning and image-based malware detection.

Chapter 2 presents the state of the art, reviewing static analysis of PE files with machine learning, entropy and information-theoretic methods in malware detection, binary visualization techniques, image-based malware classification, commonly used datasets and benchmarks, adversarial attacks on learning-based antivirus systems, and existing antivirus architectures that integrate machine learning.

Chapter 3 covers fundamental concepts in cybersecurity and malware detection, including core principles of information security, malware classification and general characteristics (viruses, worms, trojans, ransomware, rootkits, and spyware), traditional detection methods (signature-based, heuristic, and behavioral analysis), and the limitations of classical antivirus systems.

Chapter 4 focuses on the representation of malware samples. It discusses the analysis of executable files, ways to represent binary data for automated analysis, the transformation of binaries into visual representations, the proposed image-based signature for malware classification, and the advantages and limitations of visual representations.

Chapter 5 introduces fundamentals of digital image processing. It describes digital image representation, visual features and feature extraction, image preprocessing steps (including resizing, data augmentation, and class balancing), and image classification and pattern-recognition techniques, with subsections on classical methods, Random Forest for image classification, deep learning approaches, and evaluation metrics and cross-validation.

Chapter 6 presents machine learning and neural network-based malware detection. It reviews general machine learning concepts, classical ML methods for malware detection (with a focus on Random Forest), artificial neural network fundamentals (including MLPs, CNNs, and deep networks), training and optimization, malware classification using neural networks, detection models based on visual representations, the overall classification pipeline, performance evaluation metrics, and current challenges and research directions.

Chapter 7 examines modern antivirus systems and intelligent architectures. It describes the architecture of an antivirus system, how machine learning models are integrated into antivirus software, adaptive detection and response mechanisms, and limitations and security considerations of such systems.

Chapter 8 details dataset construction and preparation. It covers the dataset design, malware sample acquisition, benign sample acquisition, dataset organization, and the methodology used to prepare the data for subsequent analysis.

Chapter 9 describes feature engineering and visual representation. It explains how entropy matrices are generated, how additional static features are derived, how the combined 199-dimensional representation is built, and how feature alignment and data integrity are maintained throughout the pipeline.

Chapter 10 focuses on model training and evaluation. It defines the binary classification objective and model configuration, the multi-class classification objective and configuration, and the evaluation artifacts generated during training, such as confusion matrices and performance curves.

Chapter 11 presents the application design and architecture. It sets out the application's purpose and design goals, user interface structure, training-aligned inference pipeline (including the use of cached artifacts), prediction workflow and result presentation, and the practical characteristics of the prototype.

Chapter 12 reports and interprets the experimental results. It discusses binary classification results, binary metrics and threshold behavior, multi-class classification results, per-class interpretation, feature-importance analysis, and an integrated discussion of the observed performance.

Chapter 13 analyzes limitations, practical reflections, and future improvements. It considers dataset-related, representation-related, model-related, and application-related limitations, and outlines several directions for future enhancement of the system.

Chapter 14 concludes the thesis by summarizing the main findings, reflecting on the significance of the proposed approach, and highlighting the most important contributions and potential avenues for future research. The work ends with a references section listing all cited sources.

2. State of the Art

THE research domain of machine-learning based malware detection has grown rapidly over the past decade, producing a rich background of work around static analysis, visualization techniques, classical and deep learning classifiers and adversarial robustness. This chapter goes through the key technologies, methods, and datasets that form the academic and technical context for the system developed in this thesis.

2.1. *Static Analysis of PE Files Using Machine Learning*

Static analysis of Portable Executable files is one of the most studied approaches to malware detection because it offers the ability to extract important features from binaries without executing them, eliminating the risks and infrastructure requirements associated with dynamic analysis. Features extracted from PE headers have been shown to carry strong evidence for distinguishing malware from benign executables.

Studies confirm that malware detection systems that combine more feature categories extracted from PE files can achieve high accuracy on large datasets, separated in multiple malware categories. Combining multiple feature categories significantly improves classifier accuracy against samples that evade detection based on only one feature type [14].

2.2. *Entropy and Information-Theoretic Methods in Malware Detection*

Entropy analysis is a foundational technique in malware research, based on Shannon's information theory and widely applied to detect packed, encrypted and obfuscated code regions within executable files. The Shannon entropy H for a byte sequence is defined as:

$$H = - \sum_{i=0}^{255} p_i \log_2 p_i$$

Equation 1. Shannon Entropy

Where p_i is the probability of byte value i occurring in the analyzed region. Regions of legitimate compiled code typically exhibit moderate, structured entropy, while compressed or encrypted payloads approach maximum entropy, close to 8 bits per byte, reflecting an almost uniform byte distribution [13].

Studies from 2007 established the foundational academic case for entropy analysis as a malware detection tool, demonstrating that encrypted and packed malware consistently produces high entropy sections that are easily distinguishable from those of benign files. More than that, it was shown that analyzing entropy at the section level provides more reliable data, as individual sections can present radically different entropy profiles within the same binary [13].

2.3. *Binary Visualization Techniques*

The visualization of malware binaries as images was introduced around 2011, when it was proposed to convert the raw byte sequence of an executable into a grayscale image by treating consecutive bytes as pixel intensity values arranged in a fixed width bitmap. The resulting images exhibit visually distinctive textures and structural patterns that usually differ between malware families, reflecting the basic code structure, data layout

and obfuscation techniques for each family. Samples within the same family produce nearly identical images due to shared code lineage, making possible the classification by visual similarity without requiring disassembly or execution [15].

This foundation was extended in multiple directions such as visualized image matrices constructed from binary opcode sequences rather than raw bytes or section-aware segmentation that isolates individual PE sections withing the grayscale image and treats the resulting sub-images as separate channels in a multi-channel input. The section-aware segmentation showed that section specific visual patterns carry additional details that a single channel grayscale image cannot capture. This result is particularly relevant to this project, which also exploits PE section boundaries, but through entropy-based RGB encoding rather than grayscale segmentation [12]. RGB color encoding offers more freedom over grayscale visualization by allowing three independent information channels per pixel position.

2.4. Image-Based Malware Classification

The transformation of malware binaries into image like representations have made possible the application of computer vision techniques, particularly convolutional neural networks, to surpass traditional signature-based and many ML-based approaches on standard benchmarks.

The Maling dataset, introduced alongside Nataraj et al.'s original visualization work, contains 9339 grayscale malware images across 25 families and has become the primary benchmark for image-based malware classification research. On this dataset, CNN architectures consistently achieve high accuracy [16].

Transfer learning has proven particularly effective for image-based malware classification, allowing models pre-trained on large natural image datasets to be fine-tuned on malware images, overcoming the limited sample sizes of most malware datasets. In 2022, Deep Transfer Malware Image Classification (DTMIC) was introduced, which is a strategy leveraging pre-trained CNNs for malware image classification that achieves 98.92% test accuracy on Maling and 93.119% on the Microsoft BIG2015 dataset, with the performance gap between datasets reflecting the greater inter-family diversity of the larger BIG2015 dataset [17].

In 2016 there was demonstrated an alternative approach that bridges visualization and classical ML: converting malware binaries to images and applying Random Forest classification directly on image-derived features, achieving 0.9562 accuracy, a strong performance obtained without any deep learning models [2]. This is the closest academic work related to the present thesis and confirms that Random Forest can serve as a competitive classifier for image-based malware signatures when the images encode structurally informative features.

2.5. Malware Datasets and Benchmarks

The availability of labeled, high-quality malware datasets is a requirement for research in malware detection. Several datasets have become standards in the field, each with distinct characteristics relevant to different research use cases.

Malware Bazaar, operated by abuse.ch and Spamhaus, is a continuously updated community repository containing over 10 million malware sample that are freely accessible to researchers. Its large scale and recency make it suitable for training and evaluating models on contemporary threat samples, though the absence of standardized labels and the lack of benign samples require researchers to supplement it with benign file collections and perform their own labeling workflows.

Dumpware10, developed at Hacettepe university in 2020, is a dataset specifically designed for image-based malware classification research. It contains 3868 malware samples and 608 benign samples across 10 malware families, pre-rendered as RGB images under four different width rendering schemes (224, 300, file-size square root and 4096 pixels) to enable studies of how image resolution affects classification accuracy. The dataset supports open set classification, enabling models to learn to reject files that do not belong to any known training family.

Contagio Malware Dump provides 11.960 malicious and 16.800 clean files across multiple formats such as .exe, .pdf, .doc, .xls, .rtf, .mach-o, .elf and .jar, collected from threat intelligence sources and annotated for research use. Its multi-format composition makes it particularly useful for research addressing file-format-agnostic detection, though its predominantly document format malware focus limits its utility for PE-specific visualization projects.

Elastic Malware Benchmark for Empowering Researchers (EMBER), introduced in 2018, is the most widely adopted benchmark for static PE malware machine learning. It contains pre-extracted features from 1.1 million PE files (300.000 malicious, 300.000 benign, 300.000 unlabeled training samples, 100.000 malicious test samples, 100.000 benign test samples).

Maling was introduced in 2011 and it remains the standard benchmark for image-based malware classification. It contains 9449 grayscale images of malware binaries spanning 25 malware families, organized so that all images within a family exhibit visually similar textures. Despite its relatively small size, Maling continues to serve as the primary dataset for publications in the visualization-based detection domain.

Table 1. Comparison of Malware Datasets

Dataset	Samples	Format	Labels	Best Use
MalwareBazaar	10M+	Binary	Malware only	Large scale training
Dumpware10	4294	RGB images	Malware + benign	Image-based classification
Contagio	28.760	Multi-format Binary	Malware + benign	Multi-format, document malware
EMBER	1M	Pre-extracted features	Malware + benign	Static PE ML benchmarking
Maling	9339	Grayscale images	Malware only	Visualization benchmarking

2.6. *Adversarial Attacks on Machine Learning Based Antivirus Systems*

As ML-based malware classifiers have evolved, adversarial attacks specifically targeting these systems have become a major research concern. Adversarial attacks in this domain differ from image-domain adversarial examples in a very important point of view: any perturbation applied to a malware binary must preserve the binary's functional malicious behavior. An executable that no longer executes or whose payload is neutralized defeats the attacker's purpose.

For entropy and visualization-based systems specifically, the most relevant adversarial technique is entropy camouflage, which means the deliberate manipulation of a malware binary's section-level entropy to match the statistical profile of benign software. This motivates the multi-metric RGB encoding strategy of the present thesis, as by encoding entropy together with statistical measures across separate color channels, the system is more robust to single-metric manipulation than a purely entropy-based detector.

2.7. Existing Antivirus Architectures Incorporating Machine Learning

Commercial and academic antivirus systems have progressively integrated machine learning into their detection pipelines over the past decade, evolving from single-engine signature scanners toward multi-layer adaptive systems that combine ML, behavioral monitoring and cloud-backed intelligence.

Symantec Endpoint Protection's Advanced Machine Learning (AML) engine exemplifies the production integration of static ML: a model trained on global telemetry and deployed to endpoints via LiveUpdate, complemented by the cloud-based Intelligent Threat Cloud Service (ITCS) that handles uncertain cases by performing deeper analysis server-side. Microsoft Defender Similarly uses multiple on-device ML models operating on static PE features alongside a behavioral monitoring engine that analyzes process activity at runtime. These hybrid architectures reflect the general agreement in both industry and academia that no single detection method is sufficient, as effective endpoint protection requires combining static, dynamic and cloud-based detection in a layered defense.

The PAFE system represents the most directly comparable academic prototype to the approach of this thesis, being a lightweight visualization-based fast malware classification method that achieves high accuracy while specifically targeting efficiency on resource-constrained endpoints. PAFE demonstrates that carefully engineered visualization pipelines can match or approach the performance of a deep CNN-based system at substantially lower computational cost [7]. Building on these foundations, the system developed in this thesis pursues the same efficiency-accuracy balance through a new entropy and section aware RGB encoding combined with a Random Forest classifier.

3. Fundamentals of Cybersecurity and Malware Detection

3.1. Fundamental principles of information security

INFORMATION security is based on the CIA triad, CIA standing for “Confidentiality, Integrity and Availability”, this being a model that has guided practitioners and researchers for nearly four decades since the first models first addressed confidentiality and data integrity.

Confidentiality denotes the property that information is accessible only to authorized entities. Mechanisms enforcing confidentiality include access control lists, encryption algorithms, and identity management systems. Any unauthorized disclosure of sensitive data, be it through network interception, insider threats or improper configuration, constitutes a breach of confidentiality [18],[19].

Integrity ensures that data remains accurate, consistent, and unaltered by unauthorized parties throughout its lifecycle. Cryptographic hash functions, digital signatures, and version-control mechanisms serve as primary integrity controls. Tampering attacks, such as man-in-the-middle modification of data in transit, directly violate this principle [18], [19].

Availability guarantees that systems, services, and data remain accessible to authorized users when needed. Redundancy, failover architectures, load balancing, and disaster recovery plans are common availability controls. Denial-of-Service (DoS) attacks and ransomware encryption are two prominent threats that target availability by rendering resources inaccessible [18].

3.2. Malware Classification and General Characteristics

Malware, short for malicious software, is any program intentionally designed to cause harm to a computer system, network or user. The diversity and volume of malware variants have grown exponentially. Studies show that malware comes in a wide range of variations, including viruses, worms, Trojans, rootkits, backdoors, botnets, spyware, and adware, and that these classes are not mutually exclusive, meaning a single sample may exhibit characteristics of multiple other categories simultaneously. Malware has become the primary medium for launching large-scale attacks, including compromising computers, bringing down servers, sending spam, crippling critical infrastructure, and penetrating data centers [20], [21], [22].

Table 2. Malware Category Comparison

Category	Propagation Method	Requires Host?	Primary Payload	Example Families (Dataset)
RAT	Delivered as compact PE; user or dropper launch	No	Remote control, keylogging, data theft	Remcos, NanoCore, njRAT, AsyncRAT
Worm	Self-propagating via network vulnerabilities	No	Backdoor install, malware dropper	Conficker, Sality, Ramnit, Gamarue

Trojan	Hides as a legit app	No	Credential theft, loader for other malware	Emotet, Dridex, AgentTesla, RedLine
Ransomware	Email phishing, exploit kits, worms	No	File encryption, extortion demand	LockBit, WannaCry, Conti

3.2.1. Viruses

A computer virus is a self-replicating piece of malicious code that attaches itself to a legitimate host or program or file and requires user action (or host execution) to activate and propagate. Once activated, a virus can modify or delete data, corrupt system files, consume computing resources, and spread to other files on the same machine, or to other systems via shared media. Viruses may be classified by their infection target. File viruses attach to executable programs, boot-sector viruses infect the master boot record, macro viruses exploit scripting capabilities in document formats, and script viruses target interpreted code environments. Obfuscation techniques such as polymorphism and metamorphism allow viruses to alter their binary representation across infection cycles while retaining their malicious payload, thereby evading signature-based detection [20], [21], [22].

3.2.2. Worms

Worms are self-propagating malware that, unlike viruses, do not require a host program or human intervention to replicate. They exploit vulnerabilities in network services or operating systems to spread autonomously from one machine to another across networks. Worms can be classified by their scanning strategy as: random-scanning-worms (e.g., CodeRed v2, SQL Slammer) that generate random IP addresses to probe for vulnerable hosts; topological worms that use information from the infected hosts network connections; and hit-list worms that carry a pre-generated target list. The self-propagating nature of worms enables them to spread at extremely high velocities. Beyond simple propagation, modern worms often deliver additional payloads, such as installing backdoors for botnet recruitment or deploying ransomware [20], [22], [23].

3.2.3. Trojans

A Trojan horse (or simply just Trojan) is a malicious program that disguises itself as legitimate software to deceive users into installing and executing it. Unlike viruses and worms, Trojans do not self-replicate, their propagation depends on social engineering to induce a genuine user to run the application. Trojans are classified by their attack functionality: Remote Access Trojans (RATs) grant an attacker remote control of the victim's system; Trojan-Spies monitor user activity such as keystrokes and screenshots; Trojan-Droppers secretly install additional malware payloads; Trojan-Downloaders fetch and execute additional malicious code from the internet; Destructive Trojans delete or corrupt data. The increasing evolution in sophistication of Trojans, including integration with rootkit capabilities to hide from the operating system, makes them one of the most persistent and dangerous classes of malware [24].

3.2.4. Ransomware

Ransomware is a category of malware that restricts access to a victim's files or system, typically by encrypting data with strong cryptographic algorithms, and demands payment (often in cryptocurrency) in exchange for restoring access. Modern ransomware operations frequently use double extortion (encrypting data and threatening to leak stolen data) and triple extortion (adding DDoS threats or targeting the victim's clients), significantly increasing the pressure to pay. During The COVID-19 pandemic we witnessed a surge in ransomware attacks, particularly targeting healthcare, financial and governmental institutions, fueled in part by the expanded attack surface of remote work environments [25], [26].

3.2.5. Rootkits and spyware

Rootkits are a class of malware designed to conceal their own presence and the presence of other malicious software on a compromised system. They achieve stealth by modifying kernel data structures, hooking system calls, or manipulating operating system objects, thereby making malicious processes, files, and network connections invisible to standard diagnostic and security tools [27].

Spyware is malware designed to covertly collect information about a user's activities, including keystrokes, browsing habits, credentials, and personal data, and transmit this information to a third party without consent [28].

3.3. *Traditional Malware Detection Methods*

Malware detection methods can be broadly categorized into three families: signature-based, heuristic, and behavioral analysis. Each approach has distinct operational principles, strengths and limitations. A short representation of summary of detection methods is shown in Table 3.

Table 3. Detection Method Comparison

Method	How it works	Key Strength	Key Weakness	Fails Against
Signature-Based	Compares file byte patterns against a database of known malware	High precision, fast scanning, low false-positive rate	Needs prior knowledge about viruses	Zero-day malware, polymorphic and metamorphic malware
Heuristic	Evaluates structural indicators	Detects previously unseen variants that share patterns with known threats	High false-positive rate	New malware with new structural patterns
Behavioral / Dynamic	Executes the file in a sandbox and monitors activity	Detects zero-day and heavily obfuscated threats	Time-consuming, malware can detect sandbox and hide its payload	Environment-aware malware, fileless attacks

3.3.1. Signature-Based Detection

Pattern matching currently represents one of the most prevalent approaches in malware detection, with signature-based detection standing as the dominant technique within this domain. A signature can be defined as a distinct characteristic associated with a specific file, functioning similarly to a digital fingerprint of an executable.

Signature-based methods identify malware by comparing extracted patterns from known malicious files, offering higher efficiency and speed than alternative detection strategies. The uniqueness of these signatures, carefully derived to minimize duplication, results in detection systems with relatively low error rates. This high accuracy has made signature-based detection the preferred methodology in most commercial antivirus software.

However, these methods are limited in their ability to detect previously unknown malware variants and demand significant human effort, time, and financial resources to generate unique signatures. These factors represent the primary drawbacks of signature-based detection approaches. Additionally, such methods are ineffective against malware that alters its code with each infection, including polymorphic and metamorphic variants, further constraining their reliability. [29].

3.3.2. Heuristic Detection

Heuristic detection was developed to address the inability of signature-based methods to identify previously unknown malware. Rather than relying on exact matches against known patterns, heuristic analysis evaluates files based on indirect attributes, structural properties, or behavioral indicators that suggest malicious intent [30].

There are studies that overview various heuristic features, including API call patterns, opcode frequency distributions, n-gram analysis, and control flow graph properties, and discuss how these can be combined with machine learning classifiers to achieve rates of 95-98% for new malware strains. The key advantage of heuristic detection is its capacity to generalize to previously unseen variants by leveraging statistical patterns shared across malware families, rather than requiring an exact signature for each sample [30].

However, heuristic methods introduce a higher false-positive rate compared to signature-based detection, since benign programs that coincidentally exhibit suspicious features may be incorrectly flagged. Tuning the detection threshold represents a trade-off: lowering it increases sensitivity (catching more malware) but also increases false alarms [30].

3.3.3. Behavioral Analysis

Behavioral analysis (also named dynamic analysis) examines the runtime behavior of a program by executing it in a controlled environment, such as a virtual machine, emulator or sandbox, and monitoring its interactions with the operating system, file system, registry and network [31].

The dynamic feature extraction is more resistant to packing and polymorphism than static analysis because malware must eventually unpack and execute its real payload to achieve its malicious goals, therefore revealing behavioral indicators that static analysis cannot observe. Using sandboxing techniques reveals that realistic user behavior emulation within the sandbox causes malware samples to execute their payloads more thoroughly, providing richer behavioral data for detection. Monitored behavioral indicators typically include file system modifications (creation, deletion, or encryption of files), registry changes, network communication patterns, process creation and injection, and attempts to disable security software [31], [32].

Behavioral analysis can detect zero-day malware and sophisticated threats that evade both signature-based and heuristic methods. However, it suffers from several limitations: dynamic analysis is time-consuming compared to static approaches; malware may employ environment-aware evasion techniques to detect virtual machines or sandboxes and alter its behavior accordingly; and dynamic analysis cannot guarantee coverage of all possible execution paths [20], [31].

3.4. Limitations of Classical Antivirus Systems

Despite their widespread deployment and continued importance, classical antivirus systems face fundamental limitations that increasingly undermine their effectiveness against modern threats.

Inability to detect zero-day and unknown malware is the most critical limitation of signature-based antivirus. A signature can only be created after a malware sample has been captured and analyzed, leaving a window of exposure to novel threats. The signature-based methods have high precision but inherently low recall against previously unseen malware, and the process of manually crafting signatures is labor-intensive and fails to scale to the hundreds of thousands of new malware samples produced daily [20], [33].

Evasion by polymorphic and metamorphic malware is another big limitation of classical antivirus systems. Modern malware authors often employ polymorphic techniques (changing the binary representation of code during each propagation cycle while preserving its functionality) and metamorphic techniques (rewriting the entire code structure) to produce an effectively unlimited stream of unique variants that evade exact-match signature detection. Polymorphic attacks achieve a 100% evasion rate against CNN-based and signature-based malware classifiers on both Linux and Windows datasets, underscoring the severity of this vulnerability [20], [22], [34].

High false-positive rates and performance overhead represent one other big limitation of classical antiviruses. Heuristic and behavioral detection methods, while more capable of detecting unknown malware, introduce higher false-positive rates that degrade user trust and increase analysis workload. An early survey on malware analysis techniques observes that a considerable amount of legitimate files are incorrectly classified as malware by heuristic methods, and long scanning times impose significant computational overhead on endpoint systems [35], [36].

Sandbox and VM detection evasion represents another huge limitation because advanced malware frequently incorporates routines to detect whether it is executing inside a virtual machine, emulator, or sandbox environment. When such an environment is detected, the malware halts its malicious activity or exhibits only benign behavior, defeating dynamic analysis [20].

Scalability challenges become a problem as the number of new malware samples continues to grow at a rate of hundreds of thousands per day, classical antivirus systems face fundamental scalability challenges. Manual signature generation cannot keep pace with the volume of emerging threats, leading to a need of developing intelligent methods for effective and efficient malware detection from the real and large daily sample collection [37].

These accumulated limitations motivate the exploration of machine-learning-based and visualization-based approaches, including the image-based malware detection method proposed in this thesis.

4. Representation of Malware Samples

4.1. Analysis of Executable Files



Figure 1. PE File Structure

EXECUTABLE files constitute the main vehicle for malware targeting desktop and server environments. The Pocket Executable (PE) format is a data structure used by the Windows operating system to describe the layout of executable code and data in memory, serving as the standard container for .exe, .dll, .sys, and more related file types. A PE file begins with a Disk Operating System header (DOS header) of 64 bytes that contains the value 0x5A4D and the offset e_lfanew which points to the New Technology headers (NT headers), followed by a DOS stub that serves as a legacy compatibility message. The NT headers comprise a four-byte PE signature (0x4550), a File Header with metadata such as machine architecture, the number of sections and a time

stamp, and an Optional Header that is mandatory for executable files and contains critical fields such as the entry point address, image base, and Data Directory [38], [39].

Immediately after the headers, the PE file is partitioned into one or more sections, each being marked by a Section Header entry that contains the section name, virtual size, relative virtual address, raw size, file pointer, and a set of permission flags (executable, readable, writable). Common sections include .text (compiled machine code), .data (initialized global and static variables), .rdata (read-only data), .bss (uninitialized data), .idata (import directory), .reloc (base relocation entries) and .rsrc (embedded resources) (Figure 1). Each section has a different statistical profile, as .text sections contain executable bytecode with moderate entropy typical of compiled code, and .data and .rdata often show lower entropy being composed of structured or string-like content [38], [39].

The static analysis of PE files doesn't require execution of the program, which makes it both safe and fast compared to dynamic approaches. A static malware detection system extracts all relevant information from PE files, including Dynamic Link Library (DLL) names, Application Programming Interface (API) function imports, PE header fields, and section metadata, without running the executable [14], [40].

4.2. Representation of Binary Data for Automated Analysis

Converting raw binary content into machine-readable feature representations is a prerequisite for applying automated classification algorithms to malware detection. Binary data is unstructured from a traditional text-processing perspective. Bytes can take any of 256 possible values with no direct semantic interpretation for a classifier, making it necessary to derive structured feature spaces from the raw byte sequence [41].

Feature extraction from PE header metadata, such as the number of sections, presence of specific DLLs, flags in the Optional Header and section size ratios, provides an interpretable representation that works well with tree-based and linear classifiers. Byte n-gram models treat binary content as sequences of overlapping fixed-length byte windows and build statistical language models over the resulting vocabulary, but they suffer from extremely high dimensionality (256^n features) that requires big dimensionality reduction. Operation code (opcode) analysis first disassembles the executable and then extract distributions or n-grams over assembly mnemonics, providing a more semantically meaningful representation, but it requires successful disassembly which fails on packed samples [14], [41], [42].

Entropy-based representation is a good middle ground that avoids the need for disassembly while still capturing meaningful statistical structure. A study made in 2020 demonstrates that entropy profiles, sequences of Shannon entropy values computed over fixed-size chunks of a binary, achieve 97.4% classification accuracy on disk-resilient malware with a 3% false positive rate, confirming that chunk-level entropy captures consistent structural patterns across malware families [43].

The Shannon entropy of a byte sequence of length N , comprising bytes drawn from an alphabet of size 256, is defined as:

$$H = - \sum_{i=0}^{255} p_i \log_2 p_i$$

Equation 2. Shannon Entropy of a Byte Sequence of Length N

Where p_i is the empirical probability of byte value i in the sequence, and H is a notation for the entropy of the chunk. For a chunk of 256 bytes, this formula produces a value in the range of $[0,8]$, with 0 indicating a

perfectly uniform sequence and 8 indicating a maximally random distribution across all 256-byte values. Entropy values in the range 4-7 are typical of compiled executable code, values above 7 suggest the presence of packed, compressed, or encrypted code, and values below 2 suggest repetitive fill or null-byte padding regions [44].

4.3. Transforming Binary Files into Visual Representations

The transformation of binary executable files into images is a well-known technique in malware analysis research, enabling the application of image processing and machine learning methods to the malware detection and classification problem.

The image generation process in entropy-based visualization departs from the raw byte-to-pixel mapping by computing statistics, entropy values, over fixed-size chunks of the binary, and then mapping these values to a color space to produce the final image. The resulting image encodes structural properties of the binary rather than raw content, making them more robust to minor code mutations that preserve functional structure while altering individual bytes.

In the approach adopted in this project, the first 16,384 bytes (16KB) of each executable are divided into 64 non-overlapping chunks of 256 bytes each. For each chunk I , the Shannon entropy H_i is computed using the formula above, producing a vector of 64 entropy values. These values are then normalized to the range $[0,1]$ and mapped to an RGB color encoding using a green-yellow-red colormap:

- Low entropy ($H_i \approx 0$): mapped to green (R=0, G=255, B=0), indicating structured or repetitive content
- Medium entropy ($H_i \approx 0.5$): mapped to yellow (R=255, G=255, B=0), indicating typical compiled code or data
- High entropy ($H_i \approx 1$): mapped to red (R=255, G=0, B=0), indicating packed, encrypted, or otherwise highly compressed regions

The 64 RGB color values are then arranged into an 8×8 pixel matrix in row-major order, producing a compact color image of dimensions $8 \times 8 \times 3$ (192 raw feature values). This design directly encodes the entropy signature of the file's initial 16 KB into a visually interpretable spatial representation.

4.4. Image-Based Signature for Malware Classification

An image-based signature, in the context of malware detection, is a compact visual representation of a file's binary or statistical content that can be used by a classifier to determine whether the file is malicious. Unlike traditional byte-sequence signatures that require exact or nearly exact matching, image-based signatures encode higher-level structural and statistical properties that are preserved across malware variants and are therefore more generalizable [45].

Malware images, constructed by a raw byte-to-grayscale technique, can be directly used as feature vectors for a Random Forest classifier, achieving high accuracy on a dataset with big variation. The visual similarity of malware variants within the same family is preserved sufficiently for tree-based classifiers operating on flattened pixel feature vectors to learn discriminative boundaries without requiring any image feature extraction preprocessing. This result is significant for the approach taken in this thesis, which similarly uses flattened image features as input to a Random Forest classifier [2].

The concept of an image-based signature differs fundamentally from traditional antivirus signatures from three important points of view. First, the image-based signatures are learned from data rather than manually

crafted, enabling generalization to previously unseen samples that share statistical and structural properties with training examples. Second, they are resistant to minor mutations, code transpositions, or variable name changes that would produce a different byte sequence while leaving the entropy profile mainly unchanged. Third, they are generated without code execution, eliminating the need for sandboxing processes [44].

4.5. *Advantages and Limitations of Visual Representations*

Advantages:

Visual representation techniques offer several well-documented advantages over both traditional signature-based detection and raw-feature machine learning methods.

No execution or disassembly required. Visualization-based methods operate entirely on the static binary content of a file, requiring no execution environment, virtual machine or disassembler. This property eliminates sandbox evasions and makes the approach safe and fast [45].

Robustness to common obfuscation. Entropy-based visualization is resistant to superficial code mutations, simple obfuscation, and variable padding because the entropy profile of the binary's structural regions remains stable across such transformations [15].

Interpretability and human-in-the-loop analysis. Color matrix images are directly interpretable by human analysts, red regions signaling high-entropy (potentially packed or encrypted) content, while green regions indicate structured or repetitive data.

Efficiency and scalability. Image-based feature extraction is computationally lightweight compared to disassembly-based or dynamic analysis pipelines. The entire feature extraction process for the approach in this thesis can be performed in milliseconds per file.

Compatibility with standard classifiers. Once generated, image-based signatures are compatible with both traditional machine learning classifiers (Random Forests, Support Vector Machines) and deep learning architectures (CNNs, transfer learning).

Limitations:

Despite having advantages, visual representations of malware also carry several limitations that must be acknowledged.

Adversarial vulnerability. Attackers who are aware that a target system uses visualization-based detection can develop adversarial malware samples designed to produce images similar to benign files.

Sensitivity to file size variations and truncation. The approach of extracting only the first 16 KB of a file means that malware payloads embedded in later sections, beyond the selected 16KB, would be completely missed. Larger or variable size files introduce normalization challenges that affect the spatial consistency of the resulting image [1].

Limited to static structure. Visual representations capture the static structural and entropy properties of a binary but cannot capture dynamic behaviors such as API call sequences, network communication patterns, process injection or in-memory decryption at runtime. As noted in Section 3.2, malware samples employing runtime decryption may appear structurally similar to benign files at the static analysis [46].

Dataset dependency and generalization. Classification models trained on image-based signatures are strongly dependent on the distribution of the training dataset. Models trained on particular malware families or collection periods may fail to generalize to newer families, targeted attacks or zero-day samples whose entropy profiles differ substantially from the training distributions [6].

Confusion between high-entropy benign files and malware. Legitimate software that uses compression, strong encryption or obfuscated code may produce entropy profiles and color matrices similar to those of packed or encrypted malware, leading to false positives.

5. Fundamentals of Digital Image Processing

5.1. Representation of Digital Images

A digital image is a discrete, finite two-dimensional function $f(x, y)$, where x and y denote spatial coordinates and the value f at any pair (x, y) is called the intensity or gray level of the image at that point. When x , y , and the intensity values are all finite discrete quantities, the image is called a digital image. In practice, a digital image is stored as a rectangular matrix of pixels, where each pixel holds one or more numeric values representing its color or brightness. The spatial resolution of an image is defined by the number of pixel rows and columns in the matrix [47], [48].

The most widely used color model for digital images in computer vision and image processing is the RGB (Red, Green, Blue) model, an additive color model in which three independent color channels, red, green and blue, are combined in varying proportions to reproduce a long range of colors. In the standard 8-bit per channel representation, each channel takes an integer value in the range $[0, 255]$, creating $256^3 = 16777216$ distinct colors. An RGB color image requires three bytes per pixel, one for each component, and is stored as a three-dimensional array (or tensor) of shape $H \times W \times 3$, where H and W are the image height and width in pixels [47].

Alternative color spaces address specific limitations of the RGB model. The HSV (Hue, Saturation, Value) model decouples chromatic information from luminance, making it more robust to lighting variations. For grayscale images, a single intensity channel suffices, typically computed from RGB values as a weighted luminance combination [48]. In the context of this thesis, the RGB model is the best choice because the three channels directly encode three independently meaningful entropy-related quantities: the color of each pixel in the generated malware signature image encodes a normalized entropy value mapped through a green to red colormap, with all three channels jointly encoding the specific color corresponding to the entropy level of the corresponding 256-byte chunk.

This project uses a digital image as a feature matrix where from a machine learning point of view, a digital image is a high-dimensional numerical array. An 8×8 RGB image, as used in this thesis, is a tensor of shape $8 \times 8 \times 3 = 192$ scalar values. When this tensor is flattened into a one-dimensional feature vector, it produces a vector of 192 features, one for each R, G or B channel value at each of the 64 pixel positions, arranged in row-major order. This flattened representation is the direct input to the Random Forest classifier.

5.2. Visual Features and Feature Extraction

Feature extraction is the process of transforming raw image data into a compact numerical representation suitable for analysis or classification. Raw pixel values constitute the most basic feature representation, however, they are high-dimensional and sensitive to noise, illumination changes and spatial shifts. Structured feature extraction methods aim to derive representations that are more discriminative, compact and robust than raw pixel arrays [49].

Statistical features characterize the distribution of pixel intensities within an image or region include measures such as mean, variance, skewness and entropy of the intensity histogram [49].

Color features capture the distribution and organization of color values across the image. Typical color features include color histograms (the frequency distribution of each channel's values), color moments (mean, standard deviation, and skewness per color channel), and dominant color descriptors [49]. In the malware image classification context, color histograms and per-channel statistical moments computed over the 8×8 RGB matrix directly reflect the distribution of entropy values across the 64 file chunks: a predominantly green histogram indicates low entropy (most probably benign, structured) file, while a predominantly red histogram indicates a high entropy (packed or encrypted) file. These color statistics can serve as interpretable and human readable summaries of the file's entropy profile.

For small, fixed-size images such as the 8×8 matrices generated in this thesis, flattening the image into a raw pixel feature vector is a more effective approach. Each pixel's red, green and blue channel values are concatenated row by row into a 192-dimensional vector, which then serves as the feature representation for classification. The small dimensionality (192 features) of the 8×8 representation mitigates the problem of dimensionality, reduces the risk of overfitting, and enables fast training [1], [2].

5.3. Image Preprocessing

Image preprocessing refers to the set of operations applied to raw images before feature extraction or model training, with the objective of enhancing data quality, ensuring consistency across samples, and improving classifier performance. Preprocessing is a really important step when images are generated for heterogeneous sources or when the classifier is sensitive to variations in scale, contrast or noise [50].

Pixel value normalization is one of the most fundamental preprocessing steps. It rescales raw integer pixel values to a floating-point range such as $[0, 1]$ using min-max scaling:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Equation 3. Pixel Value Normalization

or to zero-mean, unit-variance representation (z-score standardization):

$$x_{std} = \frac{x - \mu}{\sigma}$$

Equation 4. Pixel Value Z-Score Standardization

where μ is the mean and σ is the standard deviation of the pixel values across the training dataset. Normalization ensures that no single feature dimension dominates the learning process simply by virtue of its numerical scale, and is essential for gradient-based optimizers and distance-based classifiers. For tree-based classifiers such as Random Forests, normalization is less critical because split thresholds are learned directly from data distribution, but it remains good practice for ensuring consistent feature scales and facilitating fair comparison across feature dimensions [51].

In the context of entropy-based malware images, normalization is applied at the entropy computation stage as each Shannon entropy value $H_i \in [0, 8]$ is normalized to the range $[0, 1]$ before color mapping by dividing by 8:

$$H_{i,norm} = \frac{H_i}{8}$$

Equation 5. Normalized Shannon Entropy

This maps entropy values linearly onto the green to red colormap, ensuring that all images share a consistent color scale regardless of the absolute entropy levels of individual files (Figure 2).

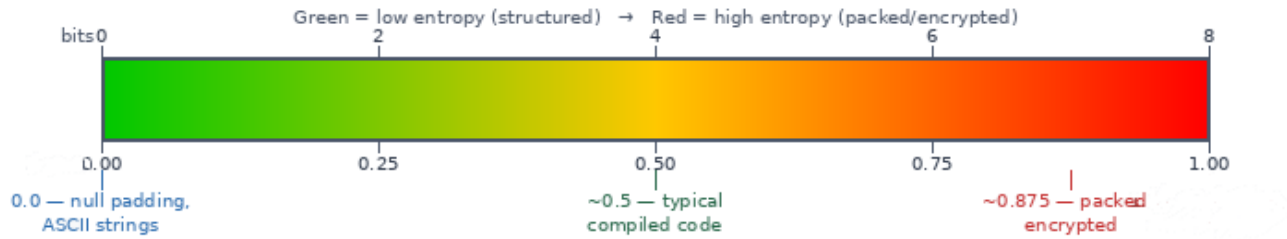


Figure 2. Entropy-to-RGB Color Mapping

5.3.1. Resizing and Spatial Consistency

Ensuring that all images have identical dimensions is a prerequisite for batch processing by fixed-input-size classifiers. In the proposed project, image dimensions are fixed by construction at 8×8 pixels per sample, since the 64 entropy values computed from the 64 fixed-size chunks always map to exactly 64 pixels arranged in an 8×8 grid. No resizing is required. This choice guarantees spatial consistency across all samples in the dataset and eliminates the distortions that could result from interpolation-based resizing of variable size images.

5.3.2. Data Augmentation

Data augmentation uses techniques that artificially expand the training dataset by generating modified copies of existing samples, therefore improving generalization and reducing overfitting, particularly when the original dataset is of limited size. Common augmentation techniques for standard images include horizontal and vertical flipping, rotation, cropping, color jittering, and noise injection. Data augmentation is one of the most consistently effective methods for reducing overfitting in deep learning image classifiers [52].

For the entropy-based 8×8 malware color matrices used in this thesis, standard spatial augmentations must be applied with care because the columns of the image correspond to neighboring 256-byte chunks of the binary file arranged left to right, top to bottom in the order of their appearance in the file, spatial transformations that reorder pixel positions would destroy the positional semantics embedded in the image.

5.3.3. Class Balancing

In malware detection datasets, class imbalance, where malware samples substantially outnumber benign samples or vice versa, is a common challenge that can cause classifiers to be biased toward the majority class. Preprocessing practices to address imbalance include undersampling the majority class, oversampling the minority class, or applying class-weight adjustments during training. Random Forest classifiers support built-in class weighting that penalizes misclassification of the minority class more heavily, providing a straightforward mechanism for handling moderate class imbalance without requiring explicit resampling [53].

5.4. *Image Classification and Pattern Recognition*

Image classification is the task of assigning a predefined category label to an input image based on its visual content. It is one of the central problems in computer vision and pattern recognition, with applications ranging from medical diagnosis and remote sensing to security and biometrics [54].

5.4.1. Classical Machine Learning Approaches

In classical machine learning, image classification is performed in two stages: feature extraction followed by classification (training a model to map feature vectors to class labels). Common classifiers applied to image feature vectors include Support Vector Machines (SVMs), k-Nearest Neighbors (k-NN), Decision Trees and Random Forests [54].

5.4.2. Random Forest for Image Classification

The Random Forest algorithm is an ensemble learning method that constructs a large number of decision trees during training and outputs the class predicted by the majority of individual trees for a new input. Each tree is grown on a bootstrap sample of the training data, and at each internal node of each tree, only a randomly selected subset of features is considered for the optimal split, a technique known as feature subsampling that decorrelates the individual trees and improves the ensemble's generalization. The theoretical basis for the algorithm's strong generalization is the forest convergence theorem: as the number of trees grows, the generalization error converges to an upper bound determined by the strength of individual trees and the correlation between them [55].

5.4.3. Deep Learning Approaches

Deep learning methods, in particular Convolutional Neural Networks, have become dominant in image classification, achieving state of the art accuracy on large benchmark datasets. CNNs learn hierarchical feature representations directly from raw pixel data through alternating convolutional and pooling layers, followed by fully connected layers that produce class probability outputs [54].

For malware image classification, CNNs and transfer learning methods have been shown to achieve detection accuracies of 95-99% on standard benchmarks such as Maling when trained on sufficient data. However, deep CNN architectures require substantially more training data, computational resources, and hyperparameter tuning compared to Random Forest. The choice of Random Forest for the proposed system is a decision that prioritizes computational efficiency, interpretability, and suitability for the compact 192-feature input space over maximum benchmark accuracy [6], [56].

5.4.4. Evaluation Metrics for image Classification

The performance of image classifiers is typically evaluated using a standard set of metrics derived from the confusion matrix, a table that records the counts of true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN) across the classification task [57]:

- Accuracy: $\frac{TP+TN}{TP+TN+FP+FN}$, the proportion of all correctly classified samples.
- Precision: $\frac{TP}{TP+FP}$, the proportion of positive predictions that are truly positive.

- Recall (Sensitivity): $\frac{TP}{TP+FN}$, the proportion of actual positives correctly detected.

- F1-Score: $\frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$, the harmonic mean of precision and recall, providing a single balanced measure.

In malware detection, recall is particularly critical because an undetected malware sample (false negative) represents a security failure with potentially severe consequences, while a false positive (benign file classified as malware) disrupts normal system operation.

5.4.5. Cross-Validation and Generalization

To obtain reliable performance estimates and guard against overfitting, particularly on datasets of moderate size, k-fold cross-validation is the standard methodology. In k-fold cross-validation, the dataset is partitioned into k equal sized folds; the model is trained on k-1 folds and evaluated on the remaining fold, repeating this process k times so that every sample serves exactly once as a test sample. The final performance estimate is the mean (and standard deviation) of the metrics across all k folds. Stratified k-fold cross-validation further ensures that the class distribution is preserved in each fold, which is important for imbalanced malware detection datasets [2], [55].

6. Machine Learning and Neural Network Based Malware Detection

6.1. Fundamentals of Machine Learning

MACHINE learning (ML) is a subdiscipline of artificial intelligence concerned with the design and study of algorithms that improve their performance on a task through experience, learning statistical patterns from data rather than relying on explicit, hand-coded rules. The central distinction between ML types lies in whether labeled data is available during training. In supervised learning, the algorithm is presented with a set of input-output pairs (x_i, y_i) and learns a mapping $f: X \rightarrow Y$ that minimizes a defined loss between predicted and true outputs and the learned model is then applied to new, unseen outputs. In unsupervised learning, no labels are provided and the algorithm discovers intrinsic structure in the input space, for example, clustering samples into groups or learning a lower-dimensional representation. Semi-supervised learning combines small amounts of labeled data with large amounts of unlabeled data, which is particularly relevant in malware detection where labeling is expensive and time-consuming [58], [59].

The malware classification task addressed in the thesis is a binary supervised classification problem: given a feature representation x derived from an executable file, the model must predict a label $y \in \{\textit{benign}, \textit{malware}\}$. The quality of a supervised learning model is characterized by generalization, its ability to perform well on previously unseen data drawn from the same distribution as the training set. Generalization is controlled by the bias-variance tradeoff: models with high bias are overly simplistic and underfit the training data, while models with high variance capture noise in the training set and overfit, failing to generalize [58], [59].

The training pipeline for supervised ML involves:

- Collecting and labeling a dataset;
- Splitting it into training, validation, and test sets;
- Selecting and training a model by minimizing a loss function;
- Evaluating performance on held-out data using appropriate metrics;
- Iterating on the model or feature representation based on evaluation results.

In the context of this thesis, these steps map directly to the entropy-based image generation pipeline where the dataset consists of labeled PE executables, features are the 192-dimensional flattened RGB pixel vectors derived from entropy color matrices, and training is performed using a Random Forest classifier.

6.2. Classical Machine Learning for Malware Detection

Prior to the deep learning era, classical Machine Learning classifiers were the primary vehicle for automated malware detection. Supervised classifiers including Decision Trees, SVMs, k-NNs and Random Forests were trained on feature sets extracted from PE files, byte n-grams, or API call sequences [58].

Decision Trees recursively partition the feature space by selecting the feature and threshold that maximally reduces impurity at each internal node. They are interpretable and fast to evaluate but prone to high variance when grown deep on limited data [58].

Support Vector Machines find the maximum margin hyperplane separating classes in a kernel-defined feature space. They generalize well on small to medium datasets with high dimensional features and are effective on text and byte n-gram representations of malware [58].

k-Nearest Neighbors classifies a test sample by the majority label among its closest training samples in feature space. It is simple, non-parametric but computationally expensive at inference time and sensible to irrelevant features [58].

6.2.1. Random Forest

The Random Forest algorithm, constructs an ensemble of T decision trees, each trained independently on a bootstrap sample of the training data. At each internal node of each tree, only a randomly selected subset of $m \leq p$ features (where p is the total number of features) is considered for the optimal split and it is a technique called feature subsampling or random subspace projection. The final class prediction for a new sample is the majority vote across all trees [55].

By decorrelating trees through feature subsampling, Random Forests achieve lower generalization error than individual deep decision trees or bagged trees while remaining robust to overfitting. Empirically, the algorithm achieves strong performance with minimal hyperparameter tuning. Typically, only the number of trees T and the subset size m require adjustment. Breiman (2001) proved a convergence theorem that showed that the generalization error of a Random Forest converges almost surely to an upper bound as $T \rightarrow \infty$, with the bound determined by two quantities: the accuracy of individual trees and the correlation between them [2], [55].

For the classification pipeline in this thesis, the Random Forest is trained on 192-dimensional feature vectors, using all training samples split into T trees with $m = \sqrt{192} \approx 14$ features per split as recommended for classification tasks. The trained forest is then used at inference time to classify new executables: the entire pipeline from file ingestion to prediction executes in milliseconds, satisfying the latency requirements for endpoint protection scenarios [55].

6.3. *Artificial Neural Networks Fundamentals*

Artificial Neural Networks (ANNs) are computational models loosely inspired by the structure of biological neural systems, in which simple processing units (neurons) are organized into layers and connected by weighted edges that are adjusted during training to minimize a loss function. ANNs can learn hierarchical, nonlinear representations from raw data, making them particularly powerful for tasks where hand-crafted features are difficult to engineer [60].

6.3.1. The Artificial Neuron Model

The artificial neuron is the fundamental computational building block of all neural network architectures. The first formal mathematical model of a neuron was proposed by McCulloch and Pitts in 1943 being a binary threshold unit that fires (output = 1) if the weighted sum of its inputs exceeds a fixed threshold, and is silent otherwise. This model demonstrated that simple interconnected threshold units could compute any logical function, establishing the theoretical basis for neural computation [60].

Rosenblatt's Perceptron (1958) extended McCulloch-Pitts model by introducing numerical weights and a learning rule. The perceptron adjusts its weights incrementally based on misclassified examples, converging to a correct solution whenever one exists. The modern artificial neuron computes:

$$z = \sum_{j=1}^n w_j x_j + b$$

Equation 6. Computation Formula of an Artificial Neuron

Where x_j are the inputs, w_j are the learned weights, b is a bias term, and z is the pre-activation value, also called net input. A nonlinear activation function $\sigma(\cdot)$ is then applied to produce the neuron's output $a = \sigma(z)$. The activation function is critical for enabling neural networks to learn nonlinear decision boundaries because without it, any multi-layer network would collapse to a linear function of the input [60], [61].

6.3.2. Multilayer Perceptron

The Multilayer Perceptron (MLP) is the classic feedforward neural network architecture that consists of an input layer, one or more hidden layers, and an output layer, with all neurons in adjacent layers fully connected. Each neuron in a hidden layer computes a weighted sum of all outputs from the preceding layer, applies an activation function and passes its output to all neurons in the next layer. The depth and width of the network is determined by the number of hidden layers and respectively by the number of neurons per layer and they determine the expressive capacity of the network.

6.3.3. Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are specialized feedforward architectures designed to exploit the spatial structure of image data through three key architectural features: local receptive fields (each neuron connects to a small spatial region of the preceding layer rather than full input), shared weights (the same set of weights, that are called a convolutional filter or kernel, is applied at every spatial position) and pooling layers (which down sample feature maps to provide spatial invariance and reduce dimensionality) [62].

For malware image-based detection, CNNs have become the dominant deep learning architecture, being demonstrated that CNN-based classifiers applied to malware images achieve high accuracy, consistently outperforming simpler architectures on image feature extraction tasks [4].

6.3.4. Deep Neural Networks

Deep Neural Networks refer to neural networks with multiple hidden layers, enabling them to learn hierarchical feature representations where lower layers detect simple patterns, like edges and textures, and higher layers compose these into more meaningful features. The term of Deep Neural Networks encompasses deep MLPs and deep CNNs.

Deep Neural Networks became the dominant category used in computer vision, natural language processing and security analysis after finding that CNNs, which are a subclass of Deep Neural Networks, excel in malware analysis by providing superior feature extraction from binary representations. However, Deep Learning Networks require big labeled training data, high GPU computational resources and strict hyperparameter tuning [62], [63].

6.4. Training and Optimization

Training a neural network involves minimizing a loss function $L(\theta)$ that quantifies the discrepancy between the network's predictions and the real labels over the training dataset, where θ denotes all trainable parameters (weights and biases). The binary cross-entropy is standard for binary classification:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - \hat{y}_i) \log(1 - \hat{y}_i)]$$

Equation 7. Binary Cross-Entropy

Where $y_i \in \{0,1\}$ is the true label, $\hat{y}_i$ is the predicted probability of class 1, and N is the number of training samples [64].

Gradient descent is another way to optimize a neural network, and it iteratively updates the parameters in the direction of the negative gradient of the loss:

$$\theta = \theta - \alpha \frac{\partial}{\partial \theta} L(\theta)$$

Equation 8. Gradient Descent

Where α is the learning rate, a hyperparameter that controls the step size of each update. In practice, Stochastic Gradient Descent estimates the gradient using a mini-batch of samples rather than the full training set, enabling faster convergence and better generalization through regularization from gradient noise. Adaptive optimizers such as Adam (Adaptive Moment Estimation) combine momentum and per-parameter learning rate scaling, outperforming the Stochastic Gradient Descent on most deep learning benchmarks [65].

Another optimization way is through backpropagation, this being the algorithm used to compute the gradient of the loss with respect to all network parameters by applying the chain rule of calculus in reverse through the computational graph.

Regularization techniques prevent overfitting by constraining model complexity. Common methods include L2 weight decay (penalizing large weights), Dropout (randomly zeroing a fraction of neuron outputs during training to prevent co-adaptation), and early stopping (terminating training when validation performance begins to degrade). In this case, for a tree-based method such as Random Forest, regularization is achieved through the ensemble mechanism itself, where averaging over many decorrelated trees it reduces variance without increasing bias [55], [62], [65].

6.5. Neural Network-Based Malware Detection

The theoretical established in the sections before, find direct practical application in the domain of malware detection. As traditional signature-based and heuristic methods have proven insufficient against polymorphic and zero-day threats, neural networks have emerged as a better alternative, because they learn particular representations directly from data rather than relying on manually curated rules.

6.5.1. Malware Classification Using Neural Networks

The application of neural networks on malware classification has grown substantially over the past decade, driven by the availability of large labeled datasets, improvements in network architectures and the inefficiency of traditional signature-based methods for detecting polymorphic and zero-day threats. Neural

networks operate directly on raw or minimally preprocessed representations of malware, learning discriminative features from data rather than relying on manually engineered indicators.

Deep neural network architectures used for malware detection using image-based representations achieve strong binary classification (malware vs. benign) with training-test splits of 80/20. By combining image-based features and numeric PE metadata, there was achieved an accuracy over 95%, outperforming standalone imagery and numeric modalities. These results highlight the flexibility of neural networks architectures in this field [66], [67].

6.5.2. Detection Models Based on Visual Representations

Detection models based on visual representations of malware use the spatial and statistical structure encoded in malware images to drive classification decisions. Grayscale byte-to-pixel images of malware show strong intra-family similarity and inter-family dissimilarity, making image-based feature extraction and classification a principled approach. RGB colormap-based malware images, where binary content is transformed through a colormap-based function into a three-channel image, consistently show higher classification accuracy than grayscale representations [4], [15].

6.6. *The Classification Pipeline*

The classification pipeline implemented in this thesis consists of the following stages:

1. **Dataset collection and labeling.** PE executable files are collected from repositories of known malware and benign software. Each file is assigned a binary label (malware = 1, benign = 0). The dataset is balanced to avoid class bias.
2. **Byte extraction.** For each executable, the first 16 KB are extracted. Files shorter than 16 KB are zero-padded to ensure uniform input size.
3. **Segmentation.** The 16 KB block is divided into 64 non-overlapping chunks of 256 bytes each.
4. **Entropy computation.** The Shannon entropy is computed for each of the 64 chunks, producing a vector $H = [H_1, H_2, \dots, H_{64}]$ with values in $[0, 8]$.
5. **Normalization.** Each entropy value is normalized.
6. **Color mapping.** Each normalized entropy value is mapped to an RGB color through a green-to-red gradient colormap, where low entropy maps to green, medium entropy maps to yellow and high entropy maps to red.
7. **Image construction.** The 64 RGB values are arranged row major into an 8×8 pixel matrix and saved as a PNG image file.
8. **Feature extraction.** Each $8 \times 8 \times 3$ RGB image is flattened into a 192-dimensional feature vector and additional static entropy-related features are extracted.
9. **Model Training.** A Random Forest classifier is trained on the labeled feature vectors using stratified k-fold cross-validation. Hyperparameters, such as number of trees, maximum depth and minimum samples per leaf are tuned via grid search.
10. **Inference.** A new executable is processed through stages 2-8, and the resulting feature vector is passed to the trained Random Forest models, which returns a binary classification (malware/benign) and a malware family classification, both with a confidence score.

6.7. *Performance Evaluation Metrics*

The performance of the classification model is evaluated using a standard set of metrics computed from the confusion matrix, which records the counts of true positives, true negatives, false positives and false negatives. For this thesis were used: accuracy (proportion of all correctly classified samples), precision (proportion of predicted malware that is truly malicious), recall (proportion of actual malware samples correctly detected), specificity (proportion of benign files correctly identified as benign) and F1-Score (harmonic mean of precision and recall).

In the malware detection context, recall carries the most operational importance, because a missed detection (false negative) leaves a malicious file undetected on the endpoint, potentially leading to system compromise or data breach. Precision manages false alarms as an excessive false -positive rate would cause legitimate software to be blocked, disrupting normal user operations. The F1-Score captures both simultaneously. The complete evaluation is presented further in the following chapters.

6.8. Challenges and Current Research Directions

The fundamental challenge in any static classification system, including the one in this thesis, is the detection of malware samples that differ substantially from the training distributions. Zero-day malware refers to threats exploiting previously unknown vulnerabilities or exhibiting new behavior not yet seen, for which no labeled training examples exist. Supervised ML models, trained on historical samples, cannot generalize to most zero-day threats in the same way they generalize across variants of known families. Concept drift refers to the phenomenon in which the statistical properties of the malware distribution shift over time, causing a static trained model's performance to degrade as the deployment environment diverges from the training environment. This thesis does not work with the concept of concept drift, but it represents a good direction for future work [68].

Adversarial attacks on malware classifiers involve deliberate manipulation of malware samples to cause misclassification while preserving malicious functionality. For visualization-based systems, byte-level perturbations in non-executable regions can alter the entropy profile of targeted chunks, shifting the color of corresponding pixels in the generated image and potentially changing the classifier's prediction [69].

Current research directions include multimodal fusion, explainability and interpretability, continual and online learning and adversarial training.

7. Modern Antivirus Systems and Intelligent Architectures

THE continuous escalation of the malware threat landscape has driven a fundamental transformation in antivirus system design. First-generation AV products relied on static signature databases that proved structurally inadequate against the volume and sophistication of modern malware. In response, contemporary endpoint protection platforms integrate multiple detection modalities, embed machine learning directly into the endpoint agent, and use cloud-scale telemetry to respond to emerging threats in near real-time. This chapter goes through the architecture of these systems, the integration of ML-based detection, adaptive response mechanisms and the persistent limitations that define the boundaries of current approaches.

7.1. *Architecture of an antivirus system*

A modern AV system is a multi-component software platform whose primary function is to detect, quarantine and remediate malicious software on protected endpoints. While early AV products consisted of little more than a signature scanner and a quarantine directory, contemporary systems have evolved into layered architectures that combine multiple detection engines, a real-time file system driver, cloud-based intelligent infrastructure and automated response capabilities [70].

At the client level, a production AV system is composed of the following components:

- **File System Driver:** it is a kernel-level component that intercepts file system operations (open, read, write, execute) and passes file handles or content to the detection engine before the operation is permitted to complete.
- **Detection Engine:** it is the core analysis module responsible for evaluating files against signatures, heuristic rules, machine learning models, and behavioral profiles. For example, Microsoft Defender, employs multiple on-client ML models that render verdicts within milliseconds.
- **Signature and Model Database:** it is a locally cached repository of known malware fingerprints, heuristic rules and serialized ML model weights, updated periodically through cloud synchronization.
- **Quarantine Manager:** a module responsible for isolating files identified as malicious by moving them to an encrypted storage area from which they cannot execute, preserving them for analysis or restoration.
- **Remediation Engine:** it is a component that removes or repairs files and registry entries created by detected malware, attempting to restore the system to a clean state.
- **Real-Time Protection Monitor:** it is a background service that continuously monitors running processes, network connections, and file system activity to detect threats at runtime than only at scan time.

At the backend level, modern AV platforms maintain centralized infrastructure including threat intelligence aggregation, model training pipelines, analyst review workflows and a global telemetry network that pools anonymized detection data from millions of endpoints to detect emerging threats and update models. For example, Symantec's Intelligent Threat Cloud Service (ITCS), receives metadata or file submissions from clients when on-device ML models are uncertain, then performs deeper on cloud analysis, and returns a verdict that may

override the initial client verdict. This use of a hybrid client-cloud architecture enables near-real-time detection of new threats without requiring that every endpoint maintain a locally complete knowledge base.

7.2. Integration of machine learning models into antivirus software

The integration of machine learning into commercial AV systems has changed how detection decisions are made, shifting the workflow from exact pattern matching against manually made databases toward data-driven statistical inference that can generalize to previously unseen threats.

Static ML-based AV components analyze file content before execution, typically extracting features from PE headers, byte n-gram distributions, import tables, section entropy values and string patterns, then classifying the resulting feature vector using a trained model. In 2018 EMBER (Endgame Malware Benchmark for Research) was introduced, a labeled benchmark dataset of 1.1 million PE files with pre-extracted features, alongside a gradient-boosted decision tree model trained with LightGBM that achieves a Receiver Operating Characteristics – Area Under the Curve (ROC-AUC) of 0.9991 on the test set. The EMBER dataset and baseline model have since become a standard reference point for evaluating new static ML-based malware detection approaches, enabling reproducible comparison across research publications. Commercial AV products have adopted similar approaches [71].

Beyond static analysis, ML models have been integrated into the behavioral monitoring layer of AV systems, where they analyze sequences of system calls, API invocations, network connections and file system modifications at runtime. This approach maintains a continuously updated model of normal process behavior and flags deviations that match patterns associated with malware activity [70].

Next-Generation Antivirus (NGAV) references a category of endpoint protection solutions that moved beyond signature-based detection to rely primarily on ML, behavioral analysis and cloud intelligence. It is defined as a cloud-based technology using AI, ML, and behavioral analysis to protect endpoints against malware and other threats, capable of detecting unknown malware and malicious behavior in almost real time. The main characteristics that distinguish NGAV from traditional AVs are signatureless detection where threats are identified by behavioral patterns and statistical models, rather than exact byte signatures; zero-day threat prevention by using advanced heuristics, sandboxing and ML are used to proactively identify threats with no prior signatures; automated response and remediation using AI-driven automation that enables automatic isolation of infected endpoints and rollback of malicious changes without human intervention [72].

7.3. Adaptive Detection and Response Systems

The nature of the malware threat domain that is characterized by continuous mutation, polymorphism and the emergence of entirely new threat categories, creates a requirement on AV systems to adapt rather than classify based on a static model trained at a fixed point in time.

Adaptive malware detection systems incorporate mechanisms for continuous model updating in response to new threat observations. These systems are using an Adaptive Machine Learning Scalable Framework that evaluates different classifiers and proves that the models are able to continuously improve as new labeled data becomes available, making them suitable for dynamic threats by supporting real-time detection [73].

Concept rift means the degradation of model performance as the malware distribution evolves and it represents the primary challenge for adaptive ML-based AV systems. Concept drift is dealt with through multiple

strategies such as periodic full retraining on updated datasets or online learning algorithms that update model weights incrementally as new examples appear. The automated retraining infrastructure and overall adaptivity are as critical to a systems effectiveness as the initial model architecture [73].

Modern adaptive AV and EDR systems incorporate automated response capabilities that execute containment actions without waiting for human review. Upon a high confidence detection of a malicious event, an automated response module may isolate the affected endpoint from the network, suspend the process and roll back file system changes, all within seconds of detection. This shift from detection-only to detect-and-response architectures reflects that the speed of modern malware propagation makes traditional AVs response times insufficient for minimizing the damage [70].

7.4. Limitations and security considerations

Despite significant advances in ML-based and adaptive AV architectures, contemporary antivirus systems retain several limitations that constitute active areas of research and practical concern.

The most important vulnerability of ML-based AV systems is their susceptibility to adversarial attacks, that are intentionally crafted malware samples designed to evade classifier detection while preserving malicious functionality.

Another operational limitation of AV systems is the generation of false positives, where benign files are classified as malicious, which disrupts normal user operations, destroys trust in the AV and creates alert fatigue that reduces analyst responsiveness to real threats.

One other vulnerability is that ML-based AV models are constrained by the training data they were built on. Models trained on historical datasets, even large ones like EMBER, may fail to generalize to new malware families, targeted attacks or threat categories that did not exist when the training data was collected [71].

The last vulnerability that will be discussed here is that cloud integrated AV systems that collect and transmit file metadata or full file samples to external frameworks introduce privacy concerns, particularly in companies or governmental institutions. Organizations handling sensitive information may be constrained from enabling full cloud usage. Balancing detection effectiveness against privacy is an ongoing problem in this field.

8. Dataset Construction and Preparation

8.1. Dataset Design Acquisition

The practical component of this thesis required a dataset that supports both binary classification and malware family classification. For this reason, the dataset was designed to contain representative malicious Windows executables grouped into four malware categories: ransomware, trojan, Remote Access Trojan (RAT) and worm, together with a separate benign class used both as a control group and as a negative class for binary detection.

The dataset construction strategy followed a simple workflow. Instead of relying on an already made image dataset, raw executables were collected first, then transformed into entropy-based visual representations and other useful static descriptors, which ensured full control over the preprocessing pipeline used later for model training and application development.

8.2. Malware Sample Acquisition

Malicious samples were gathered manually with a custom collection script that queried the MalwareBazaar API and downloaded PE samples using predefined tag groups for each malware category. The script defined four target categories, ransomware, trojan, RAT and worm, and associated each class with multiple family names and aliases such as LockBit, WannaCry, GandCrab, Emotet, QakBot, Remcos, NanoCore, Conficker and Sality in order to improve category coverage beyond a single generic tag.

In order to maintain category balance, in the script it was imposed a cap of 500 samples per category and used a per-tag query limit of 50 returned items. Only files identified as executable PE-related types were downloaded, such as .exe, .dll, .pe32, and .pe32+, while duplicate files were filtered through a seen set function before download. Also, the downloads were spaced 0.7 seconds apart to respect API rate limits.

The download phase extracted password-protected MalwareBazaar archives using their standard password (“infected”), after which the contained samples was renamed using a SHA-256 prefix and stored inside the corresponding category directory. The naming modifications were added in order to simplify later matching between raw files, entropy images, and CSV feature records.

8.3. Benign Sample Acquisition

Because the project also targets malware-versus-benign discrimination, a separate benign dataset was created using a similar script to the one for the malware. This script searched trusted Windows system and program directories, including System32, SysWOW64, Program Files, and Program Files (x86), and copied executable files that satisfied the constraints between 20 KB and 50 MB.

A target of 100 benign files was used in the script configuration for the multiple malware category predictor, and also a target of 2000 benign files was used for the malware-versus-benign only dataset in order to maintain category balance. The files were randomly shuffled before selection, and a filename collision handling was implemented in order to avoid overwriting when two executables shared the same name. This approach created a complete and trusted benign dataset as all the files are drawn mainly from the local Windows software ecosystem.

This approach also comes with an important limitation as the files that are mainly from the local Windows ecosystem may not cover the full-diversity of legitimate executables encountered in daily use.

8.4. Dataset Organization

After collection, the dataset was organized into class-specific directories so that the parent folder of each file directly encoded its label. This decision was reused through later preprocessing scripts, where category names were determined from folder names and then used to generate per-class CSV files and image datasets.

This organization reduced labeling complexity and supported a simple end-to-end workflow: raw sample collection, entropy image generation, static feature extraction, model training and application interference all depended on consistent filename base name and category directories (shown in Figure 3).

Table 4. Dataset Organization

Class Label	Category	Numeric Label	Example Families	Source	Approx. Count
ransomware	Ransomware	0	Lockbit, WannaCry, REvil	MalwareBazaar API (tag-based query)	500
trojan	Trojan	1	Emotet, Dridex, Trickbot, RedLine	MalwareBazaar API (tag-based query)	500
rat	Remote Access Trojan	2	Remcos, PlugX, QuasarRAT, AsyncRAT	MalwareBazaar API (tag-based query)	485
worm	Worm	3	Conficker, Sality, Remnit, Gamarue	MalwareBazaar API (tag-based query)	254
benign	Windows PE	4	-	Windows directories	500

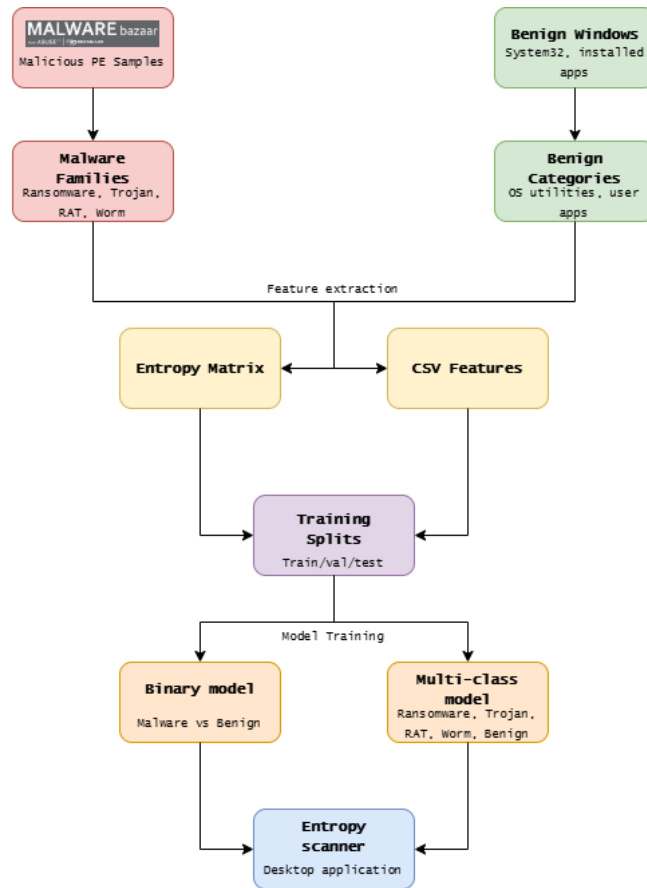


Figure 3. Full Project Workflow

8.5. Dataset Preparation Methodology

A major design choice in this work was to process only the first 16KB of each executable during entropy-based analysis. This reduced computational cost and standardized the representation length for all files, regardless of their total size. Smaller files were zero-padded to the same length, ensuring that every sample could be represented by the same 8×8 entropy structure and the same seven static descriptors.

The uniformity is essential because the classifier expects a fixed-length feature vector. The used representation may omit discriminative information stored deeper in the file, but it enables a lightweight and repeatable preprocessing pipeline.

9. Feature Engineering and Visual Representation

9.1. Entropy matrix generation

THE first stage of feature engineering transformed each executable into a compact entropy-based image. For every file, the first 16KB were read and split into 64 consecutive chunks of 256 bytes each. The Shannon entropy of each chunk was then computed, normalized by division by 8 and converted into a color using a green-to-yellow-to-red scale, where lower entropy regions appeared greener, and higher entropy regions appeared redder, as shown in Figure 4 and 5.

The resulting 64 color values were arranged in an 8×8 RGB image and the upscaled for display purposes using nearest-neighbor scaling. In methodological terms, the logical representation remained an 8×8 grid, while the enlarged image served only to improve human readability in the user interface and exported illustrations.

The use of entropy as the primary visual encoding was motivated by the fact that packed, encrypted, compressed or obfuscated regions usually exhibit higher entropy compared with ordinary code or data sections. Although entropy alone cannot uniquely identify malware, it provides a compact structural signal that can be exploited both visually and statistically.

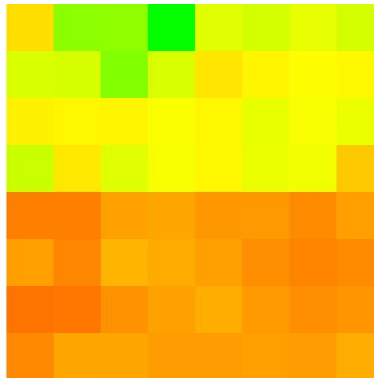


Figure 4. Benign Entropy Matrix



Figure 5. Malware Entropy Matrix

9.2. Additional Static Features

Because the initial visual representation alone did not deliver satisfactory multi-class discrimination, the feature space was extended with additional hand computed static descriptors. A second preprocessing script extracted seven numerical features from the same 16KB analysis window:

- Logarithm of file size: $\log_file_size(x) = \ln(1 + |x|)$
- Total window entropy: $WindowEntropy(x) = H(\omega)$
- Mean chunk entropy: $MeanChunkEntropy(x) = \frac{1}{64} \sum_{i=1}^{64} H(c_i)$
- Standard deviation of chunk entropy: $StdChunkEntropy(x) = \sqrt{\frac{1}{64} \sum_{i=1}^{64} (H(c_i) - \mu)^2}$
- Maximum chunk entropy: $MaxChunkEntropy(x) = \max_{1 \leq i \leq 64} H(c_i)$
- Minimum chunk entropy: $MinChunkEntropy(x) = \min_{1 \leq i \leq 64} H(c_i)$
- Ratio of the chunks whose entropy exceeded a threshold of 7.0: $HighEntropyRatio(x) = \frac{1}{64} \sum_{i=1}^{64} I_i$

Where x is a file with $|x|$ bytes, ω is the first 16 KB window that is split in 64 chunks $c_1, \dots, c_{64}$, each of 256 bytes and $H(c_i)$ is the Shannon entropy of the chunk c_i in bits per byte.

These features complement the visual matrix in two ways. First, they summarize global distributional properties that may not be captured correctly when the entropy grid is flattened into raw RGB values. Second, they introduce more interpretable statistical indicators that can help a tree-based model separate broad behavioral patterns such as uniformly packed files, mixed-structure binaries or low entropy benign executables.

9.3. Combined 199-Dimensional Representation

The final training representation used by both Random Forest classifiers was a concatenation of image-derived and static features. Each entropy image was loaded in RGB format, resized back to 8×8 , and flattened into 192 numerical values, corresponding to 64 cells $\times$ 3 color channels. These 192 values were concatenated with the seven handmade static features, providing a 199-dimensional feature vector per sample.

This combined representation is important to this thesis. Instead of choosing between purely image-based analysis and purely scalar-feature analysis, the system integrates both modalities into a single lightweight vector that remains compatible with classical machine learning methods such as Random Forests.

9.4. Feature Alignment and Data Integrity

A practical challenge in the pipeline was ensuring that every entropy image matched the corresponding static feature row. This was solved by aligning files through a shared filename stem where the image filename without the extension had to match the filename stem stored in the per-category CSV file.

Samples without a matching pair were skipped during dataset assembly. The design is simple but it is important to be documented as it directly affects the effective dataset size during training and ensures that the final models are fitted only on consistent image-feature pairs.

The visual and numerical representation chosen in this thesis intentionally favors efficiency, reproducibility, and compatibility with a practical desktop application. It does not attempt to recover instruction semantics, control flow or imported API behavior. Instead, it captures the whole structural regularities from a file

window and uses them as a basis for supervised classification. This makes the method suitable for fast static pre-screening. At the same time, the design imposes constraints like information that may be outside the first 16KB and entropy patterns may overlap across malware families or between packed malware and benign compressed software.

10. Model Training and Evaluation

10.1. Binary Classification Objective

THE first predictive model was designed to as a binary classifier that decides whether a file is malicious or benign. In this stage, all malware families were merged into a single malicious class, while the benign executables formed the negative class. This formulation reflects the most fundamental requirement of a malware scanner, that being determining whether a selected file should be considered suspicious before attempting a more clear family attribution.

The binary training set was built from entropy images stored under malware family folders together with the benign folder, and each sample was represented through the 199-dimensional combined feature vector described previously. The script confirms through output 192 image features and 7 static features, confirming the extract structure of the input fed to the classifier in the case of dataset related errors.

10.1.1. Binary Model Configuration

The binary classifier used a Random Forest with 300 trees, maximum tree depth of 10, balanced class weighting, a fixed random state of 42 and parallel execution across available cores. The use of balanced class weights is particularly relevant because the benign class was smaller than the composed malicious set and this parameter reduces the risk that the classifier would favor the majoritarian class.

The dataset was split using a stratified 70/15/15 partition into training, validation and test subsets. Stratification preserved class proportions across the three subsets and improved the reliability of later evaluation results.

10.2. Multi-Class Classification Objective

The second predictive model is a five-class classifier that predicts one of the following labels: ransomware, trojan, RAT, worm or benign. Unlike the binary model, this classifier had to separate malware families that often share overlapping structural properties, making the task substantially more difficult.

The multi-class model used the same 199-dimensional representation as the binary model, which ensured more consistency between the two experiments. This allowed a direct interpretation of the observed performance difference as any accuracy gap comes primarily from the increased task difficulty rather than from a change in preprocessing or feature design.

10.2.1. Multi-Class Model Configuration

The second Random Forest was also trained with 300 trees, but used a deeper configuration with maximum tree depth of 15, square-root feature sampling, balanced weights and the same fixed random seed. Increasing the maximum depth made sense in this context because family-level classification requires more complex decision boundaries than binary malware detection. Balanced weights were used because the sample size for worms was almost half of the others due to their rarity on MalwareBazaar.

The same stratified 70/15/15 split strategy was retained for the multi-class training. This consistency makes the two models directly comparable from an experimental design perspective.

10.3. Evaluation Artifacts Generated During Training

The training scripts were designed not only to fit models but also to produce evaluation artifacts suitable for discussion in this thesis. For the binary model, the generated plots include confusion matrices, normalized confusion matrices, pre-class precision/recall/F1 plots, ROC curves, precision-recall curves, probability distribution histograms and top feature importance plots. For the multi-class model, the generated outputs included confusion matrices, normalized confusion matrices, per-class metrics plots and top feature importance rankings.

This step was important as there can be a direct reference to these outputs as evidence rather than describing performance only in text. The visual artifacts will be included in one of the following chapters where they will be discussed more thoroughly.

11. Application Design and Architecture

11.1. Application Purpose and Design Goals

THE practical outcome of the thesis is a desktop application called Entropy Malware Scanner, developed to operationalize the preprocessing and classification pipeline in an end-user setting. The application allows a user to select a local file, generate the same entropy-based representation used at training time, classify the file with the trained models and visualize the resulting entropy matrix together with the predicted label and confidence. After discovering that a file is malicious, the user may choose to permanently delete that file from their computer.

A key design objective was consistency between training and deployment. The application reconstructs the exact preprocessing logic used during model creation, rather than implementing a simplified or approximate inference pathway. This is important because even minor preprocessing mismatches can cause model degradation at deployment time.

11.2. User Interface Structure

The application is implemented as a graphical desktop interface built with CustomTkinter. The window is divided into a left-side control area and a right-side visualization/result area. The control side contains the file-selection workflow, the scan trigger, explanatory text, the button that triggers file deletion and a button that clears the cache. The right side shows the computed 8×8 entropy matrix and two result panels: one for the malware-versus-benign decision and one for the family prediction, both with confidence scores (Figure 6).

This double result design improves usability because it separates the broad security verdict from the finer categorization. In practice, a user can immediately see both whether the file is considered malicious and, if so, which family the model regards as most likely.

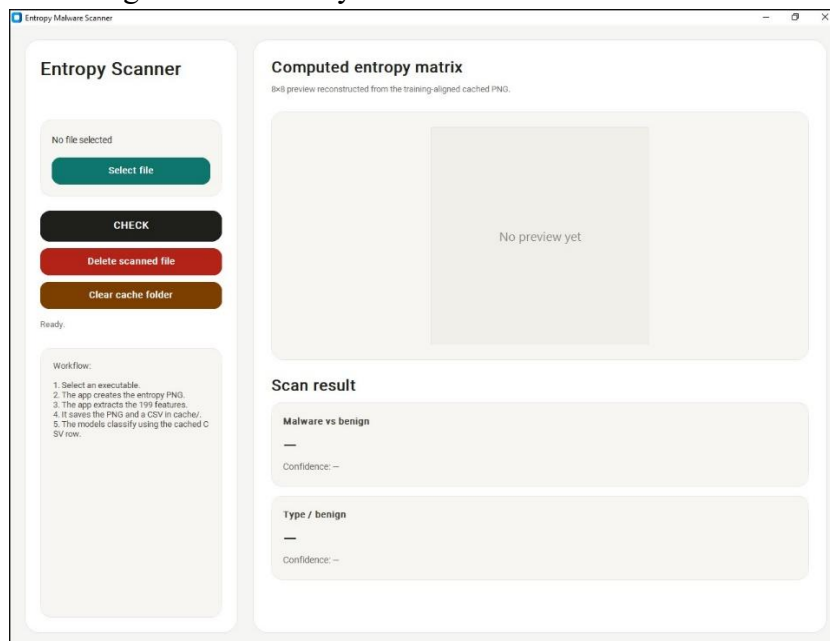


Figure 6. Main Application Window

11.3. Training-Aligned Inference Pipeline

When a user selects a file and starts a scan, the application first builds the entropy image using the same 16 KB window, 64 chunks, Shannon entropy computation, normalization and green-to-red RGB mapping used during dataset preparation. The resulting image is cached locally and then reopened, converted to RGB, resized to 8×8 , flattened and concatenated with the same seven static features used during training.

This step is important in the application logic as it is an exact reconstruction of the training-time 199-feature representation. This design was chosen because it minimizes discrepancy between offline model training conditions and real-time use in the final scanner.

11.3.1. Use of Cached Artifacts

During scanning, the application stores the generated entropy PNG and the extracted feature vector as CSV files inside a cache directory. This creates a transparent and inspectable intermediate representation that can be used for debugging, demonstration or manual verification. It also gets rid of all the problems that may appear because of discrepancy between model training and application usage. After the user decides that he doesn't need the cache files anymore, or the cache folder became too large, they can just press the dedicated button and clear all the saved cache data.

In this case the cache mechanism is a useful implementation detail because it helped a lot in debugging as the intermediate entropy visualization and feature row could be both inspected after analysis, making the results easier to validate.

11.4. Prediction Workflow and Result Presentation

Once the cached feature vector row is created, the application loads it and feeds the same feature vector into both trained models. The binary model returns a malicious/benign decision, while the multi-class model returns one of the five class labels together with class probabilities used to compute confidence values.

The user interface updates the entropy matrix preview and displays the predicted labels with color-coded styling that resembles the one used for the dataset creation. Malicious outcomes are shown in red, a warning color, and the benign outcomes are displayed in green. After receiving the result, the user can choose to delete to permanently delete the scanned file (Figures 7 and 8).

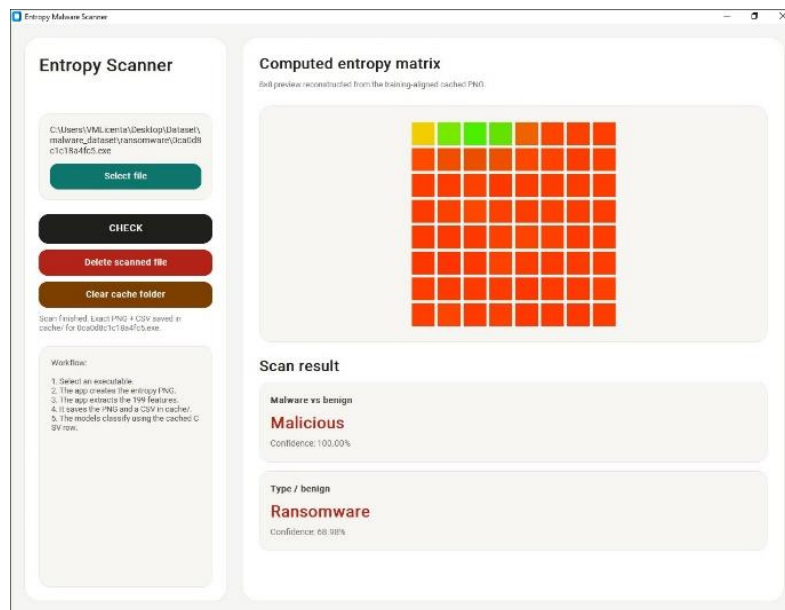


Figure 7. Malware File Scanned

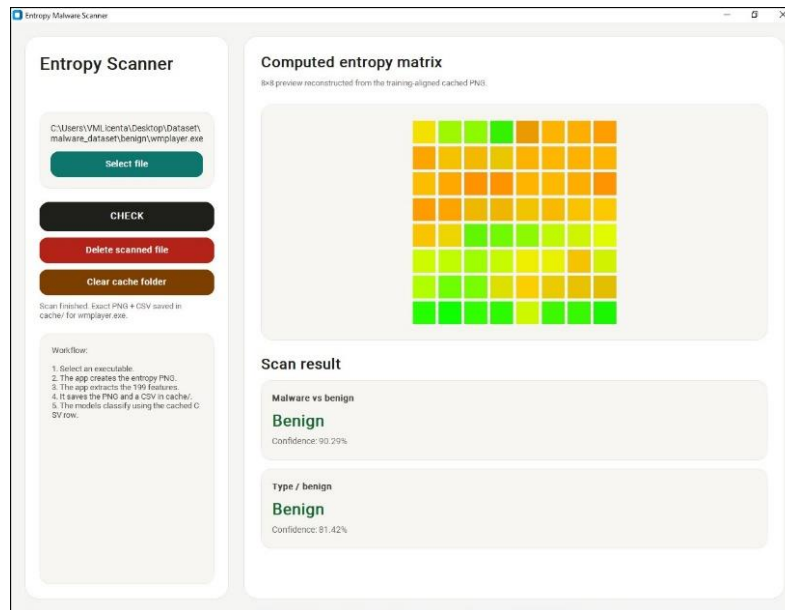


Figure 8. Benign File Scanned

11.5. Practical Characteristics of the Prototype

The final application is a static-analysis prototype intended for local file scanning. It does not execute files, emulate behavior or inspect runtime system activity. Its strengths lie in speed, low complexity, deterministic preprocessing and the visual interpretability of the entropy matrix representation.

Its limitations also follow directly from this design. Because the model relies on a short file prefix and handcrafted entropy-derived features, it may miss threats whose distinctive characteristics lie beyond the initial 16 KB or that are carefully crafted to mimic benign entropy profiles.

12. Experimental Results and Interpretation

12.1. Binary Classification Results

THE binary Random Forest, trained to distinguish malicious executable from benign ones, achieved an overall accuracy of approximately 95% on the test partition. This result was obtained on a stratified 70/15/15 train/validation/test split, which ensured that each split preserves the class distribution of the full dataset, preventing biasing caused by accidental class imbalance. The dataset used for binary training assigned numeric label 0 to all samples belonging to the four malicious categories (ransomware, trojan, RAT, worm) and numeric label 1 to benign executables, reducing the multi-family structure to a single discrimination boundary.

The 95% accuracy is particularly meaningful given the asymmetry in dataset size as the malware collection was targeted to 500 samples per malicious category, while the benign set, even though it was targeted to 2000 samples, it was able to collect only approximately 1000 samples from the Windows directories. To compensate for this imbalance, the Random forest was configured with balanced class weights which internally adjust the contribution of each sample during training inversely proportional to its class frequency. Without this correction the classifier would tend to predict the majoritarian class more aggressively, inflating raw accuracy while degrading recall on the minority benign class.

From a theoretical point of view, the result validates the core hypothesis of this thesis: that the first 16 KB of a PE-format executable, when encoded as an 8×8 RGB entropy color matrix and augmented with seven global statistical features, produces a feature space that a classical classifier can exploit effectively without executing the file. The specific 199 features consist of 192 values derived from the flattened image ($8 \text{ rows} \times 8 \text{ columns} \times 3 \text{ RGB channels}$) and 7 values derived from the static feature extraction pipeline.

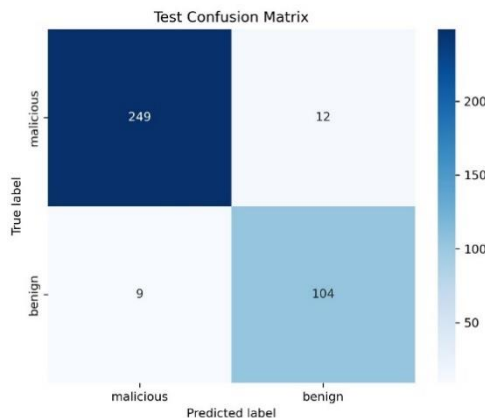


Figure 9. Binary Test Confusion Matrix

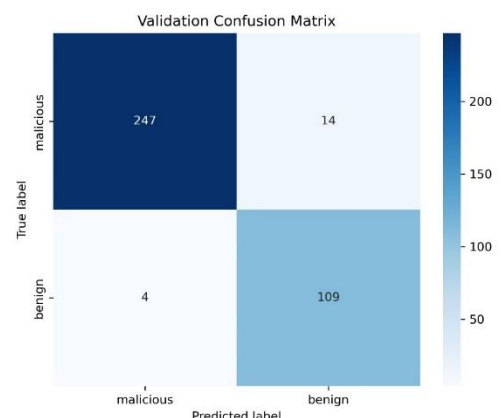


Figure 10. Binary Validation Confusion Matrix

12.2. Binary Metrics and Threshold Behavior

Beyond raw accuracy, the binary training pipeline generated a complex set of evaluation outputs that provide a multi-dimensional view of classifier performance. Understanding these outputs is essential for characterizing not only the number of correct predictions, but also the confidence, calibration and error asymmetry of the model.

The normalized confusion matrix (Figures 11 and 12) expresses each row as a fraction of the total true instances for that class, rather than as raw counts. This view makes it easy to compare recall rates across the two classes independently of class size differences. In this thesis, the malicious class is larger than the benign class and the normalized matrix reveals whether the model achieves comparable recognition for both classes or whether the smaller benign pool is handled less reliably.

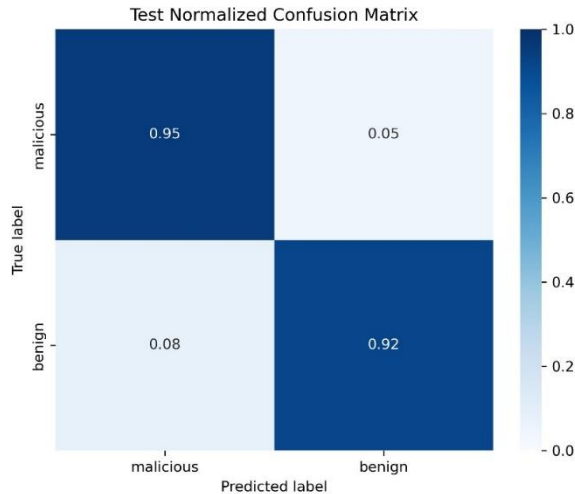


Figure 11. Binary Test Normalized Confusion Matrix

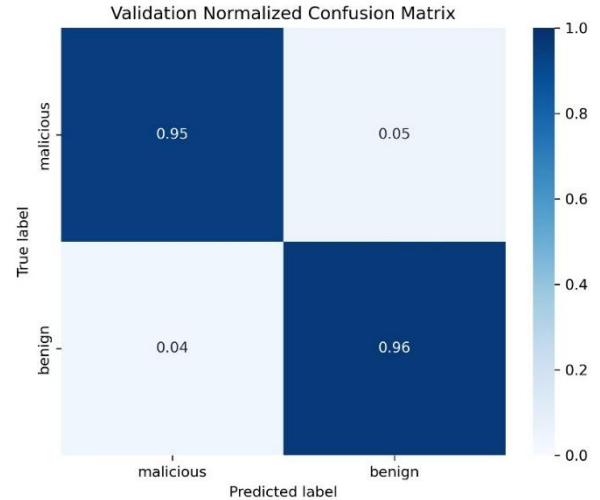


Figure 12. Binary Validation Normalized Confusion Matrix

The Receiver Operating Characteristics (ROC) curve plots (Figures 13 and 14) the true positive rate against the false positive rate at every possible decision threshold. Because the binary model outputs a continuous probability for the malicious class rather than a hard label, the decision boundary can be shifted to trade off between sensitivity (catching more malware at the cost of more false alarms) and specificity (generating fewer false alarms at the cost of missing some malware). The area under the ROC curve (AUC) is a threshold-independent summary of this trade-off where values close to 1 indicate that the model can cleanly rank malicious files above benign ones across all operating points. The training script computes both ROC curve and AUC from scikit-learn using the probability of the malicious class as the score.

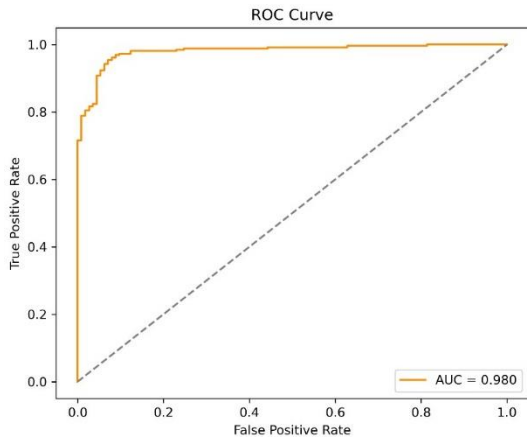


Figure 13. Binary Test ROC-AUC Curve

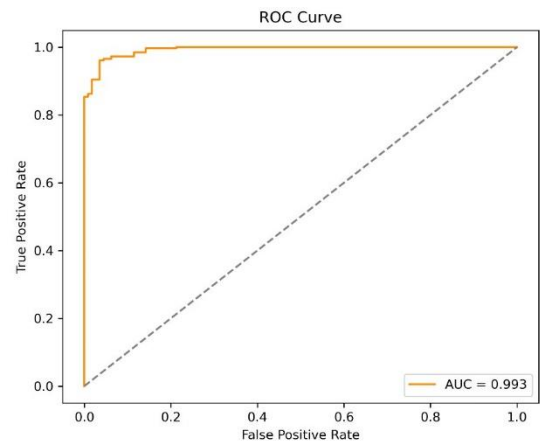


Figure 14. Binary Validation ROC-AUC Curve

The precision-recall (PR) curve (Figures 15 and 16) is a complementary view that is particularly informative when the positive class (malware) is the operationally important one. Precision measures the proportion of predicted malware files that are genuinely malicious; recall measures the proportion of all actual

malware samples that were caught. The average precision (AP) score summarizes the PR curve as a single number and is comparable to the AUC in interpretive value. A high AP confirms that the classifier maintains strong malware detection even when the decision threshold is tightened to reduce false alarms.

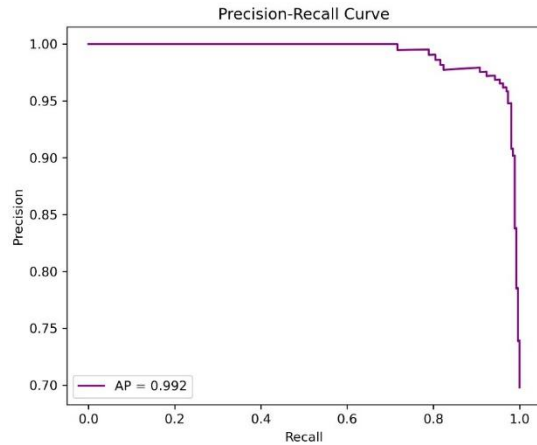


Figure 15. Binary Test Precision-Recall Curve

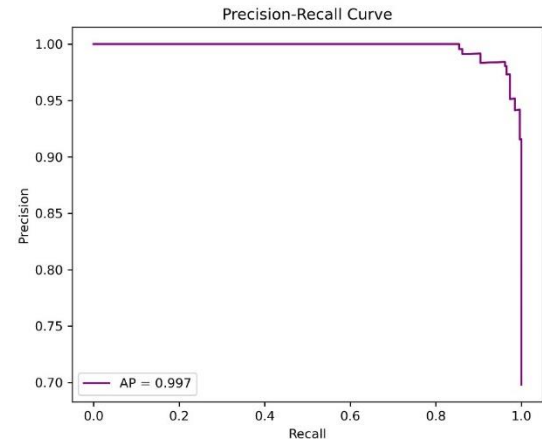


Figure 16. Binary Validation Precision-Recall Curve

The per-class precision/recall/F1 chart (Figures 17 and 18) presents all three metrics side by side for both the malicious and benign classes. This visualization is useful for identifying whether errors are symmetric or concentrated on one class. For example, a model with high precision but low recall on the benign class would be generating many false positives.

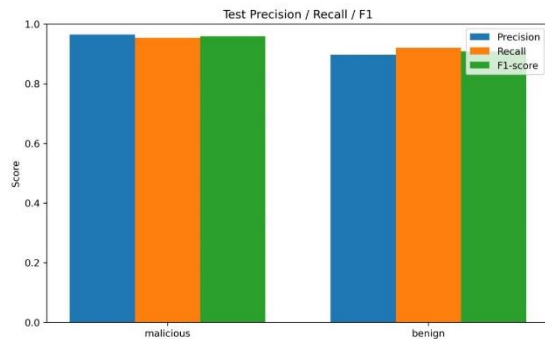


Figure 17. Binary Test Precision/Recall/F1-Score

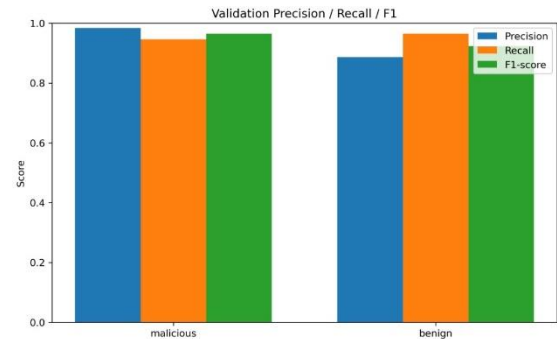


Figure 18. Binary Validation Precision/Recall/F1-Score

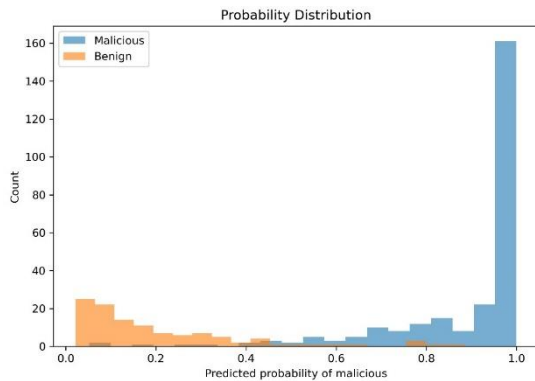
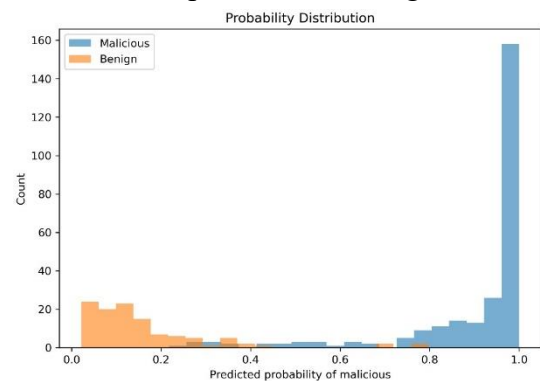
Table 5. Test Precision, Recall, F1-Score

	Precision	Recall	F1-Score	Support
Malicious	0.97	0.95	0.96	261
Benign	0.90	0.92	0.91	113
Accuracy			0.94	374
Macro Average	0.93	0.94	0.93	374
Weighted Average	0.94	0.94	0.94	374
Accuracy		0.9438502673796791		

Table 6. Binary Validation Precision, Recall, F1-Score

	Precision	Recall	F1-Score	Support
Malicious	0.98	0.95	0.96	261
Benign	0.89	0.96	0.92	113
Accuracy			0.95	374
Macro Average	0.94	0.96	0.94	374
Weighted Average	0.95	0.95	0.95	374
Accuracy		0.9518716577540107		

The probability distribution histogram (Figures 19 and 20) overlays two distributions: the predicted malicious-class probability for files that are actually benign. In a well-calibrated, confidently separating classifier, these two distributions cluster near 1.0 and 0.0 respectively with minimal overlap in the middle range.

**Figure 19.** Binary Test Probability Distribution**Figure 20.** Binary Validation Probability Distribution

12.3. Multi-Class Classification Results

The second trained model addresses the more difficult task of predicting the specific malware family, or confirming that a file is benign, from the same 199-dimensional feature vector. The five classes are: ransomware (label 0), trojan (label 1), RAT (label 2), worm (label 3), benign (label 4). This classifier achieved an overall accuracy of approximately 75% on the test set.

The almost 20% drop relative to the binary model is an expected outcome. In the binary setting, the model only needs to identify a single boundary separating the entropy signature of benign Windows executables from any malicious file, regardless of its category. In the multi-class setting, the model must additionally discriminate between four malware families that may share many structural similarities. The entropy profile of a RAT and a trojan, for instance, may be nearly indistinguishable when both are delivered as compact, packed executables.

As mentioned before, the multi-class Random Forest was configured with 300 decision trees, a maximum tree depth of 15, the square root feature selection strategy at each split and balanced class weighting.

The increased tree depth relative to the binary classifier reflects the grater complexity of the five-class boundary and the additional discriminative capacity required. A single minimum-leaf-size constraint was applied, allowing the trees to grow to full depth and capture complex patterns in the training data.

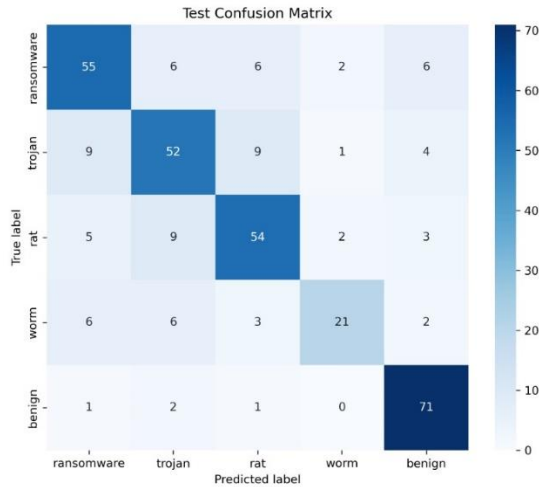


Figure 21. Multi-Class Test Confusion Matrix

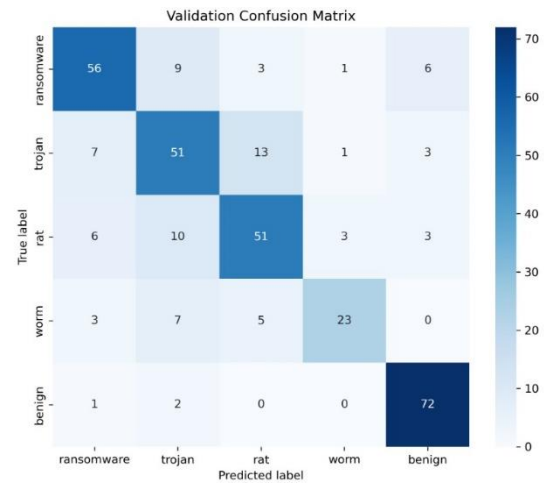


Figure 22. Multi-Class Validation Confusion Matrix

12.4. Per-Class Interpretation

The per-class metrics produced by the multi-class evaluation pipeline are essential for understanding where the 75% average accuracy originates and where it falls short. Total accuracy conceals the possibility that certain classes are nearly perfectly recognized, while others are classified at almost random levels.

Among the four malware families, ransomware is the category that exhibits the most distinctive entropy profile in the first 16KB. Ransomware families such as LockBit, Conti, WannaCry and Revil typically include an encryption routine in their early executable sections. If that encryption routine is packed or obfuscated, the opening bytes exhibit uniformly high entropy. If the early sections contain mostly code, they may exhibit moderate but consistent entropy. Either one of those two patterns is relatively stable across variants of the same family, making ransomware an easier target for entropy-based classification.

Remote Access Trojans (RATs) such as Remcos, NanoCore, njRAT and AsyncRAT are typically lightweight executables with compact early sections. Many RAT families are compiled with .NET or similar managed runtimes, which produces a characteristic low-to-moderate entropy header region followed by a high entropy code section. However, because many different RAT families share the same managed runtime structure, distinguishing one RAT from another based solely on the first 16 KB is challenging. The classifier may correctly identify a file as a RAT-like pattern while attributing it to the wrong specific label.

Trojans represent the most diverse category in the dataset, containing banking malware (Emotet, Dridex, TrickBot), stealers (AgentTesla, RedLine, Vidar) and general-purpose loaders. This diversity means the intra-class entropy variance is high, which makes the decision boundary worse than for more uniform families. The classifier is expected to show lower precision and recall for the trojan class than for more structurally coherent families.

Worms in the dataset include self-propagating executables such as Conficker, Sality, Ramnit and Gamarue. Many older worm families are not heavily packed and exhibit relatively flat entropy profiles in their

early sections. This may cause them to visually resemble certain benign executables, leading to cross-class confusion between worm and benign labels.

Inter-family confusions are not only explainable but also practically informative. For example, if the confusion matrix shows that the model frequently predicts trojan when the true label is RAT, this suggests that the two families share overlapping entropy distributions in the 8×8 feature space. This kind of finding motivates future work on section-aware feature extraction.

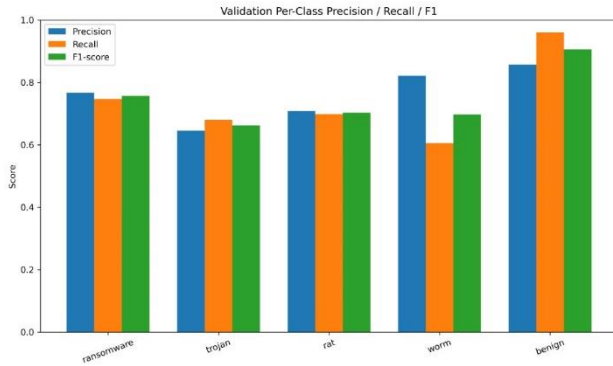


Figure 23. Multi-Class Test Precision/Recall/F1-score

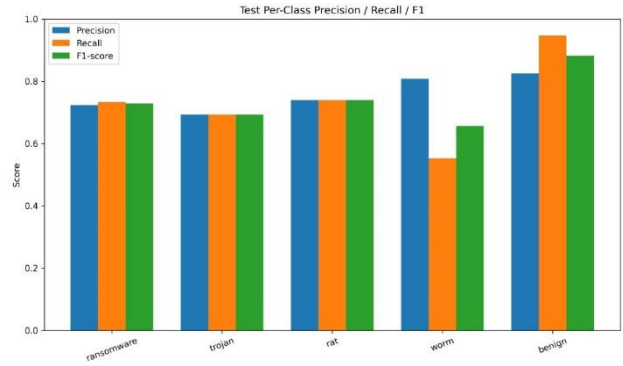


Figure 24. Multi-Class Validation Precision/Recall/F1-score

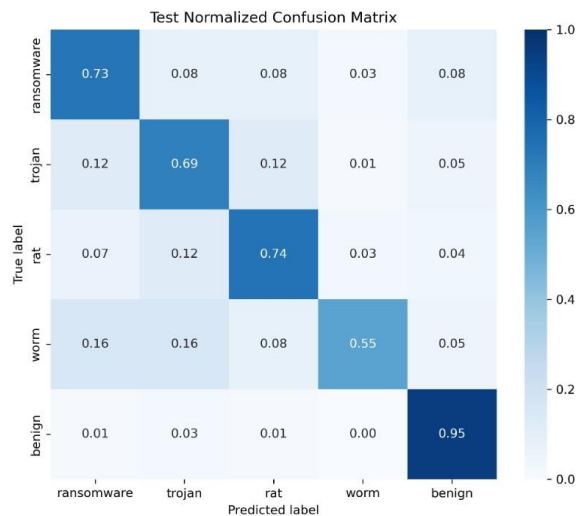


Figure 25. Multi-Class Test Normalized Confusion Matrix

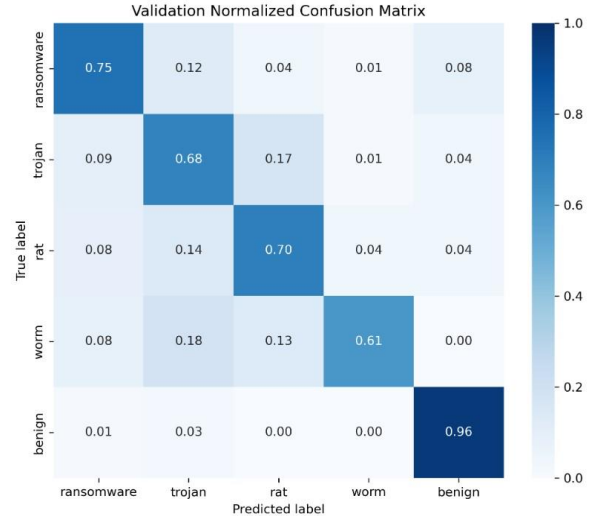


Figure 26. Multi-Class Validation Normalized Confusion Matrix

Table 7. Multi-Class Test Precision, Recall, F1-score

	Precision	Recall	F1-Score	Support
Ransomware	0.72	0.73	0.73	75
Trojan	0.69	0.69	0.69	75
RAT	0.74	0.74	0.74	73
Worm	0.81	0.55	0.66	38
Benign	0.83	0.95	0.88	75
Accuracy			0.75	336

Macro Average	0.76	0.73	0.74	336
Weighted Average	0.74	0.75	0.75	336
Accuracy	0.753			

Table 8. Multi-Class Precision, Recall, F1-Score

	Precision	Recall	F1-Score	Support
Ransomware	0.77	0.75	0.76	75
Trojan	0.65	0.68	0.66	75
RAT	0.71	0.70	0.70	73
Worm	0.82	0.61	0.70	38
Benign	0.86	0.96	0.91	75
Accuracy	0.75			336
Macro Average	0.76	0.74	0.75	336
Weighted Average	0.75	0.75	0.75	336
Accuracy	0.753			

12.5. Feature Importance Analysis

Both the binary and multi-class Random Forest models were fitted with full access to the 199-dimensional feature vector, and both training scripts extract and visualize the top 15 most important features according to the mean decrease in impurity criterion. Because feature labels are assigned as `img_0` through `img_191` for the image-derived channels and by column name (`log_file_size`, etc.) for the static descriptors, the importance plot directly reveals the relative contribution of entropy cells versus global statistical summaries.

The 192 image features correspond to the RGB channel values of the 64 cells in the 8×8 grid, laid out in row-major order with three channels per cell. Cell position in the flattened vector is meaningful, as features `img_0`, `img_1`, `img_2` correspond to the red, green, blue channel values of the top left cell, covering bytes 0-255, while `img_189`, `img_190`, `img_191` correspond to the bottom right cell, covering bytes 16.128-16.383. If the importance plot shows that the cells from specific positions dominate the ranking it is implied that the bytes in that region are the most discriminative region. The high entropy ratio is another very informative feature for packed malware, because packing typically elevates entropy globally rather than at specific positions.

The feature importance analysis (Figures 27 and 28) was produced for both models in order to allow a direct comparison of what each model relies upon. The binary classifier has a simpler discrimination task and depends on a small number of static features. The multi-class classifier, facing a harder task, uses a broader set of additional static features.

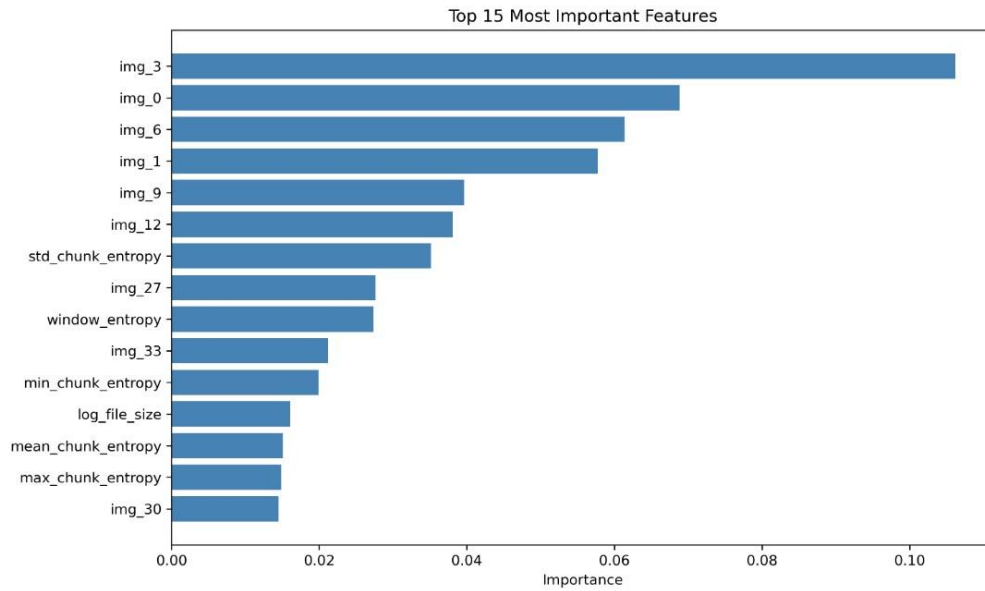


Figure 27. Binary Top Feature

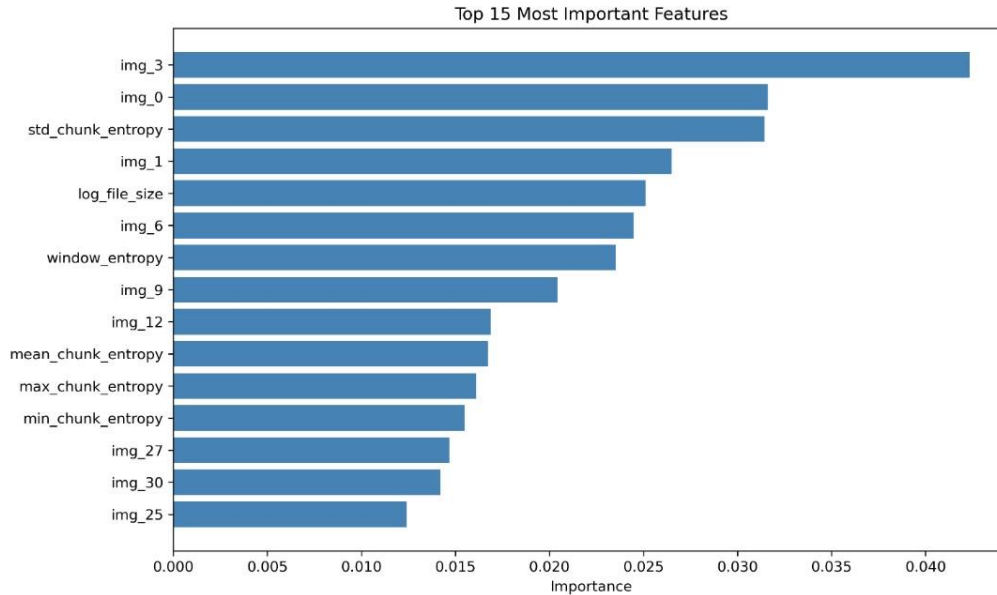


Figure 28. Multi-Class Top Feature

12.6. Discussion of Observed performance

Taken together, the two experiments reveal a clear and reliable performance pattern. The hybrid 199-dimensional representation built from an 8×8 entropy color matrix and seven global statistical descriptors is highly effective for broad malware-versus-benign triage, where the binary classifier reached approximately 95% accuracy, but substantially less reliable when the task requires distinguishing between four structurally similar malware families, where accuracy drops to almost 75-76%.

This pattern is fully consistent with the theoretical framework established in Chapters 3 and 4. The entropy color matrix encodes the spatial distribution of information density across the first 16 KB of the binary,

which differs a lot between benign Windows executables (which have moderate, structured entropy profiles) and malicious executables (which contain packed, encrypted or compressed sections). At the family level, however, all four malware classes share the basic property of malicious PE executables, and their intra-class entropy profiles may overlap, particularly when multiple families use the same packers, loaders or runtime environments.

The 75% multi-class accuracy should not be seen as a failure as it represents meaningful family attribution in a five-class problem. More importantly, the two-stage architecture of the application, where the binary model provides a high confidence malware/benign prediction and the multi-class model provides a likely family suggestion second, means that the weaker multi-class performance does not affect the primary security function of the scanner. A user who receives a “malicious – likely trojan” verdict benefits from both the confident binary flag and the approximate family label, even if the family label carries some uncertainty.

The practical implication of this result is that the current representation is well-suited fast triage, quickly identifying whether a file needs deeper investigation. It is not suited as a standalone replacement for dynamic analysis or reverse engineering but it is a good middle option that could assist more complex methods.

13. Limitations, Practical Reflections and Future Improvements

13.1. Dataset-Related Limitations

THE dataset was assembled using two related collection strategies, each of which introduces specific biases that are important to acknowledge. The malware samples were downloaded from the MalwareBazaar public repository using the API endpoint, which retrieves SHA-256 hashes of samples tagged with a specified label. Only PE-format files were retained and the downloads were spaced 0.7 seconds apart to respect API rate limits.

The tag-based collection strategy is practical and reproducible, but it introduces label noise proportional to the reliability of MalwareBazaar's community-contributed tagging. A file tagged, for example, "emotet" in the repository might be an Emotet downloader stub, a component DLL or even a misclassified sample. Because the reliability of family attribution was not independently verified through a secondary source such as VirusTotal consensus labels, the category assignments in the training set inherit the quality and consistency of the upstream tag metadata.

A second dataset related limitation is class size asymmetry between malware and benign sets and also between malware categories. While the malware downloader targeted up to 500 files per category (up to 2000 total malicious files), the benign collection script only managed to collect almost 1000 files from the Windows directories available in the Virtual Machine used. Although the balanced class weights mitigates this imbalance during training by upweighting benign samples, the lower number of benign examples limits the diversity of the benign representation.

A third limitation concerns temporal distribution as samples from MalwareBazaar tend toward recent submissions, while the benign files reflect the Windows installation on the development machine at the time of collection. This temporal mismatch may cause the scanner encounter problems in real usage as the training set does not represent the full variety of either old legacy malware or newly installed benign software.

13.2. Representation-Related Limitations

The proposed feature pipeline reads only the first 16.384 bytes (16 KB) of each target file. This design choice was made for more efficiency as the first 16 KB are sufficient to cover the PE header, The DOS stub, the NT headers, the section table and the opening portion of the code section in most executables. For small malware samples, the 16 KB window may cover a big part of the entire binary and is therefore a good representation. However, for larger executables, the window covers only a small prefix of the file and it may not reach important pieces that would represent malicious software.

The entropy-to-color mapping is also an indirect indicator, as Shannon entropy measures the degree of randomness in a byte sequence, not the specific nature of the content that produced that randomness. A high entropy may contain encrypted text, a compressed data block or a table of cryptographic constraints in a benign implementation. On the other way around, low entropy may contain repetitive null padding, readable ASCII strings or structured data tables in a malicious file. This means that entropy-based features are not always reliable without additional context from the surrounding structure of the file.

13.3. Model-Related Limitations

The Random Forest classifier was chosen for this project because of its strong performance on tabular and mixed-type feature spaces, its interpretability through feature importance, its resistance to overfitting through bagging and random feature selection and its practical ease of deployment through the JobLib library. These properties make it a good option for this type of projects. However, Random Forests have some model-level limitations in this context.

First, Random Forests behave similarly to instance-based classifiers that interpolate within the convex hull of the training data. They do not generalize well to structurally new samples that lie far outside the training distribution, such as completely new malware families, heavily obfuscated samples through uncommon packers, or benign executables from unfamiliar software ecosystems.

Second, the multi-class accuracy at approximately 75% suggests that 199 features derived from entropy statistics may not provide sufficient discriminative resolution to cleanly separate five classes with overlapping structural profiles. More expressive classifiers such as gradient-boosted trees or convolutional neural networks, working directly on the full entropy image, might improve multi-class accuracy at the cost of increased training complexity and hardware requirements.

Third, Random Forests are not robust to adversarial manipulation. A well-informed adversary who knows how the scanner works could pad the malicious content with low-entropy prefix, displacing the malicious region outside the analysis region.

13.4. Application-Related Limitations

The desktop application implemented in the final stage of the project is a static offline file scanner. It performs with no network communication, no process execution, no memory inspection, and no filesystem monitoring. When a user selects a file, the application reads the first 16 KB from disk, computes the entropy color matrix extracts the seven static features, assembles the 199-dimensional input vector, writes it to a cached CSV file, passes it sequentially to the binary and multi-class Random Forest Models. The entire inference pipeline runs locally in under a second on modern hardware.

The design has its advantages, such as no internet connectivity required, no cloud infrastructure and no elevated administrative privileges, but this also defines a clear ceiling on the types of threats it can detect. Fileless malware that resides entirely in memory and is never written to disk cannot be scanned by any static file analysis tool. Script-based attacks that do not produce PE-format executables are also outside its scope. Malware that executes entirely through legitimate signed binaries is invisible to the scanner.

13.5. Future Improvements

Several upgrades would strengthen the system beyond its current proof-of-concept scope. The most impactful improvements are as follows.

Dataset expansion and diversification. The most straightforward improvement is to increase both the quantity and diversity of training samples. The benign set should be expanded and drawn from a wider range of sources including third party software, open-source tools and portable utilities. The malware set should be augmented with more recent samples and, ideally, with samples whose tags have been independently validated.

Richer static feature engineering. The seven static features derived from chunk entropy statistics provide a useful global summary of the file's entropy profile, but they discard the structural information available in the PE header itself. Adding features derived from the PE header, such as the number of sections, section names and characteristics, the size of the import table, the number of imported DLLs, and the ratio of virtual to raw section sizes, would provide the classifier with richer signs that are robust to entropy camouflage.

Multi-window and section-aware analysis. Rather than extracting features from a fixed 16 KB prefix, a future version could divide the file into multiple overlapping windows or extract separate entropy matrices from individual PE sections. Each section has a distinct functional role, for example, the code section should have moderate entropy reflecting compiled machine code and the resource section may have high entropy if it carries encrypted or compressed payloads. A section-aware pipeline could detect anomalies in each region independently, improving sensitivity to a wider range of evasion strategies.

Improved multi-class discrimination through ensemble diversity. The approximately 75% multi-class accuracy plateau suggests that a single Random Forest is approaching the ceiling of the current 199-dimensional feature space. Replacing or augmenting the Random Forest with a gradient-boosted ensemble or a convolutional neural network operating on the 8×8 RGB image directly could improve multi-class accuracy.

Application extensions. On the user-facing side, the application could be extended to support batch scanning of entire directories, automated export of scan results to a structured report (PDF), a option that allows the user to quarantine the file instead of permanently deleting it. A scan history log would also be a valuable addition for users who wish to track which files on their system have been inspected and what verdicts were returned.

14. Conclusions

THIS thesis addressed the problem of static malware detection through the lens of binary visualization and classical machine learning, motivated by the fundamental limitations of traditional signature-based and heuristic antivirus systems when confronted with obfuscated, polymorphic and zero-day threats. The central research question asked whether a lightweight, non-execution-based representation, specifically, an entropy color matrix derived from the first 16 KB of a PE executable, could serve as the basis for an effective malware classifier deployable in a practical desktop application.

The practical work demonstrated that this question can be answered affirmatively, within clearly defined boundaries. Starting from a manually created dataset sourced from MalwareBazaar's public API and locally collected benign set, a complete analysis pipeline was implemented and validated: automated dataset construction across four malware families (ransomware, trojan, RAT, worm) plus a benign class; batch entropy image generation using a 64-chunk, 256-byte-per-chunk sliding window with a green-to-red RGB color mapping; static feature extraction producing seven aggregated entropy statistics per file; training and evaluation of a binary Random Forest for malware-versus-benign detection (around 95% accuracy); training and evaluation of a multi-class Random Forest for family attribution (around 75-76% accuracy); and deployment of both models in a desktop application.

The binary classification result confirms that the entropy color matrix, augmented with global statistical descriptors, captures a strong coarse-grained signal that reliably separates malicious from benign Windows executables. The 95% accuracy achieved under balanced class weighting, on a stratified partition and across four structurally diverse malware families provides meaningful empirical validation of the core hypothesis. The result is consistent with the literature revied in Chapters 3 and 7, which has established that entropy-based visualization is a robust and computationally efficient representation for malware analysis.

The multi-class classification result at 75% accuracy reveals the limits of the current representation at more difficult tasks. The entropy profile of the first 16 KB is informative enough to flag a file as malicious with high confidence, but not always distinctive enough to reliably attribute it to the correct malware family when multiple families share similar packing strategies, compilation toolchains or runtime environments. This finding is not a contradiction of the approach but a specification of its scope, more exactly, the system is designed and should be evaluated as a fast first-line triage tool, not as a complete malware attribution system.\

The desktop application successfully closes the loop between research prototype and a deployable tool. A user with no technical background in binary analysis or machine learning can easily use the application as they can select any executable file on their Windows System, receive a binary malware verdict and a likely family label within seconds, and visually inspect the entropy color matrix that underlies the inference. This immediacy and transparency constitutes a form of built-in interpretability that is absent from other neural network approaches operating on raw byte sequences.

The most important contribution of the practical part of this thesis is therefore a double one. First, it demonstrates that a fully functional, end-to-end malware scanner can be constructed using only open-source tools, publicly available malware repositories, classical machine learning and a feature engineering pipeline that requires no execution, no kernel access and no network connectivity. Second, it establishes a concrete, reproducible

baseline against which future improvements can be systematically measured and added. The boundaries identified through the limitation analysis in Chapter 13 are not considered weaknesses to be concealed but research directions to be pursued, and the clarity with which those boundaries have been articulated is itself a scientific contribution.

Bibliography

- [1] T. T. Son, C. Lee, H. Le-Minh, N. Aslam, and V. C. Dat, “An enhancement for image-based malware classification using machine learning with low dimension normalized input images,” *Journal of Information Security and Applications*, vol. 69, Sep. 2022, doi: 10.1016/j.jisa.2022.103308.
- [2] F. C. Carlo Garcia and F. P. Muga II, “Random Forest for Malware Classification.”
- [3] K. S. Han, J. H. Lim, B. Kang, and E. G. Im, “Malware analysis using visualized images and entropy graphs,” *Int. J. Inf. Secur.*, vol. 14, no. 1, pp. 1–14, Feb. 2015, doi: 10.1007/s10207-014-0242-0.
- [4] A. Pinhero *et al.*, “Malware detection employed by visualization and deep neural network,” Jun. 01, 2021, *Elsevier Ltd.* doi: 10.1016/j.cose.2021.102247.
- [5] K. Han, B. Kang, and E. G. Im, “Malware analysis using visualized image matrices,” *Scientific World Journal*, vol. 2014, 2014, doi: 10.1155/2014/132713.
- [6] K. Jaisinghani and S. Singh, “International Journal of INTELLIGENT SYSTEMS AND APPLICATIONS IN ENGINEERING Recent Advances in Image based Malware Classification through the Lens of Deep Learning-A Systematic Literature Review.” [Online]. Available: www.ijisae.org
- [7] S. Li, J. Wang, S. Wang, and Y. Song, “PAFE: A lightweight visualization-based fast malware classification method,” *Heliyon*, vol. 10, no. 16, Aug. 2024, doi: 10.1016/j.heliyon.2024.e35965.
- [8] J. M. A. Shaik, G. Pendharkar, S. Kumar, M. K. B, and S. Balaji, “Comparative Analysis of Imbalanced Malware Byteplot Image Classification using Transfer Learning,” Oct. 2023, doi: 10.1007/978-981-99-7216-6_26.
- [9] D. Vasan, M. Hammoudeh, and M. Alazab, “Broad learning: A GPU-free image-based malware classification,” *Appl. Soft Comput.*, vol. 154, Mar. 2024, doi: 10.1016/j.asoc.2024.111401.
- [10] H. Dasari, B. Pradhyumna Danduboina, and M. C. Rao, “Malware Prediction Classifier using Random Forest Algorithm,” 2022.
- [11] DharmendraThapa, Hari Narayan Ray Yadav, and Madhav Dhakal, “Malware Detection using the concept of Random Forest Algorithm,” *Journal of Science & Technology*, vol. 10, no. 5, pp. 38–49, May 2025, doi: 10.46243/jst.2025.v10.i05.pp38-49.
- [12] W. Nie, “Malware Classification Based on Image Segmentation,” Jun. 2024, [Online]. Available: <http://arxiv.org/abs/2406.03831>

- [13] “Entropy analysis,” 2007. [Online]. Available: <http://northfox.uw>.
- [14] M. I. Yousuf, I. Anwer, A. Riasat, K. T. Zia, and S. Kim, “Windows malware detection based on static analysis with multiple features,” *PeerJ Comput. Sci.*, vol. 9, 2023, doi: 10.7717/PEERJ-CS.1319.
- [15] L. Nataraj, “Malware Images: Visualization and Automatic Classification.” [Online]. Available: <http://vision.ece.ucsb.edu/~lakshman/Malware%20Images/album/index.html>
- [16] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, “Malware Images: Visualization and Automatic Classification,” 2010.
- [17] S. Kumar and B. Janet, “DTMIC: Deep transfer learning for malware image classification,” *Journal of Information Security and Applications*, vol. 64, Feb. 2022, doi: 10.1016/j.jisa.2021.103063.
- [18] A. Tripathi, “IJIRMPSS2504232662 Website: www.ijirmps.org Email: editor@ijirmps Evaluating the Role of Confidentiality, Integrity, and Availability in Cyber Defence.” [Online]. Available: www.ijirmps.org
- [19] S. Samonas and D. Coss, “THE CIA STRIKES BACK: REDEFINING CONFIDENTIALITY, INTEGRITY AND AVAILABILITY IN SECURITY.” [Online]. Available: www.jissec.org
- [20] A. Abusitta, M. Q. Li, and B. C. M. Fung, “Malware Classification and Composition Analysis: A Survey of Recent Developments.”
- [21] A. Hussain, S. F. Ahmad, M. Tanveer, and A. S. Iqbal, “Computer Malware Classification, Factors, and Detection Techniques: A Systematic Literature Review (SLR),” *International Journal of Innovations in Science and Technology*, vol. 4, no. 3, pp. 899–918, Aug. 2022, doi: 10.33411/ijist/2022040320.
- [22] E. Gandotra, D. Bansal, and S. Sofat, “Malware Analysis and Classification: A Survey,” *Journal of Information Security*, vol. 05, no. 02, pp. 56–64, 2014, doi: 10.4236/jis.2014.52006.
- [23] Y. Zhou, Y. Zhou, S. Chen, and O. P. Kreidl, “Limiting Self-Propagating Malware Based on Connection Failure Behavior,” Academy and Industry Research Collaboration Center (AIRCC), Dec. 2015, pp. 99–114. doi: 10.5121/csit.2015.51609.
- [24] A. Mustafa Abuzaid, M. Mohd Saudi, B. M. Taib, and Z. Hilmi Abdullah, “An Efficient Trojan Horse Classification (ETC),” 2013. [Online]. Available: www.IJCSI.org
- [25] C. Beaman, A. Barkworth, T. D. Akande, S. Hakak, and M. K. Khan, “Ransomware: Recent advances, analysis, challenges and future research directions,” *Comput. Secur.*, vol. 111, Dec. 2021, doi: 10.1016/j.cose.2021.102490.

- [26] S. Razaulla *et al.*, “The Age of Ransomware: A Survey on the Evolution, Taxonomy, and Research Directions,” *IEEE Access*, vol. 11, pp. 40698–40723, 2023, doi: 10.1109/ACCESS.2023.3268535.
- [27] M. Nadim, W. Lee, and D. Akopian, “Kernel-level rootkit detection, prevention, and behavior profiling: A taxonomy and survey.”
- [28] M. Naser, H. Al Bazar, and H. Abdel-Jaber, “Mobile Spyware Identification and Categorization: A Systematic Review,” *Informatica (Slovenia)*, vol. 47, no. 8, pp. 45–56, Sep. 2023, doi: 10.31449/inf.v47i8.4881.
- [29] *The 5th Conference on Information and Knowledge Technology : IKT*. IEEE, 2013.
- [30] A. V Kozachok, M. V Bochkov, and E. V Kochetkov, “Heuristic Malware Detection Mechanism Based on Executable Files Static Analysis.”
- [31] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, “A survey on automated dynamic malware-analysis techniques and tools,” Feb. 01, 2012, *Association for Computing Machinery*. doi: 10.1145/2089125.2089126.
- [32] Ramesh Prasad Pokhrel, “Behavioral analysis of malware using sandboxing techniques,” *International Journal of Science and Research Archive*, vol. 15, no. 3, pp. 582–586, Jun. 2025, doi: 10.30574/ijrsra.2025.15.3.1781.
- [33] E. Amer and I. Zelinka, “An ensemble-based malware detection model using minimum feature set,” *Mendel*, vol. 25, no. 2, pp. 1–10, 2019, doi: 10.13164/mendel.2019.2.001.
- [34] C. Catalano, A. Chezzi, M. Angelelli, and F. Tommasi, “Deceiving AI-based malware detection through polymorphic attacks,” *Comput. Ind.*, vol. 143, Dec. 2022, doi: 10.1016/j.compind.2022.103751.
- [35] D. Ucci, L. Aniello, and R. Baldoni, “Survey of Machine Learning Techniques for Malware Analysis,” Nov. 2018, doi: 10.1016/j.cose.2018.11.001.
- [36] R. Sihwail, K. Omar, and K. A. Z. Ariffin, “A Survey on Malware Analysis Techniques: Static, Dynamic, Hybrid and Memory Analysis,” vol. 8, pp. 4–6, 2018.
- [37] Y. Ye, T. Li, D. Adjeroh, and S. S. Iyengar, “A survey on malware detection using data mining techniques,” *ACM Comput. Surv.*, vol. 50, no. 3, May 2018, doi: 10.1145/3073559.
- [38] “A study on Malware Detection using P.E. Header Ms. Ghøjaira Sadia Iqbal 1 Dr. Prashant Itankar 2 Prof. Ankit Sanghavi 3,” 2022. [Online]. Available: www.jetir.org/b540
- [39] M. Pietrek, “Inside Windows: An In-Depth Look into the Win32 Portable Executable File Format, Part 1.” [Online]. Available: https://bytewriter.com/resources/pietrek_in_depth_look_into_pe_for...
- [40] A.-R. Belea, “Methods for Detecting Malware Using Static, Dynamic and Hybrid Analysis.”

- [41] R. Sihwail, K. Omar, and K. A. Z. Ariffin, "A Survey on Malware Analysis Techniques: Static, Dynamic, Hybrid and Memory Analysis," vol. 8, pp. 4–6, 2018.
- [42] M. Christodorescu and S. Jha, "Static Analysis of Executables to Detect Malicious Patterns *,"
- [43] N. Alshahwan, E. T. Barr, D. Clark, G. Danezis, and H. D. Menéndez, "Detecting malware with information complexity," May 01, 2020, *MDPI AG*. doi: 10.3390/E22050575.
- [44] H. D. Menéndez and J. L. Llorente, "Mimicking anti-viruses with machine learning and entropy profiles," *Entropy*, vol. 21, no. 5, May 2019, doi: 10.3390/e21050513.
- [45] V. Moussas and A. Andreatos, "Malware detection based on code visualization and two-level classification," *Information (Switzerland)*, vol. 12, no. 3, pp. 1–14, Mar. 2021, doi: 10.3390/info12030118.
- [46] A. Damodaran, F. Di Troia, V. Aaron Corrado, T. H. Austin, and M. Stamp, "A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection."
- [47] S. Jaan and M. Khan, "Study on Color Image Processing: A Review Paper," *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, vol. 2, no. 2, 2022, doi: 10.48175/568.
- [48] J. Khatib Sulaiman Dalam No, H. Majeed Zangana, A. khalid Mohammed, Z. Bibo Sallow, and F. Mahmood Mustafa, "Exploring Image Representation and Color Spaces in Computer Vision: A Comprehensive Review," *Indonesian Journal of Computer Science*.
- [49] S. Hallur and A. Gavade, "Image feature extraction techniques: A comprehensive review," Sep. 01, 2025, *Elsevier B.V.* doi: 10.1016/j.fraope.2025.100366.
- [50] T. Shahriar and H. Li, "A Study of Image Pre-processing for Faster Object Recognition."
- [51] X. Chen, X. Liu, Y. Wu, Z. Wang, and S. H. Wang, "Research related to the diagnosis of prostate cancer based on machine learning medical images: A review," Jan. 01, 2024, *Elsevier Ireland Ltd.* doi: 10.1016/j.ijmedinf.2023.105279.
- [52] L. Perez and J. Wang, "The Effectiveness of Data Augmentation in Image Classification using Deep Learning," Dec. 2017, [Online]. Available: <http://arxiv.org/abs/1712.04621>
- [53] J. Husain and S. Agarwal, "Optimizing Image Classification Through Preprocessing and Feature Engineering Techniques," 2023. [Online]. Available: <https://powertechjournal.com>
- [54] P. Wang, E. Fan, and P. Wang, "Comparative analysis of image classification algorithms based on traditional machine learning and deep learning," *Pattern Recognit. Lett.*, vol. 141, pp. 61–67, Jan. 2021, doi: 10.1016/j.patrec.2020.07.042.
- [55] L. Breiman, "RANDOM FORESTS-RANDOM FEATURES," 1999.

- [56] J. Plested, M. Phiri, and T. Gedeon, “Deep transfer learning for image classification: a survey,” Dec. 2025, [Online]. Available: <http://arxiv.org/abs/2205.09904>
- [57] R. A. Tavares, “Comparison of Image Preprocessing Techniques for Vehicle License Plate Recognition Using OCR: Performance and Accuracy Evaluation,” Oct. 2024, [Online]. Available: <http://arxiv.org/abs/2410.13622>
- [58] I. Muhammad and Z. Yan, “SUPERVISED MACHINE LEARNING APPROACHES: A SURVEY.”
- [59] T. Jiang, J. L. Gradus, and A. J. Rosellini, “Supervised Machine Learning: A Brief Primer,” *Behav. Ther.*, vol. 51, no. 5, pp. 675–687, Sep. 2020, doi: 10.1016/j.beth.2020.05.002.
- [60] W. Zamora-Cárdenas and M. Zumbado, “McCulloch-Pitts Artificial Neuron and Rosenblatt’s Perceptron: An abstract specification in Z.”
- [61] S. Bouraya and A. Belangour, “A comparative analysis of activation functions in neural networks: unveiling categories,” *Bulletin of Electrical Engineering and Informatics*, vol. 13, no. 5, pp. 3301–3308, Oct. 2024, doi: 10.11591/eei.v13i5.7274.
- [62] G. Rangel, J. C. Cuevas-Tello, J. Nunez-Varela, C. Puente, and A. G. Silva-Trujillo, “A Survey on Convolutional Neural Networks and Their Performance Limitations in Image Recognition Tasks,” 2024, *Hindawi Limited*. doi: 10.1155/2024/2797320.
- [63] P. Tsirtsakis, G. Zacharis, G. S. Maraslidis, and G. F. Fragulis, “Deep learning for object recognition: A comprehensive review of models and algorithms,” *International Journal of Cognitive Computing in Engineering*, vol. 6, pp. 298–312, Dec. 2025, doi: 10.1016/j.ijcce.2025.01.004.
- [64] O. Elharrouss *et al.*, “Task-based Loss Functions in Computer Vision: A Comprehensive Review,” Sep. 2025, [Online]. Available: <http://arxiv.org/abs/2504.04242>
- [65] K. Yadav, “A Comprehensive Study on Optimization Strategies for Gradient Descent In Deep Learning,” Jan. 2021, [Online]. Available: <http://arxiv.org/abs/2101.02397>
- [66] S. Nazim, M. M. Alam, S. Rizvi, J. C. Mustapha, S. S. Hussain, and M. M. Su’ud, “Multimodal malware classification using proposed ensemble deep neural network framework,” *Sci. Rep.*, vol. 15, no. 1, Dec. 2025, doi: 10.1038/s41598-025-96203-3.
- [67] V. Anandhi, P. Vinod, V. G. Menon, and K. M. Aditya, “Performance evaluation of deep neural network on malware detection: visual feature approach,” *Cluster Comput.*, vol. 25, no. 6, pp. 4601–4615, Dec. 2022, doi: 10.1007/s10586-022-03702-3.
- [68] A. De Paola, S. Drago, P. Ferraro, and G. Lo Re, “Detecting Zero-Day Attacks under Concept Drift: An Online Unsupervised Threat Detection System,” 2024.

- [69] H. Benkraouda, J. Qian, H. Q. Tran, and B. Kaplan, “Attacks on Visualization-Based Malware Detection: Balancing Effectiveness and Executability,” Sep. 2021, [Online]. Available: <http://arxiv.org/abs/2109.10417>
- [70] E. Kesavan, “Real-Time Adaptive Framework for Behavioural Malware Detection in Evolving Threat Environments,” *International Journal of Scientific Research and Modern Technology*, pp. 32–39, Mar. 2022, doi: 10.38124/ijsrmt.v1i3.842.
- [71] H. S. Anderson and P. Roth, “EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models,” Apr. 2018, [Online]. Available: <http://arxiv.org/abs/1804.04637>
- [72] OGHENETEGA AVWOKWURUAYE, EJINKONYE IFEOMA O., and ALIYU MUSTAPHA UMAR, “TRADITIONAL VS NEXT GENERATION ANTIVIRUS: EVALUATING THEIR ROLE IN MODERN CYBER SECURITY,” *Harvard International Journal of Engineering Research and Technology*, Sep. 2025, doi: 10.70382/hijert.v8i5.010.
- [73] S. Rehman, S. Jamil, N. Hussain, A. Latif, S. Ahmad, and P. Punjab, “ISSN (e) 3007-3138 (p) 3007-312X INTELLIGENT ADAPTIVE MACHINE LEARNING SCALABLE FRAMEWORK FOR DYNAMIC MALWARE IDENTIFICATION AND PROACTIVE THREAT PREVENTION”.

Appendix 1 – File Collectors

1.1. Benign Files (*benign.py*)

```
import os
import shutil
import random

BASE_DIR = r"C:\Users\VMLicenta\Desktop\Dataset\malware_dataset\benign2"
BENIGN_DIR = os.path.join(BASE_DIR, "benign2")
TARGET_COUNT = 2000

BENIGN_SOURCES = [
    r"C:\Windows\System32",
    r"C:\Windows\SysWOW64",
    r"C:\Program Files",
    r"C:\Program Files (x86)",
]

MIN_SIZE = 20_000
MAX_SIZE = 50_000_000

os.makedirs(BENIGN_DIR, exist_ok=True)

def collect_benign(out_dir, target=100):
    collected = 0
    candidates = []

    print("[*] Scanning for benign .exe files...")
    for src_dir in BENIGN_SOURCES:
        if not os.path.exists(src_dir):
            continue
        for root, _, files in os.walk(src_dir):
            for fname in files:
                if not fname.lower().endswith(".exe"):
                    continue
                full_path = os.path.join(root, fname)
                try:
                    size = os.path.getsize(full_path)
                    if MIN_SIZE < size < MAX_SIZE:
                        candidates.append(full_path)
                except OSError:
                    continue

    print(f"[*] Found {len(candidates)} candidate benign executables")

    if not candidates:
        print("[!] No candidates found. Check BENIGN_SOURCES paths.")
        return 0

    random.shuffle(candidates)
    selected = candidates[:target]

    for src_path in selected:
        fname = os.path.basename(src_path)
```

```
dst_path = os.path.join(out_dir, fname)

# Avoid name collisions
base, ext = os.path.splitext(fname)
i = 1
while os.path.exists(dst_path):
    dst_path = os.path.join(out_dir, f"{base}_{i}{ext}")
    i += 1

try:
    shutil.copy2(src_path, dst_path)
    collected += 1
    print(f"[{collected}/{target}] {fname}")
except Exception as e:
    print(f"[!] Failed to copy {fname}: {e}")

if collected >= target:
    break

print(f"[*] Done. Collected {collected} benign files into: {out_dir}")
return collected

if __name__ == "__main__":
    collect_benign(BENIGN_DIR, TARGET_COUNT)
```

1.2. Malware Collector (dataset.py)

```
import requests
import os
import time
import subprocess
import tempfile
import shutil

API_URL = "https://mb-api.abuse.ch/api/v1/"
ZIP_PASSWORD = "infected"
BASE_DIR = "malware_dataset"
SAMPLES_PER_CATEGORY = 500
PER_TAG_LIMIT = 100
SEVENZIP = r"C:\Program Files\7-Zip\7z.exe"

AUTH_KEY = "YOUR_API_KEY_HERE"
HEADERS = {"Auth-Key": AUTH_KEY}

CATEGORIES = {
    "ransomware": [
        "ransomware",
        "lockbit", "LockBit", "Ransomware.LockBit",
        "wannacry", "WannaCry", "Ransomware.WannaCry",
        "gandcrab", "GandCrab",
        "cerber", "Cerber",
        "dharma", "Dharma",
        "maze", "Maze",
        "phobos", "Phobos",
        "ryuk", "Ryuk",
        "conti", "Conti",
```

```
"sodinokibi", "Sodinokibi", "revil", "REvil",  
"clop", "Clop",  
"darkside", "DarkSide",  
"ragnarlocker", "RagnarLocker",  
"blackmatter", "BlackMatter",  
"babuk", "Babuk",  
"zeppelin", "Zeppelin",  
],
```

```
"trojan": [  
  "trojan",  
  "emotet", "Emotet",  
  "qbot", "Qbot", "qakbot", "QakBot",  
  "dridex", "Dridex",  
  "trickbot", "TrickBot",  
  "ursnif", "Ursnif",  
  "gozi", "Gozi",  
  "icedid", "IcedID", "bokbot",  
  "zeus", "Zeus",  
  "agenttesla", "AgentTesla",  
  "formbook", "Formbook",  
  "azorult", "Azorult",  
  "lokibot", "LokiBot",  
  "vidar", "Vidar",  
  "redline", "RedLine", "RedLineStealer",  
  "raccoon", "RaccoonStealer",  
],
```

```
"rat": [  
  "rat",  
  "remcos", "Remcos", "RemcosRAT",  
  "nanocore", "NanoCore",  
  "njjrat", "Njjrat", "njjRAT",  
  "asynccrat", "AsynccRAT",  
  "quasar", "QuasarRAT", "Quasar",  
  "netwire", "NetWire",  
  "darkcomet", "DarkComet",  
  "gh0st", "Gh0st", "Gh0stRAT",  
  "plugx", "PlugX",  
  "adwind", "Adwind",  
  "poisonivy", "PoisonIvy",  
  "xrat", "Xrat",  
],
```

```
"worm": [  
  "worm",  
  "conficker", "Conficker",  
  "sality", "Sality",  
  "gamarue", "Gamarue",  
  "autorun", "INF/Autorun",  
  "virus", "Virus",  
  "vobfus", "Vobfus",  
  "dorkbot", "Dorkbot",  
  "ramnit", "Ramnit",  
  "nimda", "Nimda",
```

```
        "kido", "Kido",
        "blaster", "Blaster",
        "sasser", "Sasser",
    ],
}

def query_by_tag(tag, limit=PER_TAG_LIMIT):
    try:
        r = requests.post(
            API_URL,
            headers=HEADERS,
            data={"query": "get_taginfo", "tag": tag, "limit": limit},
            timeout=15,
        )
        data = r.json()
        print(f"    API status: {data.get('query_status')}")
        if data.get("query_status") == "ok":
            return [
                s["sha256_hash"]
                for s in data["data"]
                if s.get("file_type") in ("exe", "dll", "pe32", "pe32+")
            ]
        except Exception as e:
            print(f"    [!] Query error for tag '{tag}': {e}")
            return []

def download_sample sha256, out_dir):
    try:
        r = requests.post(
            API_URL,
            headers=HEADERS,
            data={"query": "get_file", "sha256_hash": sha256},
            timeout=30,
        )
        if r.status_code != 200 or r.content[:2] != b"PK":
            print(f"    [!] Bad response for {sha256[:16]}")
            return False

        with tempfile.TemporaryDirectory() as tmpdir:
            zip_path = os.path.join(tmpdir, "sample.zip")
            with open(zip_path, "wb") as f:
                f.write(r.content)

            result = subprocess.run(
                [SEVENZIP, "e", zip_path, f"-p{ZIP_PASSWORD}", f"-o{tmpdir}", "-y"],
                capture_output=True,
                text=True,
            )

            if result.returncode != 0:
                print(f"    [!] 7z failed for {sha256[:16]}: {result.stderr.strip()}")
                return False
```



```
        for fname in os.listdir(tmpdir):
            if fname == "sample.zip":
                continue
            src = os.path.join(tmpdir, fname)
            dst = os.path.join(out_dir, sha256[:16] + ".exe")
            shutil.move(src, dst)
            return True

    except Exception as e:
        print(f"      [!] Error for {sha256[:16]}: {e}")
    return False

for category, tags in CATEGORIES.items():
    out_dir = os.path.join(BASE_DIR, category)
    os.makedirs(out_dir, exist_ok=True)
    print(f"\n{'='*60}")
    print(f"  Category: {category.upper()}")
    print(f"{'='*60}")

    collected = 0
    seen = set()

    for tag in tags:
        if collected >= SAMPLES_PER_CATEGORY:
            break

        print(f"  Querying tag: '{tag}'...")
        hashes = query_by_tag(tag, limit=PER_TAG_LIMIT)
        print(f"    → {len(hashes)} PE samples found")

        for h in hashes:
            if collected >= SAMPLES_PER_CATEGORY:
                break
            if h in seen:
                continue
            seen.add(h)

            if download_sample(h, out_dir):
                collected += 1
                print(f"      [{collected}/{SAMPLES_PER_CATEGORY}] {h[:20]}...")
                time.sleep(0.7)

    print(f"  Done: {collected} files in '{out_dir}'")

print("\n[✓] Complete. Dataset structure:")
for cat in CATEGORIES:
    p = os.path.join(BASE_DIR, cat)
    n = len(os.listdir(p)) if os.path.exists(p) else 0
    print(f"  {p}/  ({n} files)")
```

Appendix 2 – Features Extractors

2.1. Entropy Calculator (*Entropy_calc.py*)

```
import math
import os
import tkinter as tk
from tkinter import filedialog, messagebox
from PIL import Image

CHUNK_SIZE = 256
NUM_CHUNKS = 64
TOTAL_BYTES = CHUNK_SIZE * NUM_CHUNKS
IMAGE_SIZE = 8
DISPLAY_SIZE = 512

OUTPUT_DIR = r"C:\Users\VMLicenta\Desktop\entropy\benign2"

def compute_entropy(data: bytes) -> float:
    if not data:
        return 0.0
    freq = [0] * 256
    for byte in data:
        freq[byte] += 1
    entropy = 0.0
    length = len(data)
    for count in freq:
        if count > 0:
            p = count / length
            entropy -= p * math.log2(p)
    return entropy

def entropy_to_rgb(normalized: float) -> tuple:
    v = max(0.0, min(1.0, normalized))
    if v <= 0.5:
        r = int(v * 2 * 255)
        g = 255
    else:
        r = 255
        g = int((1.0 - v) * 2 * 255)
    return (r, g, 0)

def process_file(filepath: str) -> str:
    with open(filepath, "rb") as f:
        raw = f.read(TOTAL_BYTES)

    if len(raw) < TOTAL_BYTES:
        raw = raw + b"\x00" * (TOTAL_BYTES - len(raw))

    pixels = []
    for i in range(NUM_CHUNKS):
        chunk = raw[i * CHUNK_SIZE : (i + 1) * CHUNK_SIZE]
        entropy = compute_entropy(chunk)
        normalized = entropy / 8.0
        pixels.append(entropy_to_rgb(normalized))
```

```
img = Image.new("RGB", (IMAGE_SIZE, IMAGE_SIZE))
img.putdata(pixels)
img_scaled = img.resize((DISPLAY_SIZE, DISPLAY_SIZE), Image.NEAREST)

os.makedirs(OUTPUT_DIR, exist_ok=True)

base_name = os.path.splitext(os.path.basename(filepath))[0]
output_path = os.path.join(OUTPUT_DIR, base_name + ".png")
img_scaled.save(output_path)
return output_path

def main():
    root = tk.Tk()
    root.withdraw()

    filepaths = filedialog.askopenfilenames(
        title="Select executable files",
        filetypes=[
            ("Executable files", "*.exe *.dll *.sys *.scr *.com"),
            ("All files", "*.*"),
        ],
    )

    if not filepaths:
        messagebox.showinfo("Cancelled", "No file selected.")
        return

    outputs = []
    errors = []

    for fp in filepaths:
        try:
            out = process_file(fp)
            outputs.append(out)
            print(f"[OK] Saved: {out}")
        except Exception as e:
            errors.append((fp, str(e)))
            print(f"[ERROR] {fp}: {e}")

    if outputs:
        messagebox.showinfo(
            "Done",
            "Entropy color matrices saved:\n" + "\n".join(outputs),
        )
    if errors:
        msg = "Some files failed:\n" + "\n".join(f"{p}: {err}" for p, err in
errors)
        messagebox.showwarning("Errors", msg)

if __name__ == "__main__":
    main()
```

2.2. Features Extractor (features.py)

```
from pathlib import Path
import numpy as np
import csv
import tkinter as tk
from tkinter import filedialog
from collections import defaultdict

WINDOW_SIZE = 16 * 1024
CHUNK_SIZE = 256
NUM_CHUNKS = WINDOW_SIZE // CHUNK_SIZE
HIGH_ENTROPY_THRESHOLD = 7.0

OUTPUT_DIR = Path(r"C:\Users\VMLicenta\Desktop\features\benign2")

def shannon_entropy(byte_data: bytes) -> float:
    if not byte_data:
        return 0.0
    arr = np.frombuffer(byte_data, dtype=np.uint8)
    counts = np.bincount(arr, minlength=256)
    probs = counts[counts > 0] / len(arr)
    return float(-np.sum(probs * np.log2(probs)))

def extract_7_features(file_path: Path):
    try:
        raw = file_path.read_bytes()
    except Exception as e:
        print(f"Could not read {file_path}: {e}")
        return None

    if len(raw) == 0:
        print(f"Skipping empty file: {file_path}")
        return None

    file_size = len(raw)
    window = raw[:WINDOW_SIZE]

    if len(window) < WINDOW_SIZE:
        window = window + bytes(WINDOW_SIZE - len(window))

    chunk_entropies = []
    for i in range(NUM_CHUNKS):
        start = i * CHUNK_SIZE
        end = start + CHUNK_SIZE
        chunk = window[start:end]
        ent = shannon_entropy(chunk)
        chunk_entropies.append(ent)

    chunk_entropies = np.array(chunk_entropies, dtype=np.float32)

    return {
        "file_name": file_path.name,
        "file_path": str(file_path),
        "file_size_bytes": file_size,
        "log_file_size": float(np.log1p(file_size)),
        "window_entropy": float(shannon_entropy(window)),
    }
```

```
        "mean_chunk_entropy": float(np.mean(chunk_entropies)),
        "std_chunk_entropy": float(np.std(chunk_entropies)),
        "max_chunk_entropy": float(np.max(chunk_entropies)),
        "min_chunk_entropy": float(np.min(chunk_entropies)),
        "high_entropy_ratio": float(np.mean(chunk_entropies >
HIGH_ENTROPY_THRESHOLD))
    }

FIELDNAMES = [
    "file_name",
    "file_path",
    "file_size_bytes",
    "log_file_size",
    "window_entropy",
    "mean_chunk_entropy",
    "std_chunk_entropy",
    "max_chunk_entropy",
    "min_chunk_entropy",
    "high_entropy_ratio"
]

def save_csv(category: str, rows: list, output_dir: Path):
    output_dir.mkdir(parents=True, exist_ok=True)
    csv_path = output_dir / f"{category}_features.csv"

    with open(csv_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=FIELDNAMES)
        writer.writeheader()
        writer.writerows(rows)

    print(f"    Saved: {csv_path}    ({len(rows)} rows)")

def main():
    root = tk.Tk()
    root.withdraw()

    selected_files = filedialog.askopenfilenames(
        title="Select executable files to process",
        filetypes=[
            ("Executable files", "*.exe *.dll *.bin"),
            ("All files", "*.*")
        ]
    )

    if not selected_files:
        print("No files selected.")
        return

    print(f"\nSelected {len(selected_files)} file(s).\n")
    grouped = defaultdict(list)
    for filepath_str in selected_files:
        fp = Path(filepath_str)
        category = fp.parent.name
        grouped[category].append(fp)
```

```
total_processed = 0

for category, files in grouped.items():
    print(f"Processing category: '{category}'  ({len(files)} files)")
    rows = []

    for idx, fp in enumerate(files, start=1):
        result = extract_7_features(fp)
        if result is not None:
            rows.append(result)
        print(f"  [{idx}/{len(files)}] {fp.name}")

    if rows:
        save_csv(category, rows, OUTPUT_DIR)
        total_processed += len(rows)
    else:
        print(f"  No valid files processed for '{category}'")

print(f"\nDone. {total_processed} files processed in total.")
print(f"CSVs saved to: {OUTPUT_DIR.resolve()}")

if __name__ == "__main__":
    main()
```

Appendix 3 – Random Forest Models

3.1. Malware Versus Benign Model (*MvsB_feat_plots.py*)

```
from pathlib import Path
import numpy as np
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score,
    roc_curve,
    auc,
    precision_recall_curve,
    average_precision_score
)
from joblib import dump

IMAGE_ROOT = Path(r"C:\Users\VMLicenta\Desktop\entropy")
FEATURES_DIR = Path(r"C:\Users\VMLicenta\Desktop\features")
MODEL_PATH = Path(r"C:\Users\VMLicenta\Desktop\models\MvsB_train_plots.joblib")

PLOTS_DIR = Path("plots_binary")
PLOTS_DIR.mkdir(parents=True, exist_ok=True)

MALWARE_CLASSES = ["ransomware", "trojan", "rat", "worm"]
BENIGN_CLASS = "benign2"
CLASS_NAMES = ["malicious", "benign"]

STATIC_COLS = [
    "log_file_size",
    "window_entropy",
    "mean_chunk_entropy",
    "std_chunk_entropy",
    "max_chunk_entropy",
    "min_chunk_entropy",
    "high_entropy_ratio"
]

def load_static_features_binary(features_dir: Path):
    lookup = {}

    for class_name in MALWARE_CLASSES + [BENIGN_CLASS]:
        csv_path = features_dir / f"{class_name}_features.csv"

        if not csv_path.exists():
            print(f"Warning: CSV not found -> {csv_path}")
            lookup[class_name] = {}
```

```
        continue

    df = pd.read_csv(csv_path)

    class_lookup = {}
    for _, row in df.iterrows():
        stem = Path(row["file_name"]).stem
        feat = row[STATIC_COLS].values.astype(np.float32)
        class_lookup[stem] = feat

    lookup[class_name] = class_lookup
    print(f"  Loaded {len(class_lookup)} static feature rows for
'{class_name}'")

    return lookup

print("Loading static features from CSVs...")
static_lookup = load_static_features_binary(FEATURES_DIR)

def load_binary_combined_features(image_root: Path, static_lookup: dict):
    X = []
    y = []
    skipped = 0

    # malicious = 0
    for class_name in MALWARE_CLASSES:
        class_dir = image_root / class_name
        if not class_dir.exists():
            print(f"Warning: image folder not found -> {class_dir}")
            continue

        class_static = static_lookup.get(class_name, {})

        for img_path in sorted(class_dir.glob("*.png")):
            stem = img_path.stem
            static_feat = class_static.get(stem)

            if static_feat is None:
                skipped += 1
                continue

            try:
                img = Image.open(img_path).convert("RGB")
                img = img.resize((8, 8))
                arr = np.array(img, dtype=np.float32)
                image_feat = arr.flatten()
            except Exception as e:
                print(f"Skipping image {img_path}: {e}")
                skipped += 1
                continue

            combined = np.hstack([image_feat, static_feat])
            X.append(combined)
            y.append(0)
```



```
# benign = 1
benign_dir = image_root / BENIGN_CLASS
if not benign_dir.exists():
    print(f"Warning: image folder not found -> {benign_dir}")
else:
    benign_static = static_lookup.get(BENIGN_CLASS, {})

    for img_path in sorted(benign_dir.glob("*.png")):
        stem = img_path.stem
        static_feat = benign_static.get(stem)

        if static_feat is None:
            skipped += 1
            continue

        try:
            img = Image.open(img_path).convert("RGB")
            img = img.resize((8, 8))
            arr = np.array(img, dtype=np.float32)
            image_feat = arr.flatten()
        except Exception as e:
            print(f"Skipping image {img_path}: {e}")
            skipped += 1
            continue

        combined = np.hstack([image_feat, static_feat])
        X.append(combined)
        y.append(1)

    if skipped > 0:
        print(f"Warning: {skipped} images skipped (no matching static
features or load error)")

X = np.array(X)
y = np.array(y)
return X, y

print("\nLoading entropy images and merging features...")
X, y = load_binary_combined_features(IMAGE_ROOT, static_lookup)

print(f"Final dataset: {X.shape[0]} samples, {X.shape[1]} features each")
print(f" Image features : 192")
print(f" Static features:  7")
print(f" Total           : {X.shape[1]}")
print(f" Malicious        : {np.sum(y == 0)}")
print(f" Benign           : {np.sum(y == 1)}")

if len(X) == 0:
    print("\nError: no samples loaded. Check that image filenames match CSV
file_name stems.")
    exit()

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
```

```
        test_size=0.30,
        random_state=42,
        stratify=y
    )

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)

print("Train size:", X_train.shape[0])
print("Val size  :", X_val.shape[0])
print("Test size :", X_test.shape[0])

rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=10,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train, y_train)

def plot_and_save_confusion_matrix(y_true, y_pred, class_names, title,
save_path):
    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(6, 5))
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=class_names,
        yticklabels=class_names
    )
    plt.xlabel("Predicted label")
    plt.ylabel("True label")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.close()

def plot_and_save_normalized_confusion_matrix(y_true, y_pred, class_names,
title, save_path):
    cm = confusion_matrix(y_true, y_pred, normalize="true")

    plt.figure(figsize=(6, 5))
    sns.heatmap(
        cm,
```

```
        annot=True,
        fmt=".2f",
        cmap="Blues",
        xticklabels=class_names,
        yticklabels=class_names,
        vmin=0.0,
        vmax=1.0
    )
    plt.xlabel("Predicted label")
    plt.ylabel("True label")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.close()

def plot_and_save_class_metrics(y_true, y_pred, class_names, save_path,
title):
    report = classification_report(
        y_true,
        y_pred,
        target_names=class_names,
        output_dict=True
    )

    precision = [report[name]["precision"] for name in class_names]
    recall = [report[name]["recall"] for name in class_names]
    f1 = [report[name]["f1-score"] for name in class_names]

    x = np.arange(len(class_names))
    width = 0.25

    plt.figure(figsize=(8, 5))
    plt.bar(x - width, precision, width, label="Precision")
    plt.bar(x, recall, width, label="Recall")
    plt.bar(x + width, f1, width, label="F1-score")

    plt.xticks(x, class_names)
    plt.ylim(0, 1.0)
    plt.ylabel("Score")
    plt.title(title)
    plt.legend()
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.close()

def plot_and_save_roc_curve(y_true, y_score, save_path):
    fpr, tpr, _ = roc_curve(y_true, y_score, pos_label=0)
    roc_auc = auc(fpr, tpr)

    plt.figure(figsize=(6, 5))
    plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", color="darkorange")
    plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curve")
```

```
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig(save_path, dpi=300, bbox_inches="tight")
plt.close()

def plot_and_save_pr_curve(y_true, y_score, save_path):
    precision, recall, _ = precision_recall_curve(y_true == 0, y_score)
    ap = average_precision_score(y_true == 0, y_score)

    plt.figure(figsize=(6, 5))
    plt.plot(recall, precision, label=f"AP = {ap:.3f}", color="purple")
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title("Precision-Recall Curve")
    plt.legend(loc="lower left")
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.close()

def plot_and_save_probability_histogram(y_true, y_score, save_path):
    plt.figure(figsize=(7, 5))
    plt.hist(y_score[y_true == 0], bins=20, alpha=0.6, label="Malicious")
    plt.hist(y_score[y_true == 1], bins=20, alpha=0.6, label="Benign")
    plt.xlabel("Predicted probability of malicious")
    plt.ylabel("Count")
    plt.title("Probability Distribution")
    plt.legend()
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.close()

def plot_and_save_feature_importance(model, feature_labels, save_path,
top_n=15):
    importances = model.feature_importances_
    idx = np.argsort(importances)[::-1][:top_n]

    top_features = [feature_labels[i] for i in idx]
    top_values = importances[idx]

    plt.figure(figsize=(10, 6))
    plt.barh(range(len(top_features)), top_values, color="steelblue")
    plt.yticks(range(len(top_features)), top_features)
    plt.gca().invert_yaxis()
    plt.xlabel("Importance")
    plt.title(f"Top {top_n} Most Important Features")
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.close()

def evaluate_split(name, model, X_split, y_split):
    y_pred = model.predict(X_split)
    y_proba = model.predict_proba(X_split)[:, 0]

    print(f"\n=== {name} ===")
    print("Accuracy:", accuracy_score(y_split, y_pred))
```

```
print(classification_report(
    y_split,
    y_pred,
    target_names=CLASS_NAMES
))
print("Confusion matrix:")
print(confusion_matrix(y_split, y_pred))

plot_and_save_confusion_matrix(
    y_split,
    y_pred,
    CLASS_NAMES,
    title=f"{name} Confusion Matrix",
    save_path=PLOTS_DIR / f"{name.lower()}_confusion_matrix.png"
)

plot_and_save_normalized_confusion_matrix(
    y_split,
    y_pred,
    CLASS_NAMES,
    title=f"{name} Normalized Confusion Matrix",
    save_path=PLOTS_DIR /
f"{name.lower()}_confusion_matrix_normalized.png"
)

plot_and_save_class_metrics(
    y_split,
    y_pred,
    CLASS_NAMES,
    save_path=PLOTS_DIR / f"{name.lower()}_class_metrics.png",
    title=f"{name} Precision / Recall / F1"
)

plot_and_save_roc_curve(
    y_split,
    y_proba,
    save_path=PLOTS_DIR / f"{name.lower()}_roc_curve.png"
)

plot_and_save_pr_curve(
    y_split,
    y_proba,
    save_path=PLOTS_DIR / f"{name.lower()}_pr_curve.png"
)

plot_and_save_probability_histogram(
    y_split,
    y_proba,
    save_path=PLOTS_DIR / f"{name.lower()}_probability_histogram.png"
)

evaluate_split("Validation", rf, X_val, y_val)
evaluate_split("Test", rf, X_test, y_test)

feature_labels = [f"img_{i}" for i in range(192)] + STATIC_COLS
```

```
importances = rf.feature_importances_

print("\n--- Top 15 most important features ---")
top_idx = np.argsort(importances)[::-1][:15]
for rank, i in enumerate(top_idx, start=1):
    print(f" {rank:2d}. {feature_labels[i]:30s} importance:
{importances[i]:.4f}")

plot_and_save_feature_importance(
    rf,
    feature_labels,
    save_path=PLOTS_DIR / "top_15_feature_importances.png",
    top_n=15
)

dump(
    {
        "model": rf,
        "class_names": CLASS_NAMES,
        "static_cols": STATIC_COLS,
        "feature_count": X.shape[1]
    },
    MODEL_PATH
)

print(f"\nModel saved to: {MODEL_PATH}")
print(f"Plots saved to: {PLOTS_DIR.resolve()}")
```

3.2. Multi-Class Model (rf_train_feat_plots.py)

```
from pathlib import Path
import numpy as np
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score
from joblib import dump

IMAGE_ROOT    = Path(r"C:\Users\VMLicenta\Desktop\entropy")
FEATURES_DIR  = Path(r"C:\Users\VMLicenta\Desktop\features")
MODEL_PATH    =
Path(r"C:\Users\VMLicenta\Desktop\models\rf_train_plots.joblib")

PLOTS_DIR = Path("plots_multiclass")
PLOTS_DIR.mkdir(parents=True, exist_ok=True)

CLASS_NAMES  = ["ransomware", "trojan", "rat", "worm", "benign"]

STATIC_COLS  = [
    "log_file_size",
```

```
"window_entropy",
"mean_chunk_entropy",
"std_chunk_entropy",
"max_chunk_entropy",
"min_chunk_entropy",
"high_entropy_ratio"
]

def load_static_features(features_dir: Path, class_names: list):

    lookup = {}

    for class_name in class_names:
        csv_path = features_dir / f"{class_name}_features.csv"

        if not csv_path.exists():
            print(f"Warning: CSV not found -> {csv_path}")
            lookup[class_name] = {}
            continue

        df = pd.read_csv(csv_path)

        class_lookup = {}
        for _, row in df.iterrows():
            stem = Path(row["file_name"]).stem
            feat = row[STATIC_COLS].values.astype(np.float32)
            class_lookup[stem] = feat

        lookup[class_name] = class_lookup
        print(f"  Loaded {len(class_lookup)} static feature rows for
'{class_name}'")

    return lookup

print("Loading static features from CSVs...")
static_lookup = load_static_features(FEATURES_DIR, CLASS_NAMES)

def load_combined_features(image_root: Path, class_names: list,
static_lookup: dict):
    X = []
    y = []
    skipped = 0

    for label_idx, class_name in enumerate(class_names):
        class_dir = image_root / class_name

        if not class_dir.exists():
            print(f"Warning: image folder not found -> {class_dir}")
            continue

        class_static = static_lookup.get(class_name, {})

        for img_path in sorted(class_dir.glob("*.png")):
            stem = img_path.stem
            static_feat = class_static.get(stem)
```

```
        if static_feat is None:
            skipped += 1
            continue

        try:
            img = Image.open(img_path).convert("RGB")
            img = img.resize((8, 8))
            arr = np.array(img, dtype=np.float32)
            image_feat = arr.flatten()
        except Exception as e:
            print(f"Skipping image {img_path}: {e}")
            skipped += 1
            continue

        combined = np.hstack([image_feat, static_feat])

        X.append(combined)
        y.append(label_idx)

    if skipped > 0:
        print(f"Warning: {skipped} images skipped (no matching static
features or load error)")

    X = np.array(X)
    y = np.array(y)
    return X, y

print("\nLoading entropy images and merging features...")
X, y = load_combined_features(IMAGE_ROOT, CLASS_NAMES, static_lookup)

print(f"Final dataset: {X.shape[0]} samples, {X.shape[1]} features each")
print(f" Image features : 192")
print(f" Static features:   7")
print(f" Total                : {X.shape[1]}")

for idx, name in enumerate(CLASS_NAMES):
    print(f" Class '{name}': {np.sum(y == idx)} samples")

if len(X) == 0:
    print("\nError: no samples loaded. Check that image filenames match CSV
file_name stems.")
    exit()

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
```



```
)

print(f"\nSplit sizes -> Train: {len(X_train)}  Val: {len(X_val)}  Test:
{len(X_test)}")

print("\nTraining Random Forest...")

rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=15,
    min_samples_leaf=1,
    max_features="sqrt",
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train, y_train)

def plot_and_save_confusion_matrix(y_true, y_pred, class_names, title,
save_path):
    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(7, 6))
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=class_names,
        yticklabels=class_names
    )
    plt.xlabel("Predicted label")
    plt.ylabel("True label")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.close()

def plot_and_save_normalized_confusion_matrix(y_true, y_pred, class_names,
title, save_path):
    cm = confusion_matrix(y_true, y_pred, normalize="true")

    plt.figure(figsize=(7, 6))
    sns.heatmap(
        cm,
        annot=True,
        fmt=".2f",
        cmap="Blues",
        xticklabels=class_names,
        yticklabels=class_names,
        vmin=0.0,
        vmax=1.0
    )
    plt.xlabel("Predicted label")
```

```
plt.ylabel("True label")
plt.title(title)
plt.tight_layout()
plt.savefig(save_path, dpi=300, bbox_inches="tight")
plt.close()

def plot_and_save_feature_importance(model, feature_labels, save_path,
top_n=15):
    importances = model.feature_importances_
    idx = np.argsort(importances)[::-1][:top_n]

    top_features = [feature_labels[i] for i in idx]
    top_values = importances[idx]

    plt.figure(figsize=(10, 6))
    plt.barh(range(len(top_features)), top_values, color="steelblue")
    plt.yticks(range(len(top_features)), top_features)
    plt.gca().invert_yaxis()
    plt.xlabel("Importance")
    plt.title(f"Top {top_n} Most Important Features")
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.close()

def plot_and_save_class_metrics(y_true, y_pred, class_names, save_path,
title):
    report = classification_report(
        y_true,
        y_pred,
        target_names=class_names,
        output_dict=True
    )

    precision = [report[name]["precision"] for name in class_names]
    recall = [report[name]["recall"] for name in class_names]
    f1 = [report[name]["f1-score"] for name in class_names]

    x = np.arange(len(class_names))
    width = 0.25

    plt.figure(figsize=(10, 6))
    plt.bar(x - width, precision, width, label="Precision")
    plt.bar(x, recall, width, label="Recall")
    plt.bar(x + width, f1, width, label="F1-score")

    plt.xticks(x, class_names, rotation=20)
    plt.ylim(0, 1.0)
    plt.ylabel("Score")
    plt.title(title)
    plt.legend()
    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.close()

def evaluate_split(name, model, X_split, y_split):
```

```
y_pred = model.predict(X_split)

print(f"\n=== {name} ===")
print("Accuracy:", round(accuracy_score(y_split, y_pred), 4))
print(classification_report(y_split, y_pred, target_names=CLASS_NAMES))
print("Confusion matrix:")
print(confusion_matrix(y_split, y_pred))

plot_and_save_confusion_matrix(
    y_split,
    y_pred,
    CLASS_NAMES,
    title=f"{name} Confusion Matrix",
    save_path=PLOTS_DIR / f"{name.lower()}_confusion_matrix.png"
)

plot_and_save_normalized_confusion_matrix(
    y_split,
    y_pred,
    CLASS_NAMES,
    title=f"{name} Normalized Confusion Matrix",
    save_path=PLOTS_DIR /
f"{name.lower()}_confusion_matrix_normalized.png"
)

plot_and_save_class_metrics(
    y_split,
    y_pred,
    CLASS_NAMES,
    save_path=PLOTS_DIR / f"{name.lower()}_class_metrics.png",
    title=f"{name} Per-Class Precision / Recall / F1"
)

evaluate_split("Validation", rf, X_val, y_val)
evaluate_split("Test", rf, X_test, y_test)

feature_labels = [f"img_{i}" for i in range(192)] + STATIC_COLS
importances = rf.feature_importances_

print("\n--- Top 15 most important features ---")
top_idx = np.argsort(importances)[::-1][:15]
for rank, i in enumerate(top_idx, start=1):
    print(f" {rank:2d}. {feature_labels[i]:30s} importance:
{importances[i]:.4f}")

plot_and_save_feature_importance(
    rf,
    feature_labels,
    save_path=PLOTS_DIR / "top_15_feature_importances.png",
    top_n=15
)

dump(
    {
        "model": rf,
```

```
        "class_names": CLASS_NAMES,
        "static_cols": STATIC_COLS,
        "feature_count": X.shape[1]
    },
    MODEL_PATH
)

print(f"\nModel saved to: {MODEL_PATH}")
print(f"Plots saved to: {PLOTS_DIR.resolve()}")
```

Appendix 4 – Application (MalwareScanner.py)

```
from pathlib import Path
import csv
import math
import shutil
from tkinter import filedialog, messagebox

import numpy as np
import customtkinter as ctk
from PIL import Image
from joblib import load

APP_TITLE = "Entropy Malware Scanner"
WINDOW_SIZE = "1180x760"

CHUNK_SIZE = 256
NUM_CHUNKS = 64
TOTAL_BYTES = CHUNK_SIZE * NUM_CHUNKS
IMAGE_SIZE = 8
DISPLAY_SIZE = 512
HIGH_ENTROPY_THRESHOLD = 7.0

GRID_SIZE = 8
CELL_SIZE = 42
CANVAS_SIZE = GRID_SIZE * CELL_SIZE

BINARY_MODEL_PATH =
Path(r"C:\Users\VMLicenta\Desktop\models\MvsB_train_plots.joblib")
MULTICLASS_MODEL_PATH =
Path(r"C:\Users\VMLicenta\Desktop\models\rf_train_plots.joblib")

CACHE_DIR = Path("cache")
CACHE_DIR.mkdir(exist_ok=True)

STATIC_COLS = [
    "log_file_size",
    "window_entropy",
    "mean_chunk_entropy",
    "std_chunk_entropy",
    "max_chunk_entropy",
    "min_chunk_entropy",
    "high_entropy_ratio",
]

BINARY_LABEL_MAP = {
    0: "malicious",
    1: "benign",
}

MULTI_LABEL_MAP = {
    0: "ransomware",
    1: "trojan",
    2: "rat",
    3: "worm",
}
```

```
4: "benign",
}

ctk.set_appearance_mode("light")
ctk.set_default_color_theme("blue")

def compute_entropy(data: bytes) -> float:
    if not data:
        return 0.0
    freq = [0] * 256
    for byte in data:
        freq[byte] += 1
    entropy = 0.0
    length = len(data)
    for count in freq:
        if count > 0:
            p = count / length
            entropy -= p * math.log2(p)
    return entropy

def shannon_entropy(byte_data: bytes) -> float:
    if not byte_data:
        return 0.0
    arr = np.frombuffer(byte_data, dtype=np.uint8)
    counts = np.bincount(arr, minlength=256)
    probs = counts[counts > 0] / len(arr)
    return float(-np.sum(probs * np.log2(probs)))

def entropy_to_rgb(normalized: float) -> tuple:
    v = max(0.0, min(1.0, normalized))
    if v <= 0.5:
        r = int(v * 2 * 255)
        g = 255
    else:
        r = 255
        g = int((1.0 - v) * 2 * 255)
    return (r, g, 0)

def rgb_to_hex(rgb):
    return "#{:02x}{:02x}{:02x}".format(int(rgb[0]), int(rgb[1]),
int(rgb[2]))

def extract_static_feature_dict(file_path: Path):
    raw = file_path.read_bytes()
    if len(raw) == 0:
        raise ValueError("Selected file is empty.")

    file_size = len(raw)
    window = raw[:TOTAL_BYTES]
    if len(window) < TOTAL_BYTES:
        window = window + bytes(TOTAL_BYTES - len(window))

    chunk_entropies = []
    for i in range(NUM_CHUNKS):
        start = i * CHUNK_SIZE
```

```
    end = start + CHUNK_SIZE
    chunk = window[start:end]
    ent = shannon_entropy(chunk)
    chunk_entropies.append(ent)

chunk_entropies = np.array(chunk_entropies, dtype=np.float32)

return {
    "file_name": file_path.name,
    "file_path": str(file_path),
    "file_size_bytes": file_size,
    "log_file_size": float(np.log1p(file_size)),
    "window_entropy": float(shannon_entropy(window)),
    "mean_chunk_entropy": float(np.mean(chunk_entropies)),
    "std_chunk_entropy": float(np.std(chunk_entropies)),
    "max_chunk_entropy": float(np.max(chunk_entropies)),
    "min_chunk_entropy": float(np.min(chunk_entropies)),
    "high_entropy_ratio": float(np.mean(chunk_entropies >
HIGH_ENTROPY_THRESHOLD)),
}

def build_entropy_png_exact(file_path: Path, output_png: Path):
    with open(file_path, "rb") as f:
        raw = f.read(TOTAL_BYTES)

    if len(raw) < TOTAL_BYTES:
        raw = raw + b"\x00" * (TOTAL_BYTES - len(raw))

    pixels = []
    for i in range(NUM_CHUNKS):
        chunk = raw[i * CHUNK_SIZE : (i + 1) * CHUNK_SIZE]
        entropy = compute_entropy(chunk)
        normalized = entropy / 8.0
        pixels.append(entropy_to_rgb(normalized))

    img = Image.new("RGB", (IMAGE_SIZE, IMAGE_SIZE))
    img.putdata(pixels)
    img_scaled = img.resize((DISPLAY_SIZE, DISPLAY_SIZE), Image.NEAREST)
    img_scaled.save(output_png)

def extract_199_features_exact(file_path: Path, cache_png_path: Path):
    build_entropy_png_exact(file_path, cache_png_path)

    img = Image.open(cache_png_path).convert("RGB")
    img = img.resize((8, 8))
    arr = np.array(img, dtype=np.float32)
    image_feat = arr.flatten()

    static_dict = extract_static_feature_dict(file_path)
    static_feat = np.array([static_dict[col] for col in STATIC_COLS],
dtype=np.float32)

    combined = np.hstack([image_feat, static_feat]).astype(np.float32)
    rgb_matrix = arr.astype(np.uint8).reshape(8, 8, 3)
    return combined, rgb_matrix, static_dict
```

```
def save_feature_csv(file_path: Path, static_dict: dict, combined_features:
np.ndarray):
    csv_path = CACHE_DIR / f"{file_path.stem}_features.csv"
    with open(csv_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow([f"img_{i}" for i in range(192)] + STATIC_COLS)
        writer.writerow(combined_features.tolist())
    return csv_path

def load_features_from_csv(csv_path: Path) -> np.ndarray:
    with open(csv_path, "r", encoding="utf-8") as f:
        reader = csv.reader(f)
        header = next(reader)
        row = next(reader)
        values = np.array([float(v) for v in row], dtype=np.float32).reshape(1,
-1)
    return values

class MalwareScannerApp(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title(APP_TITLE)
        self.geometry(WINDOW_SIZE)
        self.minsize(1050, 700)
        self.configure(fg_color="#f5f5f2")

        self.selected_file = None
        self.current_cache_png = None
        self.current_cache_csv = None

        self.binary_bundle = self._load_model_bundle(BINARY_MODEL_PATH)
        self.multi_bundle = self._load_model_bundle(MULTICLASS_MODEL_PATH)

        self.binary_model = self.binary_bundle["model"]
        self.multi_model = self.multi_bundle["model"]

        self.binary_classes = list(getattr(self.binary_model, "classes_",
self.binary_bundle.get("class_names", [0, 1])))
        self.multi_classes = list(getattr(self.multi_model, "classes_",
self.multi_bundle.get("class_names", [0, 1, 2, 3, 4])))

        print("Binary model classes:", self.binary_classes)
        print("Multi model classes :", self.multi_classes)

        self._build_ui()

    def _load_model_bundle(self, model_path: Path):
        if not model_path.exists():
            raise FileNotFoundError(f"Model not found:
{model_path.resolve()}")
        bundle = load(model_path)
        if not isinstance(bundle, dict) or "model" not in bundle:
            return {"model": bundle}
        return bundle
```



```
def _build_ui(self):
    self.grid_columnconfigure(0, weight=0)
    self.grid_columnconfigure(1, weight=1)
    self.grid_rowconfigure(0, weight=1)

    left = ctk.CTkFrame(self, width=330, corner_radius=24,
fg_color="#ffffff", border_width=1, border_color="#e6e3dc")
    left.grid(row=0, column=0, sticky="nsew", padx=(22, 12), pady=22)
    left.grid_propagate(False)

    right = ctk.CTkFrame(self, corner_radius=24, fg_color="#ffffff",
border_width=1, border_color="#e6e3dc")
    right.grid(row=0, column=1, sticky="nsew", padx=(12, 22), pady=22)

    title = ctk.CTkLabel(left, text="Entropy Scanner",
font=ctk.CTkFont(size=28, weight="bold"), text_color="#1f1f1b")
    title.pack(anchor="w", padx=24, pady=(28, 8))

    subtitle = ctk.CTkLabel(left, text="", font=ctk.CTkFont(size=14),
text_color="#6e6b63")
    subtitle.pack(anchor="w", padx=24, pady=(0, 26))

    select_box = ctk.CTkFrame(left, corner_radius=18,
fg_color="#f7f6f2", border_width=1, border_color="#e8e5dd")
    select_box.pack(fill="x", padx=24, pady=(0, 18))

    self.file_label = ctk.CTkLabel(select_box, text="No file selected",
justify="left", wraplength=250, font=ctk.CTkFont(size=14),
text_color="#2d2c28")
    self.file_label.pack(anchor="w", padx=18, pady=(18, 12))

    browse_btn = ctk.CTkButton(
        select_box,
        text="Select file",
        command=self.select_file,
        height=42,
        corner_radius=14,
        fg_color="#0f766e",
        hover_color="#115e59",
        text_color="#ffffff",
        font=ctk.CTkFont(size=14, weight="bold")
    )
    browse_btn.pack(fill="x", padx=18, pady=(0, 18))

    self.check_btn = ctk.CTkButton(
        left,
        text="CHECK",
        command=self.run_scan,
        height=48,
        corner_radius=16,
        fg_color="#1f1f1b",
        hover_color="#34322d",
        text_color="#ffffff",
        font=ctk.CTkFont(size=15, weight="bold")
    )
```

```
)
self.check_btn.pack(fill="x", padx=24, pady=(4, 10))

self.delete_file_btn = ctk.CTkButton(
    left,
    text="Delete scanned file",
    command=self.delete_scanned_file,
    height=42,
    corner_radius=14,
    fg_color="#b42318",
    hover_color="#8f1d14",
    text_color="#ffffff",
    font=ctk.CTkFont(size=14, weight="bold")
)
self.delete_file_btn.pack(fill="x", padx=24, pady=(0, 10))

self.clear_cache_btn = ctk.CTkButton(
    left,
    text="Clear cache folder",
    command=self.clear_cache_folder,
    height=42,
    corner_radius=14,
    fg_color="#7c3f00",
    hover_color="#5f3000",
    text_color="#ffffff",
    font=ctk.CTkFont(size=14, weight="bold")
)
self.clear_cache_btn.pack(fill="x", padx=24, pady=(0, 10))

self.status_label = ctk.CTkLabel(left, text="Ready.",
    justify="left", wraplength=260, font=ctk.CTkFont(size=13),
    text_color="#6e6b63")
self.status_label.pack(anchor="w", padx=24)

info = ctk.CTkTextbox(left, width=280, height=220, corner_radius=18,
    fg_color="#f7f6f2", border_width=1, border_color="#e8e5dd",
    text_color="#4a473f", font=ctk.CTkFont(size=13))
info.pack(fill="both", expand=True, padx=24, pady=(22, 24))
info.insert(
    "1.0",
    "Workflow:\n\n"
    "1. Select an executable.\n"
    "2. The app creates the entropy PNG. \n"
    "3. The app extracts the 199 features.\n"
    "4. It saves the PNG and a CSV in cache/.\n"
    "5. The models classify using the cached CSV row."
)
info.configure(state="disabled")

preview_title = ctk.CTkLabel(right, text="Computed entropy matrix",
    font=ctk.CTkFont(size=24, weight="bold"), text_color="#1f1f1b")
preview_title.pack(anchor="w", padx=28, pady=(28, 6))
```

```
        preview_sub = ctk.CTkLabel(right, text="8x8 preview reconstructed
from the training-aligned cached PNG.", font=ctk.CTkFont(size=13),
text_color="#6e6b63")
        preview_sub.pack(anchor="w", padx=28, pady=(0, 20))

        self.preview_card = ctk.CTkFrame(right, corner_radius=20,
fg_color="#f7f6f2", border_width=1, border_color="#e8e5dd")
        self.preview_card.pack(fill="x", padx=28, pady=(0, 24))

        self.preview_canvas = ctk.CTkCanvas(
            self.preview_card,
            width=CANVAS_SIZE,
            height=CANVAS_SIZE,
            bg="#f7f6f2",
            highlightthickness=0,
            bd=0
        )
        self.preview_canvas.pack(padx=24, pady=24)
        self._draw_placeholder_canvas()

        results_title = ctk.CTkLabel(right, text="Scan result",
font=ctk.CTkFont(size=24, weight="bold"), text_color="#1f1f1b")
        results_title.pack(anchor="w", padx=28, pady=(0, 12))

        self.binary_result = self._result_block(right, "Malware vs benign")
        self.binary_result.pack(fill="x", padx=28, pady=(0, 14))

        self.multi_result = self._result_block(right, "Type / benign")
        self.multi_result.pack(fill="x", padx=28, pady=(0, 28))

        def _draw_placeholder_canvas(self):
            self.preview_canvas.delete("all")
            self.preview_canvas.create_rectangle(0, 0, CANVAS_SIZE, CANVAS_SIZE,
fill="#f1f1f1", outline="#e5e5e5")
            self.preview_canvas.create_text(CANVAS_SIZE / 2, CANVAS_SIZE / 2,
text="No preview yet", fill="#8a8a8a", font=("Segoe UI", 14))

        def _draw_entropy_matrix(self, rgb_matrix: np.ndarray):
            self.preview_canvas.delete("all")
            pad = 2
            for row in range(GRID_SIZE):
                for col in range(GRID_SIZE):
                    x0 = col * CELL_SIZE
                    y0 = row * CELL_SIZE
                    x1 = x0 + CELL_SIZE
                    y1 = y0 + CELL_SIZE
                    color = rgb_to_hex(rgb_matrix[row, col])
                    self.preview_canvas.create_rectangle(
                        x0 + pad,
                        y0 + pad,
                        x1 - pad,
                        y1 - pad,
                        fill=color,
                        outline="#f7f6f2",
                        width=1
```

```
)

def _result_block(self, parent, title_text):
    block = ctk.CTkFrame(parent, corner_radius=18, fg_color="#f7f6f2",
border_width=1, border_color="#e8e5dd")
    title = ctk.CTkLabel(block, text=title_text,
font=ctk.CTkFont(size=14, weight="bold"), text_color="#4a473f")
    title.pack(anchor="w", padx=18, pady=(16, 4))

    result = ctk.CTkLabel(block, text="-", font=ctk.CTkFont(size=26,
weight="bold"), text_color="#1f1f1b")
    result.pack(anchor="w", padx=18, pady=(2, 4))

    confidence = ctk.CTkLabel(block, text="Confidence: -",
font=ctk.CTkFont(size=14), text_color="#6e6b63")
    confidence.pack(anchor="w", padx=18, pady=(0, 16))

    block.result_label = result
    block.conf_label = confidence
    return block

def delete_scanned_file(self):
    if self.selected_file is None:
        self.status_label.configure(text="No scanned file selected.")
        return

    if not self.selected_file.exists():
        self.status_label.configure(text="Selected file no longer
exists.")
        return

    confirm = messagebox.askyesno(
        "Delete scanned file",
        f"Permanently delete this file?\n\n{self.selected_file}"
    )
    if not confirm:
        return

    try:
        self.selected_file.unlink()
        deleted_name = self.selected_file.name
        self.selected_file = None
        self.file_label.configure(text="No file selected")
        self._draw_placeholder_canvas()
        self.binary_result.result_label.configure(text="-",
text_color="#1f1f1b")
        self.binary_result.conf_label.configure(text="Confidence: -")
        self.multi_result.result_label.configure(text="-",
text_color="#1f1f1b")
        self.multi_result.conf_label.configure(text="Confidence: -")
        self.status_label.configure(text=f"Deleted scanned file:
{deleted_name}")
    except Exception as e:
        self.status_label.configure(text=f"Error deleting file: {e}")
```

```
def clear_cache_folder(self):
    if not CACHE_DIR.exists():
        self.status_label.configure(text="Cache folder does not exist.")
        return

    confirm = messagebox.askyesno(
        "Clear cache folder",
        f"Delete everything inside the cache
folder?\n\n{CACHE_DIR.resolve()}"
    )
    if not confirm:
        return

    deleted_count = 0
    try:
        for item in CACHE_DIR.iterdir():
            if item.is_file() or item.is_symlink():
                item.unlink()
                deleted_count += 1
            elif item.is_dir():
                shutil.rmtree(item)
                deleted_count += 1

        self.current_cache_png = None
        self.current_cache_csv = None
        self.status_label.configure(text=f"Cache cleared. Removed
{deleted_count} item(s).")
    except Exception as e:
        self.status_label.configure(text=f"Error clearing cache: {e}")

def select_file(self):
    file_path = filedialog.askopenfilename(title="Select file to scan")
    if not file_path:
        return
    self.selected_file = Path(file_path)
    self.file_label.configure(text=str(self.selected_file))
    self.status_label.configure(text="File selected. Press CHECK to
create exact cached artifacts and scan.")

def run_scan(self):
    if self.selected_file is None:
        self.status_label.configure(text="Please select a file first.")
        return

    try:
        self.current_cache_png = CACHE_DIR /
f"{self.selected_file.stem}_entropy.png"
        combined_features, rgb_matrix, static_dict =
extract_199_features_exact(self.selected_file, self.current_cache_png)
        self.current_cache_csv = save_feature_csv(self.selected_file,
static_dict, combined_features)

        features_for_models =
load_features_from_csv(self.current_cache_csv)
```

```
        print("Features shape from cache:", features_for_models.shape)

        binary_pred_class =
self.binary_model.predict(features_for_models)[0]
        binary_probs =
self.binary_model.predict_proba(features_for_models)[0]
        binary_prob_map = dict(zip(self.binary_classes, binary_probs))
        binary_conf = float(binary_prob_map[binary_pred_class]) * 100.0
        binary_label = BINARY_LABEL_MAP.get(int(binary_pred_class),
str(binary_pred_class))

        multi_pred_class =
self.multi_model.predict(features_for_models)[0]
        multi_probs =
self.multi_model.predict_proba(features_for_models)[0]
        multi_prob_map = dict(zip(self.multi_classes, multi_probs))
        multi_conf = float(multi_prob_map[multi_pred_class]) * 100.0
        multi_label = MULTI_LABEL_MAP.get(int(multi_pred_class),
str(multi_pred_class))

        self._draw_entropy_matrix(rgb_matrix)
        self._update_result_card(self.binary_result, binary_label,
binary_conf)
        self._update_result_card(self.multi_result, multi_label,
multi_conf)

        self.status_label.configure(text=f"Scan finished. Exact PNG +
CSV saved in cache/ for {self.selected_file.name}.")
        except Exception as e:
            self.status_label.configure(text=f"Error: {e}")
            print("[ERROR]", e)

    def _update_result_card(self, card, label_text, confidence):
        pretty = str(label_text).capitalize()
        color = "#b42318" if pretty.lower() in {"malicious", "ransomware",
"trojan", "rat", "worm"} else "#166534"
        card.result_label.configure(text=pretty, text_color=color)
        card.conf_label.configure(text=f"Confidence: {confidence:.2f}%")

if __name__ == "__main__":
    app = MalwareScannerApp()
    app.mainloop()
```